

# Unidade II

## 3 TIPOS DE DADOS E VARIÁVEIS

A linguagem de programação Python oferece um conjunto de ferramentas para trabalhar com tipos básicos de dados, como números e strings, assim como mecanismos eficientes para definir e manipular variáveis e operadores. Esses recursos formam a base de qualquer programa em Python, permitindo desde cálculos matemáticos simples até operações complexas de manipulação de texto e lógica.

Os **tipos numéricos** em Python incluem principalmente os inteiros, números de ponto flutuante e números complexos. Inteiros representam números sem casas decimais, enquanto números de ponto flutuante são usados para indicar valores com partes fracionárias. Números complexos, menos comuns no dia a dia da maioria dos programadores, permitem trabalhar com cálculos que envolvem uma parte real e uma imaginária. Python oferece **operadores aritméticos**, como adição, subtração, multiplicação e divisão, para manipular esses números. Além disso, operações como potência e módulo estão disponíveis, garantindo flexibilidade para cálculos matemáticos sofisticados.

As strings, por outro lado, representam **sequências de caracteres** e são amplamente utilizadas para manipulação textual. Python torna essa tarefa particularmente intuitiva, permitindo concatenar strings com o operador `+`, repetir textos com o operador `*` e acessar caracteres individuais ou substrings usando índices e divisões. Métodos embutidos, como `upper()`, `lower()`, `replace()` e `split()`, enriquecem as possibilidades, facilitando a modificação e análise de textos. A manipulação de strings em Python é complementada pela sua capacidade de formatar textos dinamicamente, utilizando métodos como f-strings e `.format()`, que permitem a inserção de valores diretamente dentro de um texto.

**Variáveis** são os contêineres fundamentais usados para armazenar esses dados em Python. A sintaxe simples de **atribuição** com o operador `=` facilita a criação de variáveis. Por exemplo, `x = 10` armazena o valor inteiro 10 na variável `x`. As variáveis não precisam ter seu tipo declarado explicitamente, já que Python realiza a inferência de tipo automaticamente, proporcionando maior simplicidade. Além disso, variáveis podem ser reatribuídas a valores de diferentes tipos ao longo da execução de um programa, característica que reflete a **tipagem dinâmica** de Python.

Os operadores desempenham um papel crucial na manipulação de dados e no controle lógico de um programa. Como já acentuamos anteriormente, operadores lógicos, como `and`, `or` e `not`, são utilizados para avaliar expressões booleanas, sendo essenciais para implementar condições e loops. Por exemplo, a expressão `x > 5 and y < 10` avalia se as duas condições são verdadeiras simultaneamente, enquanto o operador `or` retorna verdadeiro se pelo menos uma das condições for satisfeita. Python também inclui **operadores relacionais**, como `==`, `!=`, `>`, `<`, `>=` e `<=`, para comparar valores, além de operadores **de atribuição composta**, como `+=` e `*=`, que tornam a atualização de valores mais compacta e legível.

A combinação desses recursos – tipos básicos de dados, variáveis e operadores – permite construir programas desde os mais simples até os mais complexos em Python.

## 3.1 Tipos numéricos e strings

Um **objeto**, no contexto de programação, pode ser entendido como algo que combina **dados** com as **ações** que podem ser realizadas sobre esses dados. Para facilitar a compreensão, imagine que um objeto é como um "pacote" ou uma "coisa" que guarda informações e sabe como trabalhar com elas.

Por exemplo, pense em um carro no mundo real. Um carro tem **características** como cor, modelo, ano de fabricação e velocidade atual. Essas características são os dados do carro. Além disso, um carro pode **fazer coisas**, como acelerar, frear e buzinar. Essas ações correspondem às funcionalidades que o carro oferece.

Na programação, um objeto funciona de forma parecida. Ele tem:

- dados (chamados de **atributos**) que descrevem o objeto;
- ações (chamadas de **métodos**) que o objeto pode realizar ou que podem ser realizadas sobre ele.

Um número inteiro em Python é também um objeto: ele tem atributos e métodos. O dado é o valor do número, como 10, e ele pode realizar ações, como verificar quantos bits são necessários para representá-lo em binário usando o método `.bit_length()`.



### Observação

Os números são representados em binário usando apenas os dígitos 0 e 1, que são as "palavras" do sistema numérico **base 2**. Cada posição em um número binário representa uma potência de 2, de forma similar à forma como as posições em um número decimal expressam potências de 10.

Por exemplo, o número 10 no **sistema decimal** (base 10) pode ser indicado como 1010 no **sistema binário**, pois:

$$1010_{\text{bin}} = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) = 8 + 0 + 2 + 0 = 10_{\text{dec}}$$

A representação binária é crucial para computadores, pois eles funcionam com circuitos eletrônicos que reconhecem apenas dois estados: ligado (1) e desligado (0). Cada número decimal que fornecemos ao computador é convertido internamente para binário para processamento. Por exemplo, operações matemáticas, comparações e armazenamento de dados dependem dessa representação.

Em Python, objetos são a base de tudo. Até mesmo coisas que parecem simples, como números e textos, são objetos. Eles não são valores passivos: têm características e funcionalidades que podem ser acessadas e manipuladas.

E os objetos não existem isoladamente; eles seguem um modelo definido por estruturas chamadas classes. Uma **classe** pode ser entendida como uma espécie de "molde", que define as características e os comportamentos que os objetos daquele tipo devem ter. Isso significa que, quando você trabalha com um objeto em Python, como um número ou um texto, está, na verdade, lidando com uma **instância** de uma classe específica que estabelece quais dados esse objeto pode armazenar e quais ações ele pode realizar.

**Um objeto é um representante de uma classe**, assim como o automóvel Jaguar F-Pace de placa PQR2H16 é um representante (ou instância) da classe "carro".

O tipo inteiro em Python, representado pela classe `int`, é um dos tipos básicos de dados mais fundamentais e amplamente utilizados. Inteiros são números **sem casas decimais**, podendo ser positivos, negativos ou zero. A manipulação de números inteiros é essencial em diversas aplicações, desde cálculos matemáticos simples até operações complexas envolvendo algoritmos e estruturas de dados.

Eles podem representar valores de forma precisa, independentemente de sua magnitude. Diferentemente de muitas linguagens de programação, nas quais os inteiros são limitados por um número de bits fixo (como 32 ou 64 bits), em Python os inteiros têm precisão arbitrária. Isso significa que **não há limites práticos para o tamanho de um inteiro, exceto pela memória disponível no sistema**. Essa característica é particularmente útil em contextos que requerem cálculos matemáticos exatos com números extremamente grandes, como criptografia ou processamento de grandes conjuntos de dados numéricos.



### Lembrete

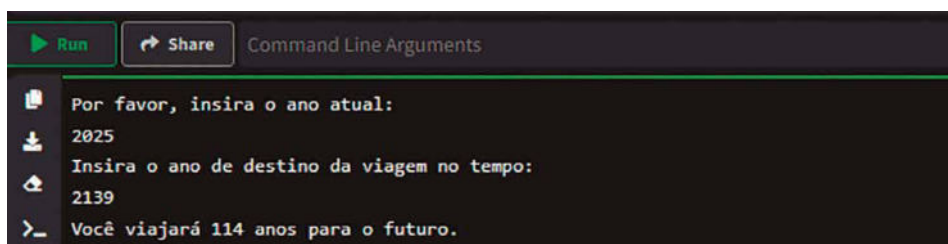
As strings representam sequências de caracteres e são amplamente usadas para manipulação textual.

Considere, a título de comparação, uma linguagem de programação muito popular chamada Java. Ela tem uma limitação de tamanho para um número (chamado tipo `long`) em 64 bits. O `long` pode representar valores entre:

$$-2^{63} \text{ até } 2^{63} - 1$$

O termo **googol** refere-se ao número 10 elevado a 100. Um número extremamente grande que não pode ser armazenado com o tipo `long` em Java, pois, como acabamos de ver, ele só comporta até o número 2 elevado a 63.





```
Run Share Command Line Arguments
Por favor, insira o ano atual:
2025
Insira o ano de destino da viagem no tempo:
2139
>_ Você viajará 114 anos para o futuro.
```

Figura 48 – Calculando a distância temporal (em anos). Nessa execução, a viagem é para o passado

O programa começa solicitando que o usuário insira o ano atual, utilizando a função `input()`. A entrada (string, por padrão) é convertida para um número inteiro com `int()` e armazenada na variável `ano_atual`. Em seguida, o programa solicita o ano de destino, que também é convertido para inteiro e armazenado na variável `ano_destino`.

A diferença em anos entre o ano de destino e o ano atual é calculada subtraindo-se `ano_atual` de `ano_destino`. O resultado é armazenado na variável `diferenca_anos`. Essa diferença indica quantos anos o viajante percorrerá no tempo. O programa usa estruturas condicionais para determinar se a viagem é para o futuro, passado ou se o viajante permanecerá no ano atual:

- Se `diferenca_anos` for maior que zero, significa que o ano de destino é posterior ao ano atual, indicando uma viagem para o futuro. O programa imprime uma mensagem informando quantos anos no futuro o viajante irá.
- Se `diferenca_anos` for menor que zero, o ano de destino é anterior ao ano atual, indicando uma viagem para o passado. Usa-se `abs(diferenca_anos)` para obter o valor absoluto da diferença (tornando-o positivo) ao informar a quantidade de anos. O programa imprime uma mensagem indicando quantos anos no passado o viajante percorrerá.
- Se `diferenca_anos` for igual a zero, o ano de destino é o mesmo que o ano atual, ou seja, não haverá viagem no tempo. O programa informa que o viajante permanecerá no ano atual.

Python oferece uma gama completa de operações aritméticas para o tipo inteiro. Operadores como adição (+), subtração (-), multiplicação (\*) e divisão inteira (/ /) permitem realizar cálculos com facilidade. A divisão inteira é especialmente útil quando o programador precisa do quociente de uma divisão sem considerar o restante decimal. Além disso, o operador de módulo (%) retorna o resto de uma divisão, enquanto o operador de exponenciação (\*\*) permite elevar um número a uma potência. A tabela a seguir mostra alguns exemplos da utilização desses operadores:

**Tabela 1 – Exemplos com operadores aritméticos para inteiros**

| Operador | Descrição       | Exemplo                        |
|----------|-----------------|--------------------------------|
| +        | Adição          | print(5 + 3) #<br>Saída: 8     |
| +        | Adição          | print(-2 + 7) #<br>Saída: 5    |
| +        | Adição          | print(0 + 10)<br># Saída: 10   |
| -        | Subtração       | print(5 - 3) #<br>Saída: 2     |
| -        | Subtração       | print(-2 - 7) #<br>Saída: -9   |
| -        | Subtração       | print(10 - 0) #<br>Saída: 10   |
| *        | Multiplicação   | print(5 * 3) #<br>Saída: 15    |
| *        | Multiplicação   | print(-2 * 7) #<br>Saída: -14  |
| *        | Multiplicação   | print(0 * 10) #<br>Saída: 0    |
| //       | Divisão inteira | print(10 // 3)<br># Saída: 3   |
| //       | Divisão inteira | print(-10 // 3)<br># Saída: -4 |
| //       | Divisão inteira | print(0 // 5) #<br>Saída: 0    |
| %        | Módulo          | print(10 % 3)<br># Saída: 1    |
| %        | Módulo          | print(-10 % 3)<br># Saída: 2   |
| %        | Módulo          | print(10 % 5)<br># Saída: 0    |
| **       | Exponenciação   | print(2 ** 3) #<br>Saída: 8    |
| **       | Exponenciação   | print(-2 ** 3)<br># Saída: -8  |
| **       | Exponenciação   | print(10 ** 0)<br># Saída: 1   |

Outra funcionalidade importante associada aos inteiros em Python é a possibilidade de realizar operações **bit a bit**, que operam diretamente sobre a representação binária dos números. Operadores como & (AND), | (OR), ^ (XOR), ~ (NOT), assim como deslocamentos para a esquerda (<<) e para a direita (>>), são amplamente utilizados em áreas como otimização de algoritmos, computação gráfica e processamento de dados binários. Por exemplo, o deslocamento à esquerda  $x \ll 1$  multiplica o número por 2, enquanto o deslocamento à direita  $x \gg 1$  o divide por 2, truncando valores decimais.



### Saiba mais

Um bit é a menor unidade de informação usada em computadores e sistemas digitais. Ele representa um único valor, que pode ser **0** ou **1**. Esses dois valores correspondem a dois estados diferentes, como "ligado" e "desligado", "verdadeiro" e "falso" ou "sim" e "não".

O livro indicado a seguir é essencial para quem deseja entender como os bits são manipulados no nível do hardware e como isso se reflete no software. Ele cobre operações lógicas, sistemas numéricos binários e como esses conceitos são usados em arquiteturas de computadores. É um excelente complemento para a prática com Python em manipulação de bits.

PATTERSON, D.; HENNESSY, J. *Computer organization and design RISC-V edition: the hardware software interface*. Burlington: Morgan Kaufmann Publishers, 2020.

Para entender melhor, imagine um interruptor de luz que só pode estar em uma das duas posições: ligado (1) ou desligado (0). O bit funciona de forma semelhante, sendo usado para armazenar ou transmitir informações em sistemas eletrônicos.

Os computadores utilizam muitos bits ao mesmo tempo, agrupando-os para formar unidades maiores de dados, como **bytes** (um conjunto de 8 bits), que permitem representar letras, números, imagens e até vídeos. Assim, mesmo sendo uma unidade simples, o bit é essencial para o funcionamento de toda a tecnologia digital que usamos no dia a dia.

Os métodos embutidos da classe `int` também enriquecem o trabalho com inteiros. O método `bit_length()`, por exemplo, retorna o número de bits necessários para representar o número em binário, excluindo o sinal. Ele é útil em situações nas quais a manipulação direta do formato binário é necessária. Já a função `int()` permite criar um número inteiro a partir de uma string ou de outro tipo numérico, com a capacidade de especificar uma base numérica, como binário, octal ou hexadecimal. Por exemplo, `int('101', 2)` converte a string binária 101 no número inteiro 5.

Além das operações diretas, os inteiros são frequentemente usados em controle de fluxo e iteração. Estruturas como loops `for` e `while` dependem de inteiros para definir contadores ou condições. Em um loop `for`, por exemplo, a função embutida `range()` gera uma sequência de inteiros que é usada para controlar as iterações, oferecendo flexibilidade para definir intervalos, passos e limites.

É importante considerar também que a precisão dos inteiros em Python vem acompanhada de um custo de desempenho em comparação com inteiros de tamanho fixo em outras linguagens. Embora esse impacto seja geralmente insignificante para a maioria das aplicações, pode ser relevante em cenários de alto desempenho, nos quais o tempo de execução é crítico.



Já os números de **ponto flutuante** em Python representam valores numéricos **que têm partes decimais**, permitindo trabalhar com números que não podem ser expressos como inteiros. Eles são amplamente usados em cálculos que exigem precisão com valores fracionários, como medições, finanças e simulações científicas. Em Python, números de ponto flutuante são tratados como objetos da classe `float`, sendo armazenados internamente no formato binário de ponto flutuante conforme o padrão IEEE 754, amplamente adotado em sistemas computacionais.

A principal característica dos números de ponto flutuante é sua capacidade de representar uma ampla gama de valores, desde números extremamente pequenos até números muito grandes, usando uma notação científica. Por exemplo, o número 0.000000123 pode ser indicado como  $1.23 \times 10^{-7}$ , o que em Python seria escrito como `1.23e-7`. Esse formato é eficiente, pois reduz o consumo de memória e facilita operações matemáticas em valores que variam significativamente em escala.

O termo **ponto flutuante** refere-se à forma como os números são representados no computador, utilizando um formato que permite que o ponto decimal (ou "ponto binário", no caso da base 2) "flutue" dentro do número, dependendo de sua magnitude. Essa abordagem é semelhante à notação científica usada na matemática, na qual os números são expressos como o produto de uma base (ou mantissa) e uma potência de 10. Por exemplo,  $1.23 \times 10^4$  representa 12300 e  $1.23 \times 10^{-3}$  indica 0.00123. Da mesma forma, em sistemas binários, o ponto decimal flutua para permitir a representação de números grandes ou muito pequenos com eficiência.

O formato de ponto flutuante divide o número em três partes: o sinal (indicando positivo ou negativo), a mantissa (ou fração, que contém os dígitos significativos) e o expoente (que ajusta a posição do ponto decimal). Essa estrutura é o que permite ao ponto "flutuar", dependendo do valor representado.

O padrão IEEE 754 é um conjunto de regras amplamente adotado para representar e manipular números de ponto flutuante em sistemas computacionais. Ele foi desenvolvido pelo Institute of Electrical and Electronics Engineers (IEEE) para garantir consistência e precisão nas operações numéricas entre diferentes hardwares e softwares. A padronização facilita a interoperabilidade entre sistemas, pois garante que os cálculos produzam os mesmos resultados, independentemente da plataforma.

No padrão IEEE 754, o formato mais comum para ponto flutuante é o de 64 bits (também conhecido como precisão dupla, ou *double precision*). Esses 64 bits são divididos da seguinte maneira:

- **1 bit para o sinal:** indica se o número é positivo (0) ou negativo (1);
- **11 bits para o expoente:** determina a posição do ponto decimal, ajustando a escala do número;
- **52 bits para a mantissa:** armazena os dígitos significativos do número.





### Observação

Apesar de sua eficiência e aplicabilidade, o padrão IEEE 754 tem limitações, especialmente em relação à precisão. Alguns números (como  $0.1$ ) não podem ser representados de forma exata no sistema binário, resultando em pequenas imprecisões que podem se acumular em cálculos complexos. Por exemplo, ao somar  $0.1 + 0.2$ , o resultado pode ser algo como  $0.30000000000000004$  em vez de exatamente  $0.3$ . Para mitigar esses problemas, em casos nos quais precisão absoluta é necessária, pode-se recorrer a bibliotecas como `decimal` em Python, que não seguem o padrão IEEE 754, mas priorizam a precisão.

Importante frisar que no Brasil utilizamos a **vírgula** para separar a parte decimal dos números. Por exemplo, o número  $1,5$  é lido em nosso país como "um-vírgula-cinco" ou "um e meio". Python usa o **ponto** como **separador decimal** porque segue a convenção internacional estabelecida pela norma ISO 80000-1, que é amplamente adotada em linguagens de programação e sistemas computacionais. Ou seja, em Python o número anterior será  $1.5$  ("um-ponto-cinco" ou "um e meio"). Essa escolha foi motivada por razões práticas e históricas que visam garantir consistência e compatibilidade entre diferentes plataformas e culturas.



### Lembrete

A principal razão para essa utilização é que o ponto decimal é o separador usado na notação matemática nos países de língua inglesa, que têm forte influência no desenvolvimento de tecnologias e padrões computacionais. Muitas das primeiras linguagens de programação, como Fortran, C e posteriormente Python, foram projetadas em ambientes nos quais o ponto era amplamente usado para números fracionários. Desse modo, adotar o ponto como separador tornou-se uma convenção prática para simplificar a implementação e garantir que a notação fosse compreendida globalmente.

Além disso, o uso do ponto evita ambiguidade em contextos nos quais a vírgula tem outros significados. Em Python, por exemplo, a vírgula é utilizada como separador de itens em listas, tuplas e argumentos de funções.

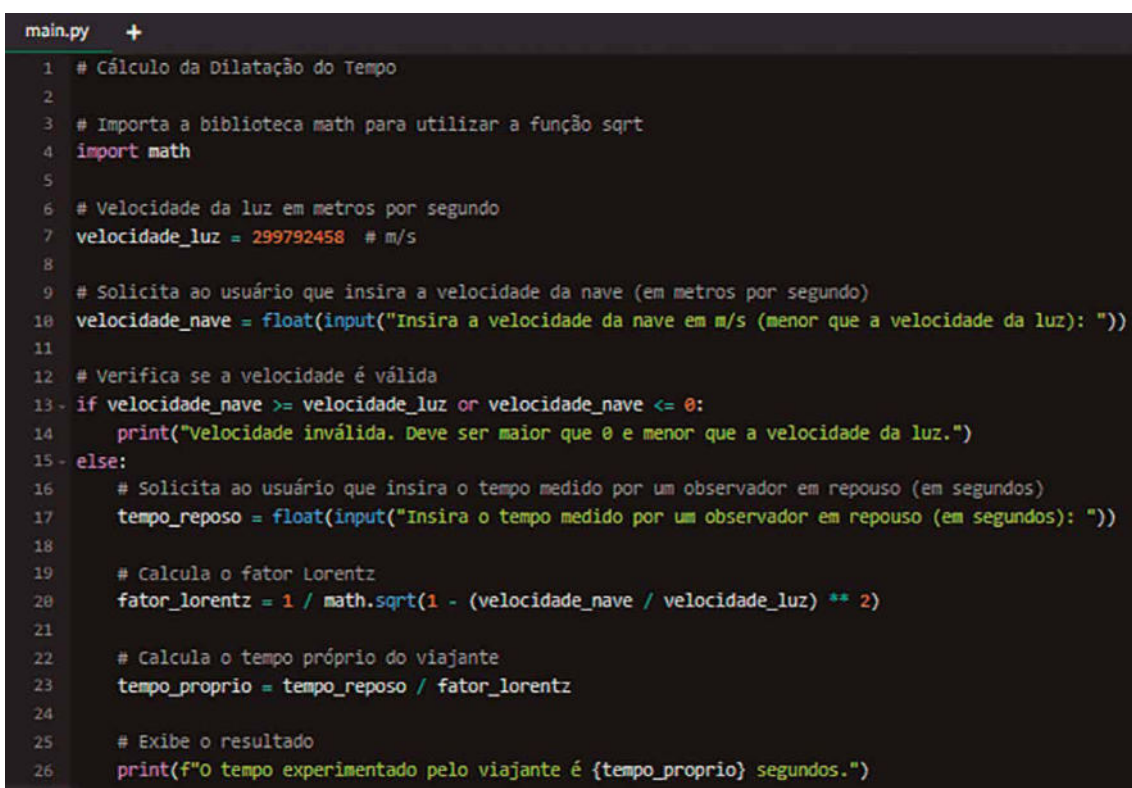
Embora em muitas culturas, como no Brasil e em vários países europeus, a vírgula seja o separador decimal padrão no contexto cotidiano, Python adota uma abordagem unificada e global ao seguir o uso do ponto, promovendo consistência entre linguagens de programação e sistemas computacionais. Em aplicações que exigem compatibilidade com convenções locais, como em softwares voltados para usuários finais, é possível formatar números usando a vírgula, convertendo-os para strings com

bibliotecas ou configurações específicas. Por exemplo, em Python, a biblioteca `locale` pode ser usada para ajustar a formatação numérica conforme o padrão regional:

```
import locale
locale.setlocale(locale.LC_ALL, 'pt_BR.UTF-8') # Configura para o padrão brasileiro
print(locale.format_string('%.2f', 1234.56, grouping=True)) # Saída: 1.234,56
```

Figura 49

No universo das viagens temporais, compreender os efeitos da relatividade é essencial para trajetórias que envolvem velocidades próximas à da luz. De acordo com a teoria da relatividade especial de Albert Einstein, o tempo percebido por um observador em movimento pode ser diferente do tempo percebido por um observador em repouso. Esse fenômeno é conhecido como dilatação do tempo. Para um viajante que se desloca a uma fração significativa da velocidade da luz, o tempo passará de forma diferente em comparação com alguém que permanece em repouso. O programa da figura a seguir calcula o tempo que um viajante experimenta durante sua jornada, considerando a velocidade de viagem e a duração do tempo medido por um observador em repouso.

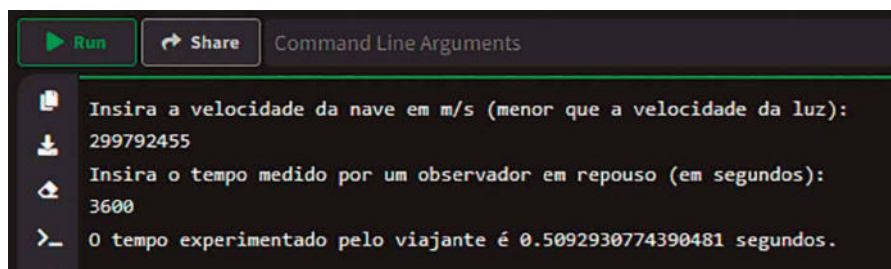


```
main.py +
1 # Cálculo da Dilatação do Tempo
2
3 # Importa a biblioteca math para utilizar a função sqrt
4 import math
5
6 # Velocidade da luz em metros por segundo
7 velocidade_luz = 299792458 # m/s
8
9 # Solicita ao usuário que insira a velocidade da nave (em metros por segundo)
10 velocidade_nave = float(input("Insira a velocidade da nave em m/s (menor que a velocidade da luz): "))
11
12 # Verifica se a velocidade é válida
13 if velocidade_nave >= velocidade_luz or velocidade_nave <= 0:
14     print("Velocidade inválida. Deve ser maior que 0 e menor que a velocidade da luz.")
15 else:
16     # Solicita ao usuário que insira o tempo medido por um observador em repouso (em segundos)
17     tempo_reposo = float(input("Insira o tempo medido por um observador em repouso (em segundos): "))
18
19     # Calcula o fator Lorentz
20     fator_lorentz = 1 / math.sqrt(1 - (velocidade_nave / velocidade_luz) ** 2)
21
22     # Calcula o tempo próprio do viajante
23     tempo_proprio = tempo_reposo / fator_lorentz
24
25     # Exibe o resultado
26     print(f"O tempo experimentado pelo viajante é {tempo_proprio} segundos.")
```

Figura 50 – Cálculo da dilatação do tempo aplicando a teoria da relatividade especial de Einstein

De forma simplificada, a dilatação do tempo ocorre quando um objeto se move a velocidades muito altas, próximas à velocidade da luz. Nessa situação, o tempo é percebido de maneira diferente para dois observadores: um que está em repouso e outro que está em movimento junto com o objeto. Essa diferença no fluxo do tempo é calculada pelo fator Lorentz, que considera a relação entre a velocidade do objeto e a velocidade da luz.

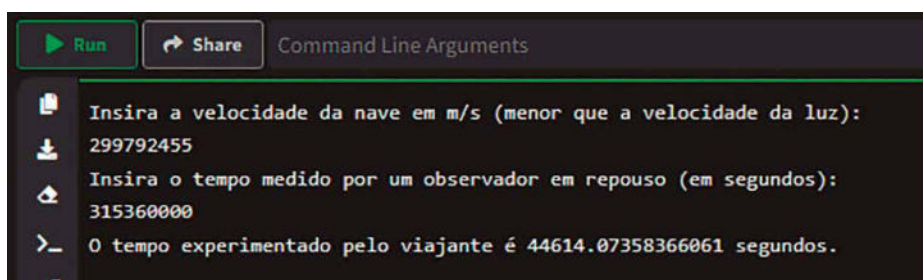
Na execução ilustrada na figura a seguir, a velocidade da nave inserida no programa foi de 299.792.455 m/s, um valor extremamente próximo da velocidade da luz, que é de 299.792.458 m/s. Essa velocidade quase máxima implica efeitos muito significativos de dilatação do tempo. O tempo medido por um observador em repouso foi de 3.600 segundos, ou seja, uma hora. No entanto, para o viajante dentro da nave, o tempo experimentado foi apenas cerca de 0,51 segundos. Essa discrepância ocorre porque, em velocidades tão elevadas, o tempo para o viajante parece "comprimir-se", passando muito mais devagar do que para o observador externo.



```
Run Share Command Line Arguments
Insira a velocidade da nave em m/s (menor que a velocidade da luz):
299792455
Insira o tempo medido por um observador em repouso (em segundos):
3600
> O tempo experimentado pelo viajante é 0.5092930774390481 segundos.
```

Figura 51 – Execução do programa que calcula a dilatação do tempo

Esse resultado ilustra de forma clara uma das consequências fundamentais da relatividade: quanto mais rápido um objeto se move, mais devagar o tempo passa para ele em relação a um referencial em repouso. Na figura a seguir fizemos uma nova execução do programa. Mantivemos a velocidade, mas mudamos o prazo do observador em repouso para 10 anos (315.360.000 segundos). Para o viajante esse tempo equivale a aproximadamente 12 horas.



```
Run Share Command Line Arguments
Insira a velocidade da nave em m/s (menor que a velocidade da luz):
299792455
Insira o tempo medido por um observador em repouso (em segundos):
315360000
> O tempo experimentado pelo viajante é 44614.07358366061 segundos.
```

Figura 52 – Execução do programa que calcula a dilatação do tempo. Um observador em repouso envelhece 10 anos, enquanto para nosso viajante não se passaram nem 13 horas

O programa da figura 50 inicia importando a biblioteca `math` (linha 4), que é necessária para utilizar a função `sqrt` (raiz quadrada) durante o cálculo. Definimos a constante `velocidade_luz` como 299.792.458 metros por segundo (linha 7), que é o valor aceito para a velocidade da luz no vácuo.

Na linha 10 é solicitado ao usuário inserir a velocidade da nave em metros por segundo, utilizando a função `input()`. A entrada é convertida para um número de ponto flutuante com `float()`, permitindo que sejam inseridos valores **não inteiros** e possibilitando maior precisão. A velocidade inserida é armazenada na variável `velocidade_nave`.

Antes de prosseguir com os cálculos, o programa verifica se a velocidade inserida é válida (linha 13). A velocidade da nave deve ser maior que zero e menor que a velocidade da luz. Se a condição `velocidade_nave >= velocidade_luz or velocidade_nave <= 0` for verdadeira, o programa informa que a velocidade é inválida e encerra a execução. Isso evita cálculos incorretos ou situações fisicamente impossíveis.

Caso a velocidade seja válida, o programa solicita que o usuário insira o tempo medido por um observador em repouso, também em segundos. Essa entrada é convertida para um número de ponto flutuante e armazenada na variável `tempo_reposo`.

O programa então calcula o fator de Lorentz (linha 20), que é essencial para determinar a dilatação do tempo. O termo `(velocidade_nave / velocidade_luz) ** 2` calcula o quadrado da razão entre a velocidade da nave e a velocidade da luz. A função `math.sqrt()` calcula a raiz quadrada. O resultado é invertido (1 dividido pelo resultado) para obter o fator de Lorentz.

Com o fator de Lorentz calculado, o programa determina na linha 23 o tempo próprio do viajante, que é o tempo experimentado pelo viajante na nave. Nesse caso, `tempo_reposo` é o tempo medido por um observador que está em repouso em relação ao referencial inicial (por exemplo, na Terra), enquanto `tempo_proprio` é o tempo que o viajante na nave espacial experimenta. Finalmente, o programa exibe o resultado utilizando a função `print()`, mostrando ao usuário o tempo que ele experimentará durante a viagem.

Trabalhar com números de ponto flutuante é essencial nesse tipo de cálculo, pois as velocidades e tempos envolvidos podem ser valores não inteiros e requerem precisão decimal. Ao realizar operações matemáticas com números de ponto flutuante, é importante considerar questões como precisão e arredondamento, pois operações sucessivas podem introduzir pequenas imprecisões devido à forma como esses números são representados em sistema binário.

Esse programa ilustrou como aplicar conceitos físicos avançados utilizando programação, permitindo que usuários explorem os efeitos da relatividade especial de forma interativa. Ao inserir diferentes valores para a velocidade da nave e o tempo em repouso, é possível observar como a dilatação do tempo se manifesta em diferentes cenários, aprofundando a compreensão desse fenômeno fascinante.

Para os físicos de plantão, convém mencionar que o programa não considera efeitos da aceleração ou da relatividade geral, que seriam relevantes em viagens espaciais reais. Além disso, as velocidades inseridas devem ser significativamente menores que a velocidade da luz para que os resultados sejam fisicamente plausíveis e seguros.

**Números complexos** são uma extensão dos números reais e são amplamente utilizados em matemática e física para resolver problemas que não podem ser tratados apenas com números reais. Um número complexo é composto por duas partes, a saber, uma real e uma imaginária, representadas na forma  $a + b*i$ , onde:

- $a$  é a parte real;
- $b$  é a parte imaginária;
- $i$  (ou  $j$  em Python) é a unidade imaginária, definida como  $\sqrt{-1}$ .



### Observação

A importância prática dos números complexos se manifesta em diversas áreas, como engenharia elétrica, processamento de sinais, mecânica quântica e análise de sistemas dinâmicos. Por exemplo, em circuitos elétricos, números complexos são usados para representar impedâncias e calcular correntes e tensões em sistemas de corrente alternada. Em física, eles são essenciais para descrever fenômenos ondulatórios e resolver equações diferenciais que surgem em sistemas complexos. Além disso, no processamento de sinais, a transformada de Fourier, uma ferramenta vital, opera naturalmente no domínio dos números complexos.

Em Python, os números complexos são **nativos**, o que significa que a linguagem oferece suporte direto para sua manipulação sem a necessidade de bibliotecas externas. Eles são representados na forma  $a + bj$ , onde  $a$  e  $b$  são números reais, e  $j$  é a unidade imaginária. Por exemplo, o número complexo  $3 + 4j$  pode ser criado diretamente em Python como: `z = 3 + 4j`. A linguagem fornece uma série de operações e métodos para trabalhar com números complexos. É possível realizar operações aritméticas básicas, como adição, subtração, multiplicação e divisão, diretamente entre números complexos ou entre um número complexo e um número real.

Além disso, Python permite acessar diretamente a parte real e a parte imaginária de um número complexo usando os atributos `real` e `imag`, respectivamente. Também é possível calcular o módulo (ou magnitude) do número, que representa sua distância da origem no plano complexo, utilizando a função `abs()`.

Imagine um viajante no tempo tentando mapear a posição de portais dimensionais em um plano hipotético, no qual cada portal é representado por um número complexo. O viajante começa em um ponto inicial e deseja aplicar uma série de deslocamentos e rotações para chegar ao portal de destino. Ao final, o programa exibirá a nova posição do viajante, sua distância da origem (magnitude) e o ângulo dessa posição em relação ao eixo  $x$ , ajudando a visualizar a localização final no plano complexo. O código a seguir implementa esse cenário:

```
import math

# Solicita ao usuário as coordenadas do ponto inicial

x_inicial = float(input("Insira a coordenada x do ponto inicial: "))
y_inicial = float(input("Insira a coordenada y do ponto inicial: "))

# Cria o número complexo representando o ponto inicial
ponto = complex(x_inicial, y_inicial)

print(f"\nPonto inicial: {ponto}")

# Solicita um deslocamento no plano (translação)
x_desloc = float(input("\nInsira o valor de deslocamento no eixo x: "))
y_desloc = float(input("Insira o valor de deslocamento no eixo y: "))

# Aplica o deslocamento ao ponto
ponto += complex(x_desloc, y_desloc)
print(f"Ponto após translação: {ponto}")

# Solicita um ângulo de rotação em graus
angulo_graus = float(input("\nInsira um ângulo de rotação em graus: "))
angulo_radianos = math.radians(angulo_graus)

# Cria um fator de rotação a partir do ângulo ( $e^{i\theta} = \cos\theta + i\sin\theta$ )
fator_rotacao = complex(math.cos(angulo_radianos), math.sin(angulo_radianos))

# Aplica a rotação ao ponto
ponto_rotacionado = ponto * fator_rotacao
print(f"Ponto após rotação de {angulo_graus}°: {ponto_rotacionado}")

# Solicita um fator de escala (multiplicará a distância do ponto à origem)
fator_escala = float(input("\nInsira um fator de escala (por exemplo, 2 para dobrar a distância): "))
ponto_final = ponto_rotacionado * fator_escala
print(f"Ponto após escalonamento: {ponto_final}")

# Calcula a magnitude (distância à origem) e o ângulo final do ponto
magnitude = abs(ponto_final)
angulo_final = math.degrees(math.atan2(ponto_final.imag, ponto_final.real))

print(f"\nPosição final do viajante no plano complexo: {ponto_final}")
print(f"Magnitude (distância à origem): {magnitude}")
print(f"Ângulo em relação ao eixo x: {angulo_final:.2f}°")
```

Figura 53

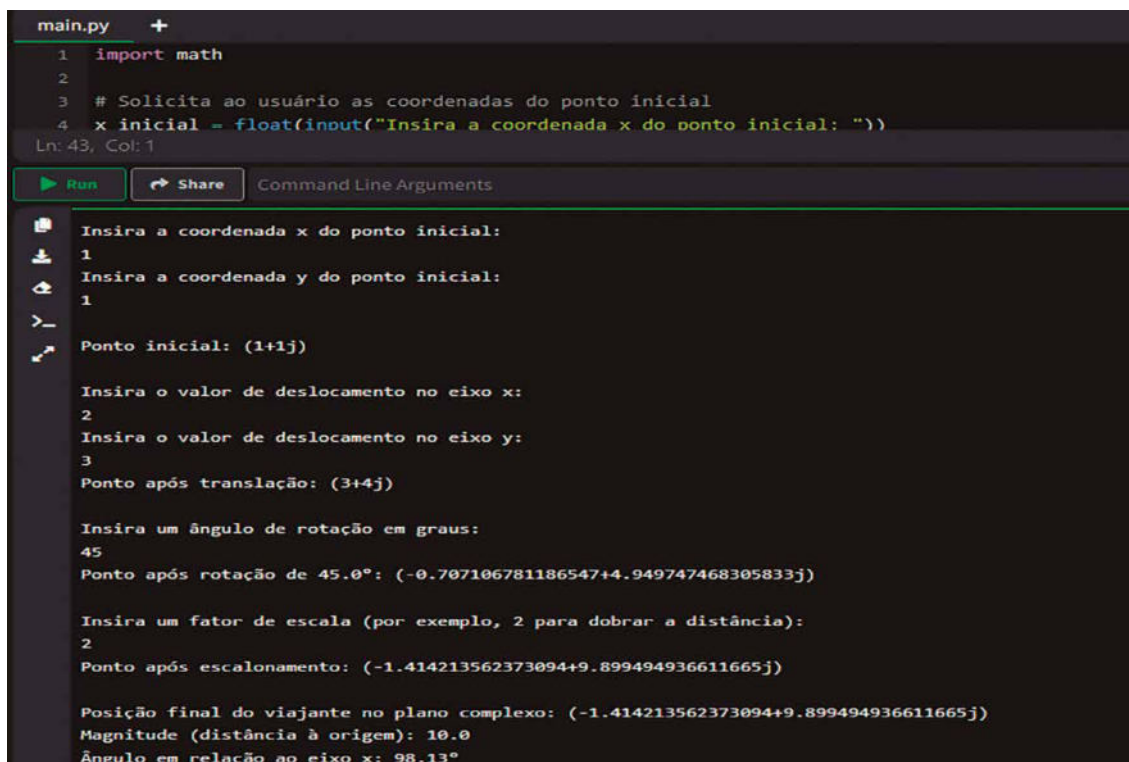
Esse programa começa solicitando ao usuário as coordenadas do ponto inicial, criando um número complexo cujas partes real e imaginária representam as coordenadas (x, y) do ponto no plano. A seguir, o código solicita um deslocamento (translação), adicionando outro número complexo ao ponto para mover sua posição.

Em seguida, o programa pede ao usuário um ângulo em graus para realizar uma rotação no plano. Essa rotação é implementada multiplicando o ponto por um fator de rotação  $e^{i\theta}$ , que é calculado a partir da função `math.cos()` e `math.sin()` sobre o ângulo convertido em radianos. Multiplicar



um número complexo por  $\cos\theta + i*\sin\theta$  resulta em uma rotação do ponto em torno da origem sem alterar sua magnitude inicial.

Após a rotação, o usuário define um fator de escala que multiplica o número complexo, afastando ou aproximando o ponto da origem. Por fim, o programa calcula a magnitude do ponto final usando `abs()`, que fornece a distância do ponto até a origem, e o ângulo final usando `math.atan2()`, que retorna o ângulo do vetor em relação ao eixo x. Um exemplo de execução do código está na figura a seguir.



```
main.py +
1 import math
2
3 # Solicita ao usuário as coordenadas do ponto inicial
4 x inicial = float(input("Insira a coordenada x do ponto inicial: "))
Ln: 43, Col: 1

Run Share Command Line Arguments

Insira a coordenada x do ponto inicial:
1
Insira a coordenada y do ponto inicial:
1
Ponto inicial: (1+1j)

Insira o valor de deslocamento no eixo x:
2
Insira o valor de deslocamento no eixo y:
3
Ponto após translação: (3+4j)

Insira um ângulo de rotação em graus:
45
Ponto após rotação de 45.0°: (-0.707106781186547+4.949747468305833j)

Insira um fator de escala (por exemplo, 2 para dobrar a distância):
2
Ponto após escalonamento: (-1.414213562373094+9.899494936611665j)

Posição final do viajante no plano complexo: (-1.414213562373094+9.899494936611665j)
Magnitude (distância à origem): 10.0
Ângulo em relação ao eixo x: 98.13°
```

Figura 54 – Execução do código que manipula números complexos para mapear a posição de portais dimensionais em um plano hipotético

Se você não tiver familiaridade com números complexos, esse exercício pode parecer confuso. Como o objetivo de aprimorar o entendimento, apresentaremos o gráfico (figura a seguir) que demonstra visualmente essas transformações no plano complexo. Ele mostra as etapas:

- **Ponto inicial** (azul): representa as coordenadas fornecidas inicialmente ( $x=1$  e  $y=1$ ).
- **Após deslocamento** (verde): o ponto foi movido no plano pelos valores de deslocamento fornecidos (2 para x e 3 para y).
- **Após rotação** (laranja): o ponto foi girado em torno da origem pelo ângulo especificado (45 graus).
- **Após escalonamento** (vermelho): o ponto final é ampliado ou reduzido conforme o fator de escala (escolhemos 2).



Cada vetor indica a posição relativa do ponto a partir da origem, ilustrando as transformações geométricas realizadas no plano.

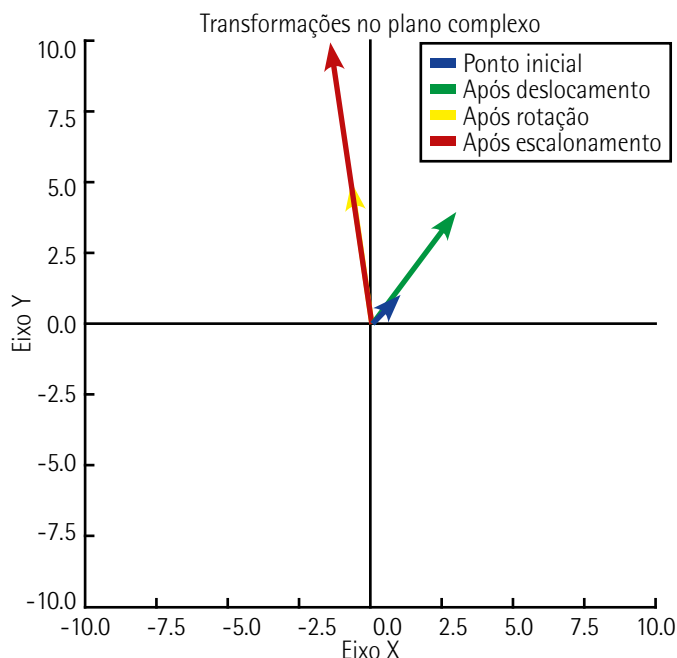


Figura 55 – Demonstração visual das transformações no plano complexo produzidas pela execução apresentada na figura 54

Em outras palavras, nosso exemplo demonstra como a manipulação de números complexos pode representar de maneira simples e elegante transformações geométricas no plano, ilustrando rotações, translações e mudanças de escala.

Convém explicar também que a instrução `import math` carrega o módulo `math` da biblioteca padrão do Python. Esse módulo contém funções matemáticas e constantes que são amplamente utilizadas em cálculos. No contexto do código, o módulo `math` é usado para:

- **Conversão de ângulo de graus para radianos:** a função `math.radians()` converte o ângulo fornecido em graus para radianos, que é a unidade exigida pelas funções trigonométricas em Python. Isso é necessário porque o fator de rotação no plano complexo é baseado na fórmula  $e^{i\theta} = \cos \theta + i \sin \theta$ , que exige o ângulo em radianos.
- **Cálculo de cosseno e seno do ângulo:** as funções `math.cos()` e `math.sin()` são usadas para calcular, respectivamente, o cosseno e o seno do ângulo em radianos.

Sem a instrução `import math`, essas funções não estariam disponíveis, e o programa não conseguiria realizar os cálculos de rotação.

Um detalhe final desse código é o uso do operador `+=`. Em Python esse é um operador de atribuição composto. Ele adiciona um valor ao valor atual de uma variável e atualiza a própria variável com o resultado. Falaremos com mais detalhes desse operador no próximo tópico.

A necessidade de compreender a passagem do tempo em diferentes unidades é fundamental para um viajante do tempo. Ao planejar saltos para o passado ou para o futuro, é importante ter clareza sobre quanto tempo se está percorrendo. Um viajante pode, por exemplo, querer saber quantos minutos distantes de seu ponto de origem ele estará se saltar cinco anos no futuro ou quantos segundos separam sua época atual de um evento histórico específico vários séculos atrás. O código-fonte a seguir apresenta um conversor de tempo que, dada uma quantidade de anos, calcula quantos meses, dias, horas, minutos e segundos correspondem a esse intervalo.

```
# Solicita ao usuário a quantidade de anos a serem convertidos
anos = float(input("Insira a quantidade de anos que deseja converter: "))

# Define parâmetros de conversão básicos
meses_por_ano = 12
dias_por_ano = 365
horas_por_dia = 24
minutos_por_hora = 60
segundos_por_minuto = 60

# Calcula o total de meses
meses = anos * meses_por_ano

# Calcula o total de dias
dias = anos * dias_por_ano

# Calcula o total de horas
horas = dias * horas_por_dia

# Calcula o total de minutos
minutos = horas * minutos_por_hora

# Calcula o total de segundos
segundos = minutos * segundos_por_minuto

# Exibe os resultados
print(f"{anos} ano(s) equivalem a aproximadamente:")
print(f"{meses} meses")
print(f"{dias} dias")
print(f"{horas} horas")
print(f"{minutos} minutos")
print(f"{segundos} segundos")
```

Figura 56

Embora não consideramos anos bissextos ou variações no número de dias, a aproximação é suficiente para ilustrar o uso de operações matemáticas, conversões entre tipos numéricos e manipulação de dados no contexto hipotético das viagens no tempo.

A linha `anos = float(input("Insira a quantidade de anos que deseja converter: "))` solicita ao usuário um valor numérico representando a quantidade de anos a serem convertidos. A função `input()` captura a entrada como uma string, que é então convertida em `float` para lidar tanto com valores inteiros quanto fracionários (por exemplo, 1.5 anos). Essa

flexibilidade é útil se o viajante quiser considerar quantidades não inteiras de anos, algo essencial quando se trata de calcular intervalos de tempo precisos para viagens temporais.

Nas linhas seguintes, são definidas variáveis como `meses_por_ano = 12`, `dias_por_ano = 365`, `horas_por_dia = 24`, `minutos_por_hora = 60` e `segundos_por_minuto = 60`. Embora simplificadas (não consideramos anos bissextos ou variações no calendário), essas constantes definem as relações fundamentais entre as unidades de tempo. A definição desses parâmetros deixa claro quais suposições estão sendo feitas, algo relevante para o viajante do tempo, que pode adaptar o código caso esteja lidando com calendários alternativos em outras épocas históricas.

A linha `meses = anos * meses_por_ano` calcula quantos meses existem no intervalo de tempo fornecido em anos. Ao multiplicar o número de anos por 12, obtém-se um valor aproximado da quantidade de meses equivalente. Se o usuário digitar 2 anos, o resultado será 24 meses.

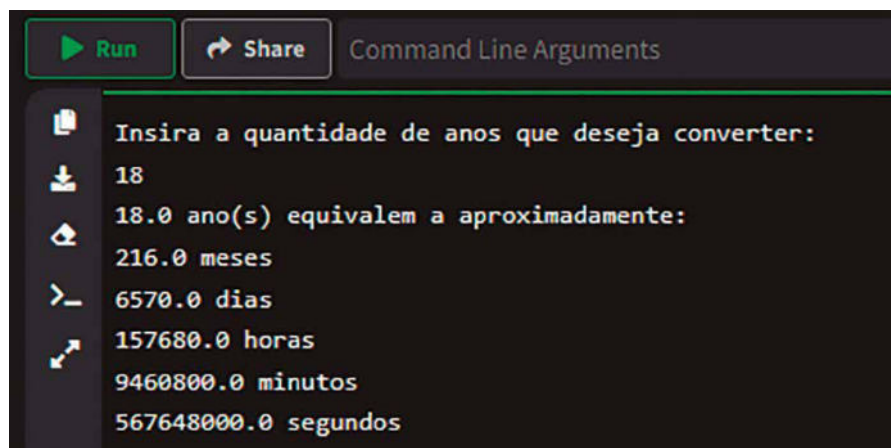
Em `dias = anos * dias_por_ano`, calculamos a quantidade de dias no período total. Multiplicar anos por 365 fornece um número aproximado de dias. Essa linha é crucial, pois a partir dela será possível chegar a unidades menores, como horas, minutos e segundos. É a ancoragem mais detalhada da conversão, já que todos os outros cálculos descem por essa cadeia de equivalências.

A linha `horas = dias * horas_por_dia` estabelece quantas horas existem no intervalo. Multiplicar o total de dias por 24 mostra quantas vezes o ponteiro imaginário do relógio dá uma volta completa nesse período. Para um viajante do tempo, isso pode ser importante se a missão exige saber quantas horas de preparação ou de espera existem antes de um evento histórico exato.

Em `minutos = horas * minutos_por_hora`, a conversão segue para uma unidade ainda mais fina. Multiplicar o número de horas por 60 revela quantos minutos compõem todo o período.

Na linha `segundos = minutos * segundos_por_minuto`, o cálculo atinge a unidade mínima proposta: o segundo. Multiplicar minutos por 60 resulta em um total de segundos. Ao lidar com quantidades grandes de anos, os números se tornam astronômicos, mostrando o quanto cada fração de tempo pode aumentar rapidamente. Para o viajante do tempo, ter uma compreensão do número de segundos pode ser interessante ao sincronizar eventos críticos, ajustar relógios temporais ou desenvolver técnicas de viagem altamente precisas.

Finalmente, as linhas de impressão `print(f"{anos} ano(s) equivalem a aproximadamente:")`, seguidas pelos prints dos resultados em meses, dias, horas, minutos e segundos, mostram ao usuário as conversões realizadas. O uso de f-strings permite inserir valores de variáveis diretamente dentro da string, resultando em uma exibição clara e amigável do resultado. Esse feedback ao usuário é essencial, pois o viajante do tempo precisa de informações imediatamente interpretáveis para planejar suas ações, sejam elas definidas em anos distantes ou em precisos segundos, garantindo que o salto no tempo será bem calculado e executado. A figura a seguir mostra o programa após a execução:



```
Run Share Command Line Arguments
Insira a quantidade de anos que deseja converter:
18
18.0 ano(s) equivalem a aproximadamente:
216.0 meses
6570.0 dias
157680.0 horas
9460800.0 minutos
567648000.0 segundos
```

Figura 57 – Resultado da execução do programa para converter 18 anos

Outra classe importante em Python é a `str`. Strings são sequências de caracteres que representam texto e são um dos tipos de dados mais importantes na linguagem. Elas permitem manipular e armazenar informações textuais de maneira eficiente, desempenhando um papel crucial em praticamente todos os programas. Desde mensagens simples exibidas ao usuário até a análise e modificação de grandes volumes de dados textuais, as strings são essenciais para o funcionamento de sistemas e aplicações.

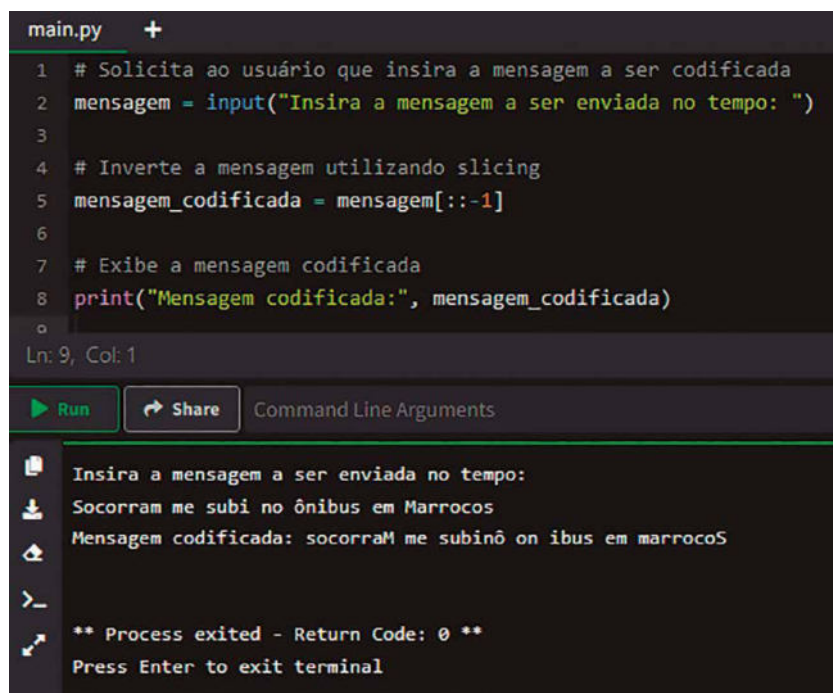
Em Python, uma string é uma classe, mais especificamente a classe embutida chamada `str`. Isso significa que cada string em Python é um objeto instanciado dessa classe e, como resultado, ela tem características (atributos) e comportamentos (métodos) definidos pela classe `str`.

As strings são definidas entre aspas simples, duplas ou até mesmo triplas, que permitem criar textos em múltiplas linhas. Essa flexibilidade na delimitação facilita a manipulação de diferentes formatos textuais, atendendo a uma ampla variedade de necessidades. Strings em Python **são imutáveis**, o que significa que, uma vez criadas, seus valores não podem ser alterados diretamente. Qualquer modificação em uma string resulta na criação de uma nova string, com o valor modificado. Essa característica promove segurança ao lidar com dados textuais e permite que strings sejam usadas de maneira eficiente em situações nas quais múltiplos processos compartilham o mesmo valor.

As strings são sequências ordenadas, o que significa que cada caractere tem uma posição única dentro da string. Essa característica permite acessar caracteres individuais por meio de seus índices, no qual o índice inicial é zero, ou realizar operações como fatiamento, que extrai partes específicas da string. A **manipulação de strings** em Python é rica em funcionalidades e permite operações como junção, separação, substituição, remoção de espaços em branco, mudança de maiúsculas e minúsculas, entre outras.

Imagine um viajante do tempo querendo enviar uma mensagem para si mesmo em uma outra época. Ao enviar uma mensagem diretamente, qualquer pessoa que a intercepte poderia ler e alterar o curso dos eventos. Para evitar isso, o viajante decide "codificar" a mensagem de uma forma simples, porém eficaz: invertendo a ordem dos caracteres. Assim, apenas alguém que saiba da técnica de inversão, ao receber a mensagem, poderá restaurá-la à forma original.

Essa técnica funciona como uma camada extra de proteção. A mensagem original é enviada para o passado ou para o futuro na forma invertida. Quando a outra versão do viajante do tempo receber o texto, basta aplicar o mesmo processo de reversão para recuperar a mensagem original. Assim, não é necessário utilizar complexos algoritmos de criptografia no fluxo temporal, apenas a inversão da string, que é uma manipulação simples, mas útil nesse cenário. A figura a seguir tem o código-fonte que implementa esse cenário e sua execução:



```
main.py +
1 # Solicita ao usuário que insira a mensagem a ser codificada
2 mensagem = input("Insira a mensagem a ser enviada no tempo: ")
3
4 # Inverte a mensagem utilizando slicing
5 mensagem_codificada = mensagem[::-1]
6
7 # Exibe a mensagem codificada
8 print("Mensagem codificada:", mensagem_codificada)
9
Ln: 9, Col: 1

Run Share Command Line Arguments

Insira a mensagem a ser enviada no tempo:
Socorram me subi no ônibus em Marrocos
Mensagem codificada: socorraM me subinô on ibus em marrocoS

** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Figura 58 – Exemplo de inversão de string

Na linha 2 do código, a função `input()` retorna sempre uma string. Independentemente do que for digitado, o resultado será um tipo `str` em Python. Caso sejam necessárias conversões para outros tipos de dados (por exemplo, inteiros ou floats), seria preciso usar funções como `int()` ou `float()`. Aqui, como estamos lidando com texto, não é preciso converter o tipo da variável.

Na linha 5, `mensagem_codificada = mensagem[::-1]`, utilizamos o recurso de slicing de strings do Python. A sintaxe `mensagem[::-1]` significa:

- Comece do início da string (`:` no começo indica início).
- Vá até o final da string (`:` no final indica ir até o fim).
- Utilize um passo de `-1`, o que significa iterar sobre a string de trás para frente, resultando em uma nova string invertida.

Por exemplo, se `mensagem = "Viagem no tempo"`, então `mensagem[::-1]` resultará em `"opmet on megaiv"`. Em Python, as strings são sequências imutáveis de caracteres. O slicing é um mecanismo para acessar partes dessa sequência. A forma geral de slicing é `[início:fim:passo]`. Quando omitimos início e fim, o Python assume o início e o fim da string. Definir `passo = -1` faz com que a string seja percorrida de trás para frente, produzindo a inversão.

Python também oferece suporte a uma vasta gama de métodos que simplificam o trabalho com strings. Esses métodos permitem executar operações comuns de forma eficiente, como verificar se uma string começa ou termina com determinados caracteres, encontrar ocorrências de substrings e substituir partes do texto. Além disso, a linguagem permite a criação de strings formatadas, nas quais valores podem ser inseridos dinamicamente em modelos textuais. Essa funcionalidade é amplamente usada para construir mensagens personalizadas, exibir resultados ou gerar relatórios.

Outro aspecto importante das strings em Python é sua capacidade de manipular texto em diferentes idiomas e sistemas de escrita, graças ao suporte nativo à **codificação Unicode**. Isso permite que caracteres de praticamente qualquer idioma, além de símbolos especiais, sejam usados sem problemas, tornando Python uma linguagem poderosa para aplicações globais. A compatibilidade com Unicode também facilita a manipulação de strings que envolvem **emojis**, caracteres de controle e outros elementos não presentes no alfabeto padrão.

A máquina do tempo chegou a uma época em que não basta apenas enviar mensagens invertidas para ocultar seu conteúdo. Em diferentes períodos históricos, a forma de escrita, o conjunto de caracteres disponíveis e mesmo a compreensão de certos símbolos mudam radicalmente. Em algumas épocas, a escrita humana está misturada com sinais complexos, mesclando caracteres latinos, ideogramas e símbolos unificados por convenções temporais. Em outras, a comunicação se torna cada vez mais universal graças a padrões gráficos conhecidos como emojis, capazes de transcender barreiras linguísticas.

O exercício da figura a seguir cria um cenário no qual um viajante do tempo deseja produzir uma mensagem compatível com múltiplos contextos históricos. Ele obtém uma mensagem do usuário, remove espaços desnecessários, converte a capitalização para um padrão coerente, substitui determinadas palavras por equivalentes simbólicos e adiciona emojis que, supostamente, são reconhecidos em qualquer período. O resultado é uma string cuidadosamente manipulada, demonstrando o uso de métodos de `str`, o emprego de caracteres Unicode e a inserção de emojis para criar uma mensagem universal.

```
main.py +
1 # Solicita ao usuário que insira uma mensagem
2 mensagem = input("Insira a mensagem a ser enviada através dos séculos: ")
3
4 # Remove espaços em branco no início e no final da mensagem
5 mensagem_limpa = mensagem.strip()
6
7 # Converte a mensagem para um formato com a primeira letra de cada palavra em maiúscula, simulando um padrão estético universal
8 mensagem_formatada = mensagem_limpa.title()
9
10 # Substitui uma palavra específica por um símbolo Unicode;
11 # neste caso, se a mensagem contiver a palavra "tempo", substituímos por um caractere especial
12 mensagem_unicode = mensagem_formatada.replace("Tempo", "TEMPO")
13
14 # Adiciona um emoji ao final da mensagem, supondo que o emoji seja reconhecido em qualquer época;
15 # aqui utilizamos um relógio para simbolizar a própria noção de tempo
16 mensagem_final = mensagem_unicode + " " + "🕒"
17
18 # Exibe a mensagem final
19 print("Mensagem através das eras:", mensagem_final)
20
```

Ln: 14, Col: 14

Run Share Command Line Arguments

Insira a mensagem a ser enviada através dos séculos:  
Nenhuma medida de tempo com você será suficiente, mas começaremos com "para sempre".  
Mensagem através das eras: Nenhuma Medida De Tempo Com Você Será Suficiente, Mas Começaremos Com "Para Sempre". 🕒

Figura 59 – Manipulação de strings

A variável `mensagem_limpa` recebe o resultado de `mensagem.strip()`. O método `strip()` remove espaços em branco no início e no final da string, permitindo eliminar caracteres que poderiam atrapalhar a uniformidade visual e semântica da mensagem. Em ambientes nos quais a apresentação da mensagem é crucial, remover espaços indesejados garante uma estética mais coerente.

Na sequência, `mensagem_formatada = mensagem_limpa.title()` converte a string para um formato em que a primeira letra de cada palavra é colocada em maiúscula, enquanto as demais letras permanecem minúsculas. O método `title()` é um dos muitos métodos de manipulação de casos em Python, que incluem `lower()`, `upper()` e `capitalize()`.

A próxima transformação aplica `mensagem_unicode = mensagem_formatada.replace("Tempo", "TEMPO")`. Aqui, `replace()` substitui a substring `Tempo` por `TEMPO`, na qual as letras da palavra são substituídas por caracteres Unicode visualmente diferentes. Esse passo ilustra o uso de Unicode para alterar a aparência da mensagem, tornando-a mais exótica ou mais universal, dependendo do contexto. Ao usar caracteres Unicode, é possível alterar sutilezas gráficas, incorporar alfabetos de outras culturas ou até mesmo usar caracteres com funções simbólicas. No contexto temporal, isso significa adaptar o conteúdo a um conjunto de caracteres que possa ser interpretado de forma interessante por diferentes civilizações ou por viajantes experientes em várias escritas.

Em seguida, `mensagem_final = mensagem_unicode + " " + "🕒"` adiciona um emoji de ampulheta ao final da mensagem. Os emojis são sequências Unicode especiais que renderizam como pequenas imagens ou símbolos coloridos, compreendidos por muitos dispositivos modernos. Ao adicionar o emoji da ampulheta, cria-se um símbolo universal do tempo, algo que, teoricamente, transcende séculos e culturas. Embora não possamos assegurar a legibilidade dos emojis em todas as épocas, em



nosso cenário fictício esses símbolos são reconhecidos como indicativos de fluxo temporal, tornando a mensagem universalmente compreensível, ou pelo menos interessante, aos olhos do receptor.

Por fim, o `print("Mensagem através das eras:", mensagem_final)` exibe o resultado. A função `print()` mostra na saída a mensagem pronta, formatada, revisada e anotada com símbolos universais. O viajante do tempo agora pode enviar essa mensagem no fluxo temporal, confiando que, independentemente de quem a receba, a estrutura visual coerente, o uso de Unicode diferenciado e a presença do emoji ajudarão na imediata compreensão do conteúdo, ou ao menos despertarão curiosidade.

Além das operações básicas, strings em Python podem ser usadas em conjunto com outras estruturas e ferramentas da linguagem para resolver problemas mais complexos. Elas podem ser facilmente iteradas em loops, analisadas com expressões regulares e combinadas com estruturas de dados como listas e dicionários para criar sistemas robustos de processamento textual. Essa versatilidade permite que strings sejam utilizadas tanto em tarefas simples, como concatenar mensagens, quanto em aplicações avançadas, como análise de sentimentos, mineração de textos e processamento de linguagem natural.

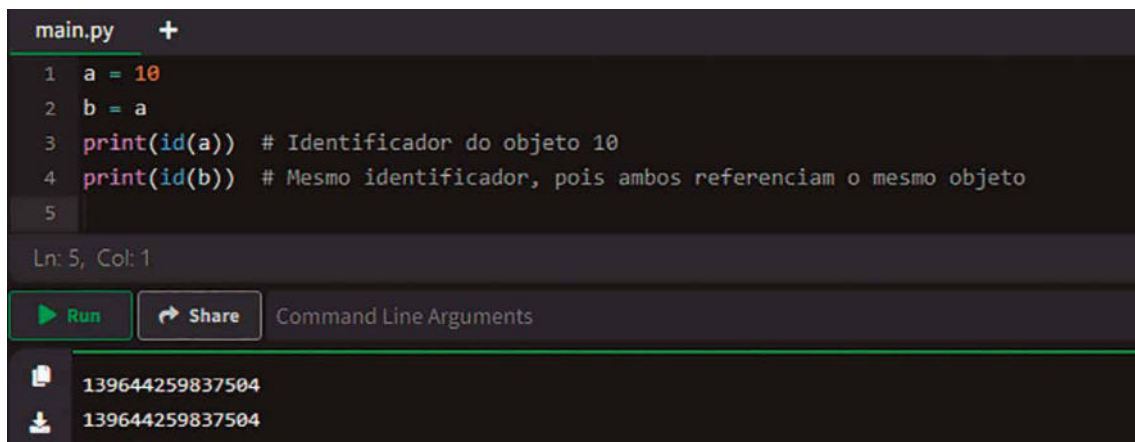
As strings são, portanto, um tipo de dado central em Python, oferecendo simplicidade para iniciantes e flexibilidade para programadores avançados. Combinando facilidade de uso, desempenho e uma ampla gama de funcionalidades, elas tornam a manipulação de texto intuitiva e eficiente, independentemente do contexto ou da complexidade do problema a ser resolvido.

### 3.2 Variáveis e operadores em Python

Em Python, variáveis e operadores são componentes fundamentais para a construção de qualquer programa, permitindo o armazenamento e manipulação de dados, bem como a execução de operações lógicas e aritméticas. A definição e o uso de variáveis em Python são caracterizados pela simplicidade e flexibilidade, pois a linguagem adota uma **tipagem dinâmica**, o que significa que **o tipo da variável é determinado automaticamente com base no valor atribuído**. Por exemplo, ao escrever `x = 10`, a variável `x` é automaticamente considerada um inteiro, enquanto em `x = "texto"` a mesma variável é redefinida como uma string. Essa abordagem elimina a necessidade de declarações explícitas de tipo, facilitando o desenvolvimento rápido de programas, mas exigindo cuidado do programador para evitar inconsistências no uso de dados.

Tecnicamente, uma variável em Python é um **nome simbólico que referencia um objeto armazenado na memória**. Ela funciona como um identificador que aponta para dados, permitindo ao programador acessar, manipular e operar sobre esses dados de forma conveniente.

Python utiliza um modelo de **gerenciamento de memória baseado em referência**. Cada variável armazena uma referência **ao endereço na memória** no qual o objeto está localizado, em vez de armazenar diretamente os dados. Esse modelo permite que diversas variáveis apontem para o mesmo objeto na memória, o que pode ser verificado usando a função `id()`, que retorna o identificador único de um objeto. No exemplo da figura a seguir usamos a função `id()`, que exibiu o endereço de memória 13964425983750:



```
main.py +
1 a = 10
2 b = a
3 print(id(a)) # Identificador do objeto 10
4 print(id(b)) # Mesmo identificador, pois ambos referenciam o mesmo objeto
5

Ln: 5, Col: 1

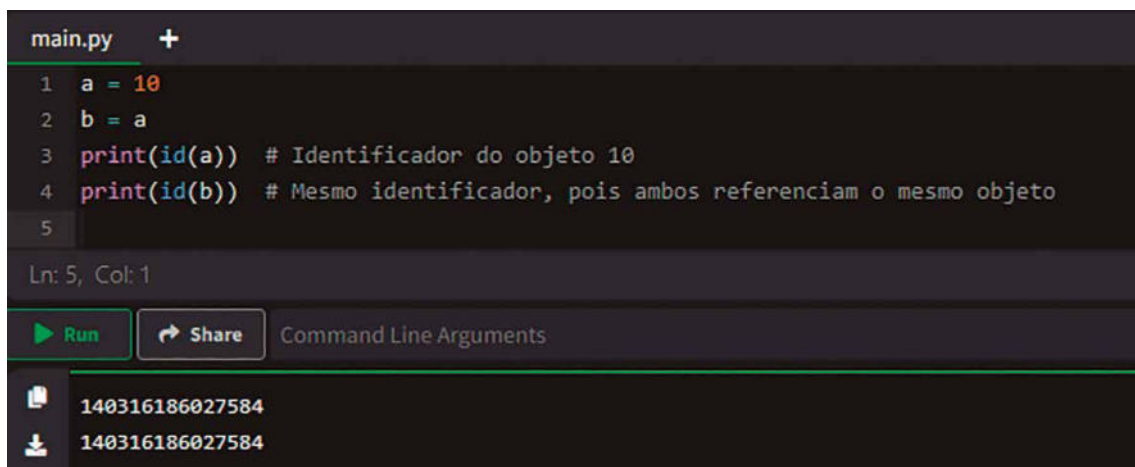
Run Share Command Line Arguments

139644259837504
139644259837504
```

Figura 60 – Exibindo endereço de memória das variáveis usando a função `id()`

Quando escrevemos `a = 10`, Python cria um objeto do tipo `int` com o valor 10 e armazena esse objeto em um endereço específico na memória. A variável `a` não contém diretamente o valor 10, mas sim uma referência ao endereço de memória onde o objeto está armazenado. Quando escrevemos `b = a`, a variável `b` passa a referenciar o mesmo objeto ao qual `a` já está associada, ou seja, ambas compartilham a mesma referência. Por isso, dentro de uma única execução, o identificador retornado por `id(a)` e `id(b)` será o mesmo, já que ambos estão apontando para o mesmo objeto.

No entanto, os endereços de memória nos quais os objetos são armazenados **são atribuídos dinamicamente** pelo sistema subjacente que gerencia a execução do Python. Isso significa que, **ao executar o programa novamente**, Python pode alocar o objeto 10 em um endereço de memória diferente. Embora o valor do objeto (10) permaneça o mesmo, seu identificador (`id`) pode variar entre execuções porque o mapeamento entre objetos e endereços de memória depende das condições e do estado do sistema no momento em que o programa é executado. Confira na figura a seguir que o novo endereço de memória atribuído às variáveis é 140316186027584, um endereço distinto da execução da figura anterior:



```
main.py +
1 a = 10
2 b = a
3 print(id(a)) # Identificador do objeto 10
4 print(id(b)) # Mesmo identificador, pois ambos referenciam o mesmo objeto
5

Ln: 5, Col: 1

Run Share Command Line Arguments

140316186027584
140316186027584
```

Figura 61 – Uma nova execução do mesmo programa da figura 60 gera um endereço de memória diferente

Esse comportamento é uma consequência do gerenciamento dinâmico de memória em Python, que **não garante** que os objetos sejam alocados nos mesmos endereços de memória **entre execuções**. Portanto, enquanto os identificadores (`id`) **são consistentes dentro de uma única execução** para variáveis que compartilham o mesmo objeto, eles **podem ser diferentes em execuções separadas** devido à alocação dinâmica de memória. Esse mecanismo garante eficiência e flexibilidade no uso dos recursos do sistema.

Imagine a memória do computador como um grande prédio de apartamentos, no qual cada apartamento tem um **número único na porta** (o endereço) e pode conter um objeto específico (o valor). Nesse prédio, as variáveis em Python funcionam como **etiquetas que você pode colar na porta** de um apartamento para identificar o que está dentro dele. O administrador do prédio é o sistema de gerenciamento de memória, responsável por organizar os apartamentos e decidir onde cada objeto será armazenado.

Quando você cria uma variável, por exemplo, `a = 10`, é como se você dissesse ao administrador: "Coloque o número 10 em um dos apartamentos disponíveis e cole a etiqueta `a` na porta desse apartamento". Se, em seguida, você fizer `b = a`, o administrador não cria um novo apartamento para o valor 10. Em vez disso, ele cola uma segunda etiqueta, `b`, na mesma porta. Agora, tanto `a` quanto `b` apontam para o mesmo apartamento, e qualquer referência a elas acessará o mesmo objeto.

No entanto, o administrador é dinâmico e organiza os apartamentos de forma diferente a cada vez que o prédio é usado. Isso significa que, se você repetir o mesmo processo (`a = 10` e `b = a`) em um dia diferente, o administrador pode escolher outro apartamento para armazenar o valor 10. O número do apartamento (o endereço de memória) onde o valor 10 está armazenado pode ser diferente a cada execução do programa, mesmo que a lógica seja idêntica. Esse comportamento reflete o sistema de alocação dinâmica de memória usado pelo Python, no qual os identificadores (`id`) de objetos podem variar entre execuções.

Agora, suponha que você mude o valor de `a` para algo diferente, como `a = 20`. Nesse caso, o administrador encontra um novo apartamento vazio, coloca o valor 20 lá e move a etiqueta `a` para essa nova porta. A etiqueta `b`, no entanto, continua colada na porta do apartamento original, onde ainda está o valor 10. Isso explica por que modificar `a` não afeta `b`, já que agora elas estão apontando para endereços diferentes. A figura a seguir ilustra exatamente esse cenário. Note que o endereço de memória da variável `a` mudou de 139942382499392 para 139942382499712, enquanto o endereço de memória da variável `b` permanece o mesmo (139942382499392).

```
main.py +
1 a = 10
2 b = a
3 print(id(a)) # Identificador do objeto 10
4 print(id(b)) # Mesmo identificador, pois ambos referenciam o mesmo objeto
5
6 a = 20
7 print("\nMudamos o valor de a, repare o novo endereço de a.")
8 print(id(a)) # Identificador do objeto 10
9 print(id(b)) # Mesmo identificador, pois ambos referenciam o mesmo objeto

Ln: 7, Col: 20
Run Share Command Line Arguments

139942382499392
139942382499392
Mudamos o valor de a, repare o novo endereço de a.
139942382499712
139942382499392
```

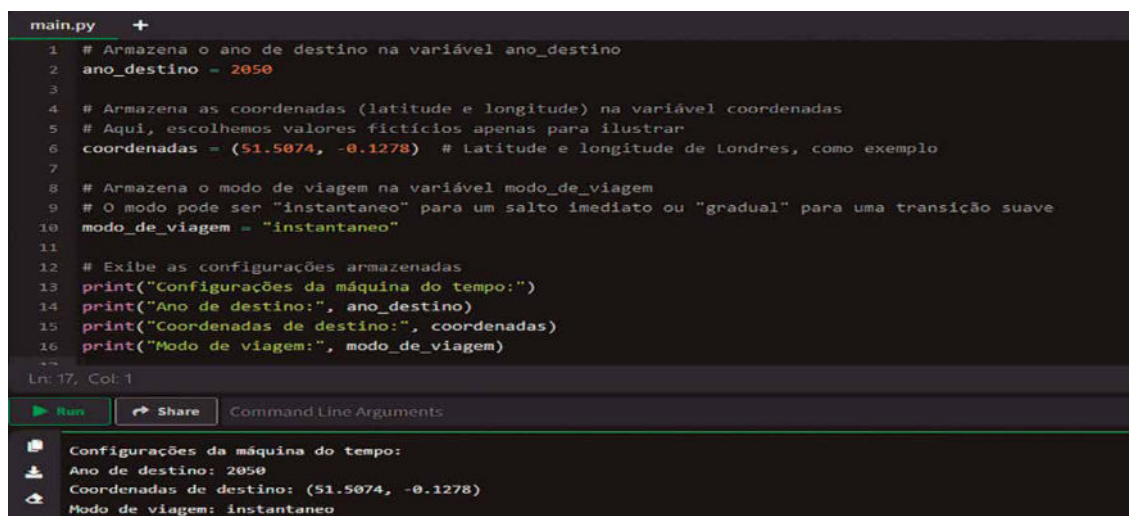
Figura 62 – Alteração de endereço de memória ao mudar o valor de a para 20

Essa dinâmica se torna mais interessante ao lidar com objetos mutáveis, como listas. Quando você cria uma lista, é como se o apartamento contivesse várias gavetas dentro dele, e cada gaveta armazenasse um elemento da lista. Se você fizer `lista1 = [1, 2, 3]` e depois `lista2 = lista1`, ambas as etiquetas são coladas na mesma porta. Se você abrir uma das gavetas e modificar um elemento (por exemplo, mudar o valor de 2 para 5), essa alteração será visível independentemente de qual etiqueta você usou para acessar a lista, porque ambas referenciam o mesmo apartamento.

Essa analogia do prédio, com apartamentos dinâmicos e etiquetas que podem ser movidas ou compartilhadas, ilustra como Python gerencia a memória e as variáveis. Ele explica por que os identificadores de objetos podem variar entre execuções, enquanto dentro de uma única execução a relação entre variáveis que compartilham um objeto é sempre consistente.

A **atribuição** de valores às variáveis utiliza o operador de **igualdade simples** (`=`), que não deve ser confundido com o operador lógico de igualdade (`==`). Atribuir `a = 20` significa que a variável `a` agora contém o valor 20, enquanto `a == 20` verifica se o valor armazenado em `a` é igual a 20, retornando um valor booleano, ou seja, `True` ou `False`. Essa distinção entre atribuição e comparação é central para evitar erros lógicos em programas. Além disso, variáveis em Python podem armazenar qualquer tipo de dado, desde números e textos até listas, dicionários e objetos mais complexos, tornando-as ferramentas versáteis.

A preparação para uma viagem temporal exige um cuidado meticuloso ao configurar a máquina do tempo. Antes que o viajante possa se lançar ao passado distante ou ao futuro incerto, é necessário definir parâmetros fundamentais que a máquina utilizará para realizar o salto através do continuum espaço-tempo. Essas configurações incluem o ano de destino, que determina quando exatamente o viajante irá emergir na linha histórica; as coordenadas de localização, que indicam onde no planeta (ou em outro corpo celeste) o viajante surgirá; e o modo de viagem, que pode indicar o tipo de deslocamento temporal escolhido, por exemplo, uma transição lenta e gradual ou um salto instantâneo. O código da figura ilustra como armazenar essas informações em variáveis, fornecendo um esboço de como um viajante do tempo ou um operador de uma máquina temporal poderia preparar a jornada antes de acionar os controles.



```
main.py +
1 # Armazena o ano de destino na variável ano_destino
2 ano_destino = 2050
3
4 # Armazena as coordenadas (latitude e longitude) na variável coordenadas
5 # Aqui, escolhemos valores fictícios apenas para ilustrar
6 coordenadas = (51.5074, -0.1278) # Latitude e longitude de Londres, como exemplo
7
8 # Armazena o modo de viagem na variável modo_de_viagem
9 # O modo pode ser "instantaneo" para um salto imediato ou "gradual" para uma transição suave
10 modo_de_viagem = "instantaneo"
11
12 # Exibe as configurações armazenadas
13 print("Configurações da máquina do tempo:")
14 print("Ano de destino:", ano_destino)
15 print("Coordenadas de destino:", coordenadas)
16 print("Modo de viagem:", modo_de_viagem)
17
18 Ln: 17, Col: 1
Run Share Command Line Arguments
Configurações da máquina do tempo:
Ano de destino: 2050
Coordenadas de destino: (51.5074, -0.1278)
Modo de viagem: instantaneo
```

Figura 63 – Utilização de variáveis para a máquina do tempo

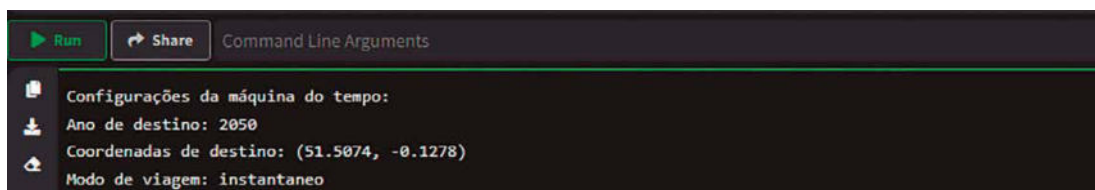
Na primeira parte do código, `ano_destino = 2050`, criamos uma variável chamada `ano_destino` e atribuímos a ela o valor inteiro 2050. Esse valor representa o ano exato para o qual a máquina do tempo deverá transportar o viajante. O uso de um inteiro (`int`) é apropriado, já que o ano pode ser facilmente representado como um número sem necessidade de frações.

Na linha `coordenadas = (51.5074, -0.1278)`, criamos a variável `coordenadas`, que recebeu uma **tupla** contendo dois valores de ponto flutuante (`float`), representando a latitude e a longitude. Uma tupla é usada para armazenar múltiplos valores em uma só variável, mantendo a ordem e permitindo acesso individual a cada elemento, se necessário. Tuplas são imutáveis, ou seja, uma vez criadas, seus elementos não podem ser alterados, adicionados ou removidos. Ao escolher `coordenadas`, como a latitude e longitude de Londres, fornecemos um exemplo de um lugar real na Terra. Em um cenário de viagem no tempo, essas coordenadas determinariam onde, no globo terrestre, o viajante emergiria no ano especificado.

Em `modo_de_viagem = "instantaneo"`, criamos a variável `modo_de_viagem` e atribuímos a ela a string "instantaneo". Como vimos anteriormente, string (`str`) é um tipo de dado adequado para armazenar palavras ou cadeias de caracteres. Aqui, a escolha de "instantaneo" é apenas um

exemplo; a lógica do programa poderia suportar outros modos, como "gradual", "seguro", "experimental" ou qualquer outro que faça sentido no contexto da tecnologia da máquina do tempo.

Por fim, as linhas de `print()` servem para exibir as configurações armazenadas. A saída dessas funções `print()` permitirá ao operador da máquina do tempo verificar as configurações antes de iniciar a viagem, garantindo que tudo esteja conforme o planejado. A figura a seguir mostra o resultado da execução do programa.



```
Run Share Command Line Arguments
Configurações da máquina do tempo:
Ano de destino: 2050
Coordenadas de destino: (51.5074, -0.1278)
Modo de viagem: instantaneo
```

Figura 64 – Resultado da execução do programa da figura 63

Uma variável pode armazenar praticamente qualquer tipo de objeto, uma vez que a linguagem adota um modelo baseado em objetos, no qual tudo é tratado como instância de uma classe. Além dos tipos básicos como inteiros, floats, strings e tuplas, que acabamos de utilizar no código da figura 63, variáveis podem conter uma vasta gama de outros objetos, o que torna Python extremamente versátil e poderoso para resolver problemas de diferentes complexidades.

Entre os tipos de objetos que podem ser atribuídos a variáveis, estão as **listas**, que são coleções ordenadas e mutáveis. Elas permitem armazenar elementos de diferentes tipos e realizar operações como adição, remoção e modificação de itens, sendo amplamente usadas para manipular sequências de dados. Outra estrutura vital são os **dicionários**, que armazenam pares de chave-valor e permitem acesso rápido aos dados com base em suas chaves, sendo ideais para representar estruturas mais complexas. Veremos em pormenores o uso de listas e dicionários depois.

Os **conjuntos** também são objetos que podem ser atribuídos a variáveis e são utilizados para armazenar coleções de elementos únicos. Python oferece tanto a versão mutável, chamada `set`, quanto a versão imutável, conhecida como `frozenset`, o que amplia sua aplicabilidade em diferentes contextos. Além disso, o suporte nativo a números complexos, representados na forma  $a + bj$ , permite realizar cálculos avançados que envolvem partes reais e imaginárias, sendo uma característica valiosa em áreas científicas e matemáticas.

**Funções**, por sua vez, **também são objetos** em Python e **podem ser atribuídas a variáveis**, passadas como argumentos para outras funções ou retornadas por elas. Essa flexibilidade facilita a construção de programas dinâmicos e modulares. **Objetos personalizados**, criados a partir de classes definidas pelo usuário, podem igualmente ser armazenados em variáveis, tornando Python ideal para POO.

Além disso, Python suporta o armazenamento de dados binários por meio de tipos como `bytes` e `bytearray`, úteis para manipular arquivos e fluxos de dados em baixo nível. A **programação assíncrona** em Python introduz outro conjunto de objetos que podem ser atribuídos a variáveis,



como corrotinas, tarefas e futuros, usados em operações não bloqueantes que lidam com múltiplas tarefas simultaneamente.

Essa programação assíncrona traz consigo ferramentas para lidar com tarefas que podem ser executadas de forma simultânea ou parcialmente sobrepostas, sem que o programa fique "parado" esperando que uma tarefa termine antes de começar outra. Em vez disso, o programa pode continuar trabalhando enquanto aguarda a conclusão de operações demoradas, como acessar dados da internet ou ler arquivos grandes.

Nesse contexto, Python introduz conceitos como corrotinas, tarefas e futuros, que são tipos de objetos utilizados na programação assíncrona e podem ser atribuídos a variáveis para facilitar seu controle.

Uma **corrotina** (no inglês, *coroutine*) é basicamente uma função especial que pode ser pausada e retomada em diferentes pontos. Ao contrário de funções normais, que começam e terminam de uma só vez, as corrotinas permitem que partes do código sejam executadas, pausadas para aguardar o término de alguma operação (como esperar uma resposta de um servidor), e retomadas assim que a operação terminar. Elas são definidas usando a palavra-chave `async` antes do `def` na definição da função e podem ser usadas com a palavra-chave `await`.

Uma **tarefa** (no inglês, *task*) é um objeto que Python utiliza para agendar e gerenciar a execução de uma corrotina. Quando você transforma uma corrotina em uma tarefa, está dizendo ao Python para começar a executá-la enquanto o programa continua trabalhando em outras coisas. Esse recurso é útil porque permite que o programa lide com várias tarefas simultaneamente sem precisar esperar que cada uma termine antes de começar a próxima.

Já um **futuro** (no inglês, *future*) é um objeto que representa o "resultado pendente" de uma operação assíncrona. Ele é como uma promessa de que o resultado de uma operação estará disponível em algum momento no futuro. Um futuro pode ser usado para verificar se a operação foi concluída, obter o resultado ou até mesmo manipular possíveis erros que tenham ocorrido durante o processo.

Esses três conceitos – *coroutine*, *tasks* e *futures* – trabalham juntos para permitir que o Python execute múltiplas tarefas de forma eficiente e não bloqueante. São especialmente úteis em aplicações modernas, como servidores web, sistemas que processam grandes volumes de dados ou programas que precisam responder rapidamente a múltiplos usuários ao mesmo tempo, garantindo que o desempenho do programa não seja comprometido por operações demoradas. Com a programação assíncrona, o código se torna mais eficiente, mesmo em situações em que várias tarefas precisam ser realizadas simultaneamente.

Python também permite que variáveis armazenem módulos e classes, possibilitando a reutilização de partes de bibliotecas ou a manipulação de tipos personalizados de forma dinâmica. Mesmo o valor especial `None`, que representa a ausência de valor ou uma referência nula, pode ser atribuído a variáveis para sinalizar estados específicos ou padrões. A linguagem também fornece tipos adicionais no módulo `collections`, como `deque` (uma fila de dois lados), `Counter` (um contador para elementos) e `OrderedDict` (um dicionário ordenado).



Essa ampla gama de possibilidades para o armazenamento de objetos em variáveis reflete a flexibilidade de Python, permitindo que a linguagem seja utilizada em tarefas simples, como cálculos matemáticos, e em sistemas complexos, como aprendizado de máquina e redes neurais.



## Observação

Essa capacidade de lidar com diferentes tipos de dados de maneira integrada é um dos fatores que fazem de Python uma das linguagens de programação mais populares e acessíveis da atualidade.

Em Python, as convenções de nomenclatura de variáveis seguem as diretrizes do PEP 8, o guia oficial de estilo para a linguagem. Essas convenções não são regras obrigatórias, mas são amplamente adotadas pela comunidade Python para garantir clareza, legibilidade e consistência no código. Observe a seguir os principais pontos:

- Letras minúsculas e underscores:
  - Variáveis devem ser escritas em letras minúsculas, com palavras separadas por underscores ( `_` ) para melhorar a legibilidade. Essa convenção é conhecida como `snake_case`. Exemplo: `minha_variavel`, `idade_do_usuario`.
- Nomes descritivos:
  - Os nomes devem ser claros e indicar o propósito da variável. Evite nomes genéricos ou muito curtos, como `x` ou `data`, a menos que seu contexto seja óbvio. Exemplo: `contador`, `nome_cliente`, `total_vendas`.
- Evitar palavras reservadas:
  - Nomes de variáveis não podem ser iguais a palavras-chave (keywords) de Python, como `if`, `else`, `class`, ou `return`, pois isso resultará em erro. Exemplo inválido: `class = "teste"`.
- Uso de underscores iniciais:
  - Um único underscore inicial ( `_` ) pode ser usado para indicar que uma variável é "privada" ou destinada a uso interno, embora isso seja apenas uma convenção, sem efeitos obrigatórios. Exemplo: `_variavel_interna`.

- Dunders ("double underscores"):
  - Variáveis com dois underscores no início e no final (`__nome__`) são reservadas para métodos e atributos especiais do Python, como `__init__` ou `__str__`. Não crie nomes personalizados nesse formato, pois isso pode causar conflitos. Exemplo: `__minha_variavel__` (evitar).
- Uso de maiúsculas:
  - Em Python, nomes de variáveis geralmente não usam letras maiúsculas, exceto quando representam constantes (valores que não devem ser alterados). Para constantes, use letras maiúsculas com underscores entre palavras. Exemplo: `TAXA_DE_IMPOSTO`, `VALOR_MAXIMO`.
- Evitar números isolados:
  - Nomes de variáveis podem conter números, mas evite usá-los isoladamente ou como parte inicial do nome. É melhor combinar números com texto descritivo.
    - **Exemplo válido:** `versao2`, `arquivo_1`.
    - **Exemplo inválido:** `2versao`
- Nomes curtos em contextos locais:
  - Para variáveis temporárias ou em contextos muito restritos, como dentro de loops, nomes curtos, como `i`, `j` ou `temp`, são aceitáveis. Exemplo: `for i in range(10):`

Essas convenções ajudam a manter o código mais legível, organizado e consistente, tanto para o programador **quanto para outros que precisem trabalhar no mesmo projeto**. Seguir essas diretrizes também reflete boas práticas de programação, promovendo a escrita de códigos claros e profissionais.



### Saiba mais

O PEP 8 é o guia oficial de estilo para escrever códigos Python, chamado de Python Enhancement Proposal 8. Ele define um conjunto de convenções que os programadores devem seguir para tornar o código mais legível, organizado e consistente, especialmente em projetos colaborativos. Criado por Guido van Rossum (o criador de Python) e outros colaboradores, o PEP 8 é amplamente adotado pela comunidade Python como padrão de boas práticas.

As diretrizes do PEP 8 abrangem diversos aspectos da escrita de código Python, como: indentação, comprimento das linhas, nomes de variáveis, funções e classes, espaçamento em torno de operadores e vírgulas, estrutura de importações, comentários, docstrings e linhas em branco.

Seus detalhes podem ser vistos em:

PEP 8. Disponível em: <https://shre.ink/gcLI>. Acesso em: 10 dez. 2024.

Além disso, ferramentas como Black, autopep8 e pylint ajudam a verificar automaticamente se o código está aderente ao PEP 8, corrigindo inconsistências e formatando o código conforme as diretrizes.

Black. Disponível em: <https://shre.ink/gcLf>. Acesso em: 10 dez. 2024.

autopep8. Disponível em: <https://shre.ink/gcLT>. Acesso em: 10 dez. 2024.

pylint. Disponível em: <https://shre.ink/gcL2>. Acesso em: 10 dez. 2024.

As variáveis em Python têm características intrínsecas à linguagem que vão além das diretrizes do PEP 8, refletindo as regras sintáticas e estruturais que definem como identificadores podem ser usados. A primeira dessas características é que os nomes de variáveis devem começar com uma letra (de A a Z, maiúscula ou minúscula) ou com um sublinhado (\_). Isso significa que números ou outros caracteres não podem iniciar um identificador, embora possam ser utilizados após o primeiro caractere. Por exemplo, `minha_variavel` e `_variavel` são nomes válidos, enquanto `2variavel` não é permitido.

Outro aspecto é que os nomes de variáveis não podem conter caracteres especiais, como @, #, ou &, nem espaços. Isso garante que os identificadores sejam consistentes e compatíveis com a sintaxe do Python, evitando ambiguidades no código. Assim, `minha_variavel` é válido, mas `minha variavel` resultaria em erro, pois o espaço interromperia a interpretação do identificador.

Python também diferencia maiúsculas de minúsculas, tornando-o uma **linguagem case-sensitive**. Dito de outro modo, nomes como `variavel` e `Variavel` são considerados identificadores distintos, mesmo que tenham a mesma sequência de letras. Esse aspecto exige atenção especial, pois pode causar erros em casos nos quais variáveis com nomes semelhantes são usadas de forma inconsistente.

**Operadores**, por sua vez, são símbolos que indicam operações a serem realizadas sobre variáveis e valores. Entre eles estão os **operadores lógicos**, que são amplamente usados para avaliar condições e controlar o fluxo de execução dos programas.

Além dos operadores lógicos, Python oferece **operadores de comparação**, como `==`, `!=`, `>`, `<`, `>=` e `<=`, que permitem comparar valores e determinar relações entre eles. Esses operadores são essenciais para construir expressões que avaliam condições, sendo usados em conjunto com os operadores lógicos para criar verificações mais complexas. Por exemplo, `x == 5 or y > 10` verifica se `x` é igual a 5 ou se `y` é maior que 10, enquanto `not (x != 5)` verifica se `x` é igual a 5 de maneira equivalente.

A comparação de anos torna-se importante quando se trata de determinar qual intervalo temporal é mais distante do presente, sobretudo se um viajante do tempo precisa decidir para qual época voltar ou avançar. Imaginemos um explorador das eras que, prestes a partir com sua máquina do tempo, quer saber entre duas datas possíveis qual delas está mais afastada do ano atual. Esse tipo de comparação ajuda a compreender o impacto da escolha, pois saltar mil anos à frente ou trezentos anos ao passado não é a mesma coisa do que se deslocar apenas algumas décadas. No código a seguir, o usuário insere dois anos e o programa verifica qual dos dois está mais distante do presente, auxiliando o viajante a estimar a magnitude da jornada temporal.

```
# Define o ano presente
ano_presente = 2030

# Solicita ao usuário que insira o primeiro ano
ano1 = int(input("Insira o primeiro ano: "))

# Solicita ao usuário que insira o segundo ano
ano2 = int(input("Insira o segundo ano: "))

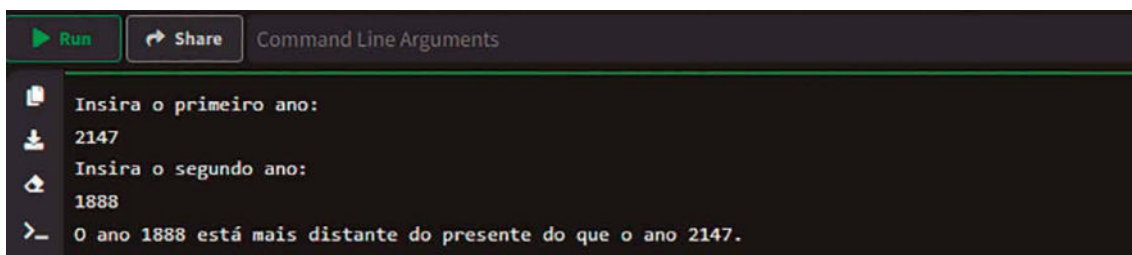
# Calcula a distância em anos do primeiro ano em relação ao presente
distancia_ano1 = abs(ano1 - ano_presente)

# Calcula a distância em anos do segundo ano em relação ao presente
distancia_ano2 = abs(ano2 - ano_presente)

# Compara as distâncias para determinar qual é mais distante do presente
if distancia_ano1 > distancia_ano2:
    print(f"O ano {ano1} está mais distante do presente do que o ano {ano2}.")
elif distancia_ano2 > distancia_ano1:
    print(f"O ano {ano2} está mais distante do presente do que o ano {ano1}.")
else:
    print(f"Os anos {ano1} e {ano2} estão igualmente distantes do presente.")
```

Figura 65

A figura a seguir apresenta a execução do programa para os anos de 2147 e 1888:



```
Run Share Command Line Arguments
Insira o primeiro ano:
2147
Insira o segundo ano:
1888
> O ano 1888 está mais distante do presente do que o ano 2147.
```

Figura 66 – Resultado da execução do programa que calcula a distância temporal

Na primeira linha relevante, `ano_presente = 2030`, criamos uma variável inteira que representa o ano atual, aqui escolhido como 2030. Embora, na prática, o ano presente pudesse ser obtido do sistema ou inserido pelo usuário, fixá-lo no código simplifica o cenário. Definir um ano presente é fundamental, pois todas as comparações serão relativas a esse marco temporal. Em um contexto de viagem no tempo, é a âncora a partir da qual mediremos a distância a outros anos.

A linha `ano1 = int(input("Insira o primeiro ano: "))` solicita ao usuário que insira um ano e, após receber a entrada como string, converte-a para inteiro (`int()`), armazenando o resultado na variável `ano1`. Esse valor é crucial, pois sem a conversão para inteiro não poderíamos realizar operações matemáticas adequadas. Ao pedir ao usuário que insira o ano, o programa simula o momento em que o viajante está considerando um possível destino temporal.

De forma análoga, `ano2 = int(input("Insira o segundo ano: "))` repete o processo para um segundo ano, armazenando-o na variável `ano2`. Agora temos dois anos distintos, ambos convertidos para inteiros, prontos para serem comparados.

Em `distancia_ano1 = abs(ano1 - ano_presente)`, calculamos a diferença absoluta entre o primeiro ano e o ano presente. A função `abs()` retorna o valor absoluto do resultado da subtração, removendo qualquer sinal negativo. Por exemplo, se o presente é 2030 e o usuário digita 1900, a diferença será  $\text{abs}(1900 - 2030) = \text{abs}(-130) = 130$ . Essa magnitude indica o quão longe no passado ou futuro o ano está.

Da mesma forma, `distancia_ano2 = abs(ano2 - ano_presente)` faz o mesmo cálculo para o segundo ano, resultando em outro valor absoluto indicando a diferença entre esse ano e o ano presente.

Com `if distancia_ano1 > distancia_ano2:`, iniciamos uma estrutura condicional para determinar qual ano é mais distante do presente. Se a distância do primeiro ano (`distancia_ano1`) for maior que a do segundo (`distancia_ano2`), isso significa que `ano1` está mais afastado de 2030 do que `ano2`. Nesse caso, o programa imprime "O ano {ano1} está mais distante do presente do que o ano {ano2}.". Essa mensagem informa ao viajante do tempo que o primeiro destino proposto envolve um salto maior no tempo.

No `elif distancia_ano2 > distancia_ano1`: ocorre a situação inversa. Se a distância do segundo ano for maior que a do primeiro, então `ano2` é o mais distante. Nesse caso, imprimimos "O ano {ano2} está mais distante do presente do que o ano {ano1}.". Assim, o viajante sabe que o segundo destino sugerido implica um deslocamento temporal maior.

Por fim, se nenhuma das condições anteriores for verdadeira, significa que as distâncias são iguais. O `else`: imprime "Os anos {ano1} e {ano2} estão igualmente distantes do presente.", indicando que ambos os anos têm a mesma quantidade de anos de distância em relação a 2030. Essa possibilidade é útil, pois o viajante pode se ver numa situação em que tanto um período do passado quanto um do futuro estão igualmente afastados do presente, tornando a decisão um mero empate técnico em termos de distância temporal.

A lógica apresentada acentua como operadores de comparação, como `>` e `==`, são aplicados para verificar relações entre valores numéricos. Além disso, mostra a utilidade de funções integradas como `abs()` na manipulação de valores absolutos.

A determinação do intervalo de tempo entre o ano atual e o ano de destino é um ponto de partida essencial para um viajante do tempo. Antes de iniciar um salto temporal, o viajante precisa saber quantos anos o separam de seu objetivo, seja ele se dirigir a um futuro distante para observar o avanço da humanidade ou retornar ao passado para estudar civilizações antigas. Na figura a seguir utilizamos operadores aritméticos para calcular a diferença entre o ano atual e o ano de destino.

```
1 from datetime import datetime
2
3 # Obtém o ano atual do sistema
4 ano_atual = datetime.now().year
5
6 # Solicita ao usuário que insira o ano de destino
7 ano_destino = int(input("Insira o ano de destino: "))
8 # Calcula o intervalo de tempo entre o ano atual e o ano de destino
9 intervalo = ano_destino - ano_atual
10 # Exibe o resultado
11 print("O intervalo de tempo entre", ano_atual, "e", ano_destino, "é de", intervalo, "anos.")
```

Figura 67 – Cálculo do intervalo em anos para auxiliar o planejamento da jornada temporal do viajante

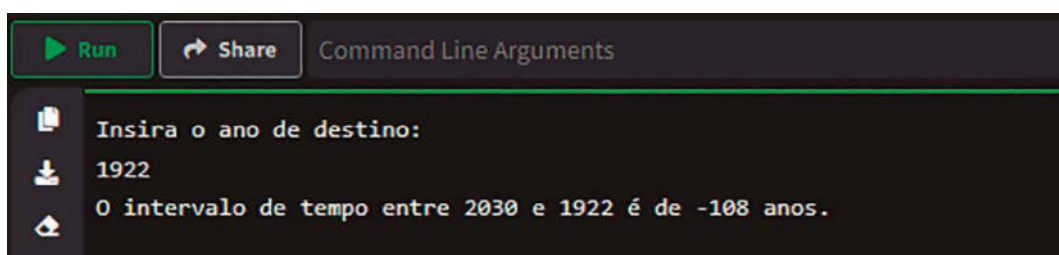
Ao importar o módulo `datetime` e chamar `datetime.now().year`, o código obtém diretamente do sistema o ano corrente no momento da execução do programa. Dessa forma, não é mais necessário manter o ano atual fixo no código.

Com `ano_atual = datetime.now().year`, obtemos um inteiro representando o ano presente. Em seguida, `ano_destino = int(input("Insira o ano de destino: "))` solicita ao usuário o ano de destino. A função `input()` captura a entrada como uma string, e a função `int()` a converte em inteiro, permitindo o uso de operadores aritméticos. Ao inserir, por exemplo, 2050, o usuário estabelece o ponto temporal final ao qual o viajante deseja chegar.



A linha `intervalo = ano_destino - ano_atual` utiliza o operador de subtração para determinar quantos anos separam o ano atual do ano de destino. Se o ano de destino for maior do que o ano atual, o resultado será positivo, indicando quantos anos no futuro o viajante percorrerá. Se o ano de destino for menor, o resultado será negativo, sinalizando quantos anos no passado está o ponto de chegada.

Por fim, `print("O intervalo de tempo entre", ano_atual, "e", ano_destino, "é de", intervalo, "anos.")` exibe a mensagem ao usuário, combinando texto e variáveis para construir uma frase coerente. Se o intervalo for positivo, o viajante sabe o quanto avançará no tempo; se negativo, sabe o quanto retornará ao passado. A figura a seguir apresenta o resultado da execução do programa que calcula o salto temporal do ano atual até 1922:



```
Run Share Command Line Arguments
Insira o ano de destino:
1922
O intervalo de tempo entre 2030 e 1922 é de -108 anos.
```

Figura 68 – Resultado da execução do programa que calcula o salto temporal

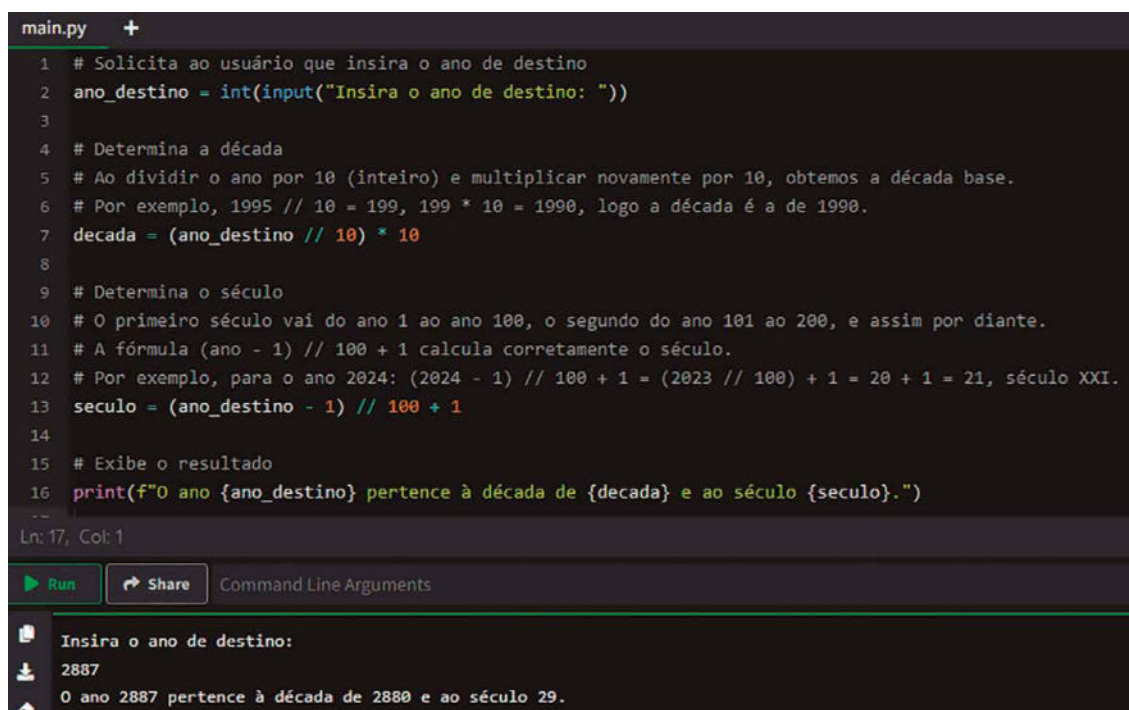
Ao utilizar a data atual do sistema, o programa se adapta automaticamente a qualquer época em que for executado, tornando-se mais flexível e conveniente para quem usá-lo como ferramenta de planejamento temporal.

Além dos operadores lógicos, de comparação e numéricos, a linguagem Python tem outras categorias de operadores. Entre as principais categorias de operadores disponíveis na linguagem, destacam-se:

- **Operadores de atribuição:** usados para atribuir valores a variáveis. Dentro dessa categoria está o operador `+=`, que é um operador de atribuição composto. Ele realiza a operação de soma e atribuição ao mesmo tempo, ou seja, `x += 5` é equivalente a `x = x + 5`. Outros operadores dessa categoria incluem `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, entre outros, que combinam diferentes operações matemáticas com a atribuição.
- **Operadores de identidade:** verificam se dois objetos referenciam o mesmo local na memória. Os operadores dessa categoria são `is` e `is not`. Por exemplo, `a is b` retorna `True` se `a` e `b` referenciam o mesmo objeto.
- **Operadores de associação:** testam se um elemento está presente em uma sequência, como listas, strings ou dicionários. Esses operadores são `in` e `not in`. Por exemplo, `'a' in 'Python'` retorna `True` se `a` estiver na string `Python`.

- **Operador de membro** (dot operator): o ponto `.` é usado para acessar atributos ou métodos de objetos, bem como para navegar por módulos e classes. Por exemplo: `objeto.metodo()` chama o método de um objeto e `modulo.classe` acessa uma classe dentro de um módulo.
- **Operadores de indexação e fatiamento**: usam colchetes `[]` para acessar elementos em sequências como listas, tuplas, strings e dicionários. O fatiamento permite acessar subpartes de sequências. Exemplos: `lista[0]` para acessar o primeiro elemento, ou `lista[1:4]` para obter uma fatia.
- **Operador de chamada**: os parênteses `()` são usados para chamar funções, métodos e classes. Exemplos: `funcao(arg1, arg2)` ou `Classe()`. Este operador também é usado para definir precedência em expressões matemáticas.
- **Operador de desempacotamento**: o asterisco `*` e o duplo asterisco `**` são usados para desempacotar elementos de listas, tuplas ou dicionários. Exemplo: `def funcao(*args, **kwargs):`

Contudo, nosso viajante não quer somente saber o salto temporal. É igualmente importante compreender o período histórico mais amplo em que o ano da chegada se insere. Determinar a década e o século pode fornecer ao viajante uma visão geral do contexto cultural, político e tecnológico que o aguarda. Assim, se um viajante seleciona um ano específico para visitar, saber se ele pertence à década de 1920 ou de 1990, ao século XV ou XX, pode ajudar a prever o ambiente que irá encontrar ao emergir naquele ponto do passado ou do futuro. O código da figura a seguir recebe um ano, realiza operações matemáticas simples e informa a década e o século atual:



```
main.py +
1 # Solicita ao usuário que insira o ano de destino
2 ano_destino = int(input("Insira o ano de destino: "))
3
4 # Determina a década
5 # Ao dividir o ano por 10 (inteiro) e multiplicar novamente por 10, obtemos a década base.
6 # Por exemplo, 1995 // 10 = 199, 199 * 10 = 1990, logo a década é a de 1990.
7 decada = (ano_destino // 10) * 10
8
9 # Determina o século
10 # O primeiro século vai do ano 1 ao ano 100, o segundo do ano 101 ao 200, e assim por diante.
11 # A fórmula (ano - 1) // 100 + 1 calcula corretamente o século.
12 # Por exemplo, para o ano 2024: (2024 - 1) // 100 + 1 = (2023 // 100) + 1 = 20 + 1 = 21, século XXI.
13 seculo = (ano_destino - 1) // 100 + 1
14
15 # Exibe o resultado
16 print(f"O ano {ano_destino} pertence à década de {decada} e ao século {seculo}.")
```

Ln: 17, Col: 1

Run Share Command Line Arguments

Insira o ano de destino:  
2887  
O ano 2887 pertence à década de 2880 e ao século 29.

Figura 69 – Exibindo a década e o século

Ao começar, o código solicita que o usuário insira o ano de destino através da função `input()`, convertendo a entrada em um inteiro com `int()`. Esse valor é armazenado na variável `ano_destino`. Suponha que o usuário insira o valor 2887, indicando que pretende viajar para esse ano específico.

Para encontrar a década, o código utiliza a expressão `(ano_destino // 10) * 10`. Aqui, o operador `//` realiza a divisão inteira, descartando qualquer parte fracionária. Dividir o ano por 10 reduz o valor ao número inteiro correspondente à quantidade de décadas desde o ano 0. Multiplicar novamente por 10 "reconstrói" a década base. Por exemplo, para 2887,  $2887 // 10 = 288$ , e  $288 * 10 = 2880$ , indicando que 2887 pertence à década de 2880. Esse método funciona para qualquer ano, garantindo a identificação da década correta.

O cálculo do século usa a fórmula `(ano_destino - 1) // 100 + 1`. A lógica é a seguinte: o primeiro século é do ano 1 ao 100, o segundo do ano 101 ao 200, e assim por diante. Subtrair 1 do ano antes de dividi-lo por 100 alinha a contagem de séculos de forma que o ano 1 pertença ao primeiro século, o ano 100 também ao primeiro, o ano 101 ao segundo, e assim sucessivamente. Para 2887,  $(2887 - 1) // 100 + 1 = (2886 // 100) + 1 = 28 + 1 = 29$ , logo o século é o XXIX.

Por fim, a função `print()` exibe o resultado, informando ao usuário em qual década e século está o ano de destino. Na execução da figura anterior (parte inferior), escolhemos o ano 2887 e a mensagem resultante foi: "O ano 2887 pertence à década de 2880 e ao século 29."

Essa informação é valiosa para o viajante do tempo, permitindo um melhor entendimento do período histórico que ele está prestes a explorar.

Com vimos, a utilização de variáveis e operadores em Python é fundamental para a lógica e a manipulação de dados em programas. As variáveis atuam como recipientes dinâmicos que armazenam informações, enquanto os operadores fornecem as ferramentas necessárias para realizar cálculos, tomar decisões e controlar o fluxo do programa. Essa combinação de simplicidade e poder torna Python uma linguagem acessível para iniciantes, ao mesmo tempo que oferece recursos robustos para programadores experientes desenvolverem soluções complexas.

### 4 FUNÇÕES EM PYTHON

As funções desempenham um papel central na organização e reutilização de código, sendo um dos pilares da programação estruturada e funcional. Elas permitem agrupar instruções logicamente relacionadas em blocos de código reutilizáveis, promovendo a clareza e a eficiência no desenvolvimento de aplicações. Uma função é definida por meio da palavra-chave `def`, seguida pelo nome da função, parâmetros opcionais e um corpo que contém as instruções a serem executadas. Ao encapsular uma lógica específica dentro de uma função, é possível chamar esse bloco repetidamente em diferentes partes de um programa, evitando a duplicação de código e facilitando a manutenção.

Além de melhorar a modularidade do código, as funções permitem a criação de estruturas mais robustas, contribuindo para a redução de erros e para a simplificação do processo de depuração. Em Python elas podem ser armazenadas em variáveis, passadas como argumentos para outras funções e até mesmo retornadas como resultados.

Um programador usa funções em um programa da mesma forma que alguém organiza receitas em um livro de culinária, priorizando praticidade, organização e eficiência. Imagine que você está preparando vários pratos para um jantar especial, e todos eles utilizam o mesmo molho caseiro como ingrediente. Em vez de anotar as etapas para fazer o molho repetidamente em cada receita, você cria uma única receita separada dedicada ao preparo do molho. Sempre que um prato precisar desse molho, basta consultar essa página, sem a necessidade de reescrever ou lembrar todas as etapas do processo.

Essa analogia reflete perfeitamente o propósito de uma função em programação. Uma função encapsula um conjunto de instruções que realizam uma tarefa específica, como calcular algo, organizar dados ou executar uma ação repetitiva. Ao chamar (ou evocar) uma função, é como recorrer à receita já escrita: você sabe exatamente como o processo será executado sem precisar repetir cada detalhe manualmente.

Além disso, funções tornam o gerenciamento de mudanças muito mais fácil. Voltando ao exemplo do livro de receitas, imagine que você decide alterar o molho, talvez adicionando um novo ingrediente ou ajustando a quantidade de temperos. Com a receita do molho centralizada em um único lugar, você faz a alteração uma única vez, e ela automaticamente se aplica a todos os pratos que dependem dela. O mesmo ocorre com funções em programação: ao ajustar a lógica encapsulada em uma função, todas as partes do programa que utilizam essa função automaticamente refletem as mudanças, economizando tempo e minimizando erros.

Por fim, a modularidade promovida pelas funções também facilita a leitura e a manutenção do programa. Assim como seria confuso incluir os detalhes do preparo do molho dentro de cada receita no livro, seria difícil compreender um código no qual cada trecho repetitivo fosse reescrito em diversos lugares. Organizar o código com funções é como manter o livro de receitas limpo e organizado, permitindo que qualquer pessoa, mesmo que nunca tenha visto o livro antes, consiga entender as instruções com clareza e replicar os pratos sem dificuldade.

## 4.1 Criação e chamada de funções

A criação de funções em Python é um processo que envolve a definição de um bloco de código que realiza uma tarefa específica e que pode ser reutilizado sempre que necessário. Para criar uma função, o programador usa a palavra-chave `def`, que indica ao interpretador que um novo bloco de código será definido. Esse comando é seguido pelo nome da função, que deve ser descritivo e indicar claramente sua finalidade. Após o nome, é necessário especificar os parênteses, que podem conter parâmetros caso a função precise receber dados para operar. Em seguida, usa-se dois-pontos para indicar que o corpo da função será apresentado na linha subsequente. O conteúdo da função, que define sua lógica, deve ser escrito com um nível de indentação consistente para que Python reconheça corretamente o escopo desse bloco de código.

Após a criação, **uma função não é executada automaticamente**. Para utilizá-la, é necessário realizar uma chamada. Essa chamada consiste em usar o nome da função seguido de parênteses. Se a função exigir parâmetros, esses valores devem ser fornecidos dentro dos parênteses no momento da chamada. Durante a execução, os dados passados como argumentos são usados para calcular ou produzir o resultado esperado. Caso a função inclua a instrução `return`, ela enviará um valor ou resultado de volta ao local onde foi chamada, permitindo que o programador armazene esse valor em uma variável ou o utilize diretamente em outra operação.

Vamos retornar ao código-fonte da figura 69, que informava a década e o século de um certo ano (que foi informado pelo usuário). Nossa tarefa agora é alterar esse código para que o século seja exibido em números romanos. No código a seguir, vamos criar uma função chamada `inteiro_para_romano` e usá-la quando necessário.

```
def inteiro_para_romano(numero):
    # Mapeamento de números inteiros para numerais romanos
    valores = [
        (1000, "M"), (900, "CM"), (500, "D"), (400, "CD"),
        (100, "C"), (90, "XC"), (50, "L"), (40, "XL"),
        (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I")
    ]
    romano = ""
    for valor, numeral in valores:
        while numero >= valor:
            romano += numeral
            numero -= valor
    return romano

# Solicita ao usuário que insira o ano de destino
ano_destino = int(input("Insira o ano de destino: "))

# Determina a década
# Ao dividir o ano por 10 (inteiro) e multiplicar novamente por 10, obtemos a
# década base.
# Por exemplo, 1995 // 10 = 199, 199 * 10 = 1990, logo a década é a de 1990.
decada = (ano_destino // 10) * 10
```

```
# Determina o século
# O primeiro século vai do ano 1 ao ano 100, o segundo do ano 101 ao 200, e
# assim por diante.
# A fórmula (ano - 1) // 100 + 1 calcula corretamente o século.
# Por exemplo, para o ano 2030: (2030 - 1) // 100 + 1 = (2029 // 100) + 1 = 20 +
# 1 = 21, século XXI.
seculo = inteiro_para_romano((ano_destino - 1) // 100 + 1)

# Exibe o resultado
print(f"O ano {ano_destino} pertence à década de {decada} e ao século
{seculo}.")
```

Figura 70

Esse código tem como objetivo converter um número inteiro em sua representação numérica romana e determinar, a partir de um ano fornecido pelo usuário, a que década e século esse ano pertence, exibindo o século no formato romano.

Nossa função `inteiro_para_romano` é responsável por realizar a conversão de números inteiros para o sistema de numeração romana. O código utiliza um mapeamento em forma de lista de tuplas, na qual cada tupla associa um valor inteiro ao numeral romano correspondente.

A lógica de conversão percorre essa lista de valores, ordenada do maior para o menor, garantindo que o maior número possível seja subtraído do valor restante do número a ser convertido. A cada subtração, o numeral romano correspondente é concatenado à string acumuladora chamada `romano`.

O loop `while` continua processando enquanto o número restante é maior ou igual ao valor da tupla em questão, garantindo que múltiplas ocorrências de um numeral sejam corretamente representadas. Ao final, a string resultante contém o número convertido para o formato romano. A função encerra (`return`), entregando o valor da variável `romano` a quem chamou a função. Ou seja, quem invocar essa função, recebe algo em troca.

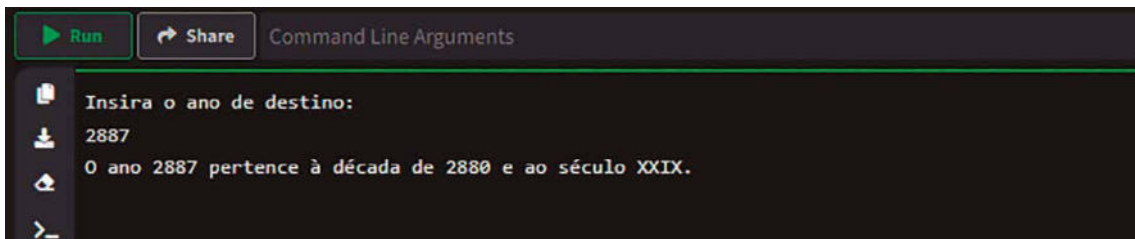
Após definir a função de conversão, o programa solicita ao usuário que insira um ano, que é então armazenado como um número inteiro na variável `ano_destino`. Em seguida, o programa calcula a década correspondente ao ano fornecido. Recapitulando, esse cálculo é feito dividindo o ano por 10 (utilizando a divisão inteira para descartar os dígitos finais), multiplicando o resultado por 10 e, assim, obtendo a base da década. Por exemplo, o ano 1995 será associado à década de 1990.

O próximo passo é determinar o século ao qual o ano pertence. A fórmula  $(ano - 1) // 100 + 1$  é usada para calcular o século de forma precisa. Subtraindo 1 do ano, garante-se que anos como 1900 sejam corretamente classificados como pertencentes ao século XIX. O resultado dessa fórmula é convertido para o formato romano utilizando a função `inteiro_para_romano`.

É exatamente nesse ponto que ocorre a **invocação da função** que havia sido criada. E é nesse momento que **a função é executada**. Ela recebe o valor  $(ano - 1) // 100 + 1$  e devolve o valor calculado com base nesse valor. O valor calculado está na variável `romano` que, após a execução da função, é atribuído na variável `seculo`.



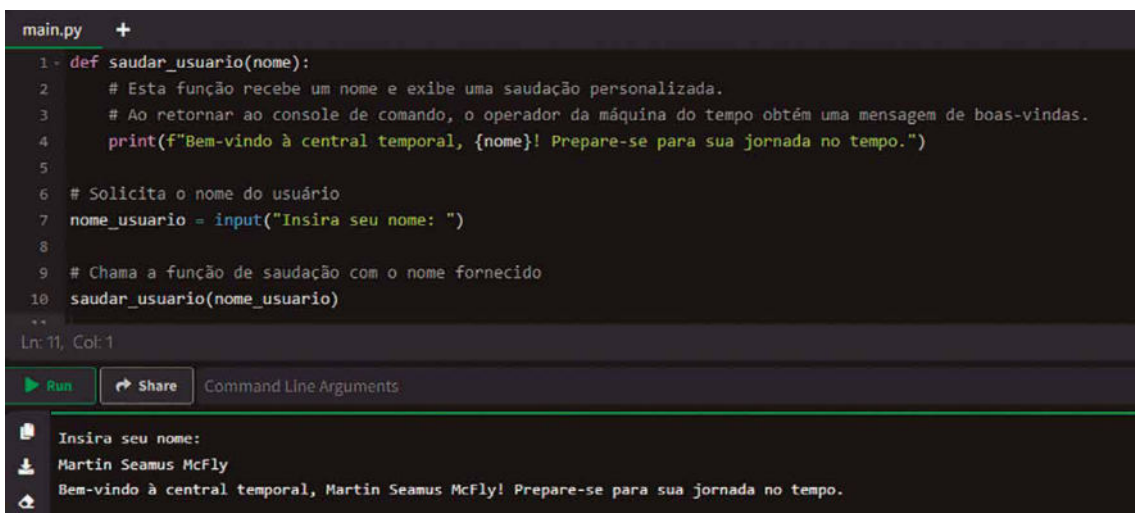
Finalmente, o programa exibe o resultado, informando o ano fornecido pelo usuário, a década correspondente e o século em sua representação numérica romana. A figura a seguir mostra o resultado da execução para o ano de 2887, que também foi usado na execução da figura anterior:



```
Run Share Command Line Arguments
Insira o ano de destino:
2887
O ano 2887 pertence à década de 2880 e ao século XXIX.
```

Figura 71 – Exibição do século em números romanos

Imagine um terminal de controle da máquina do tempo em que o operador, antes de iniciar uma jornada histórica, precisa se identificar para que o sistema possa cumprimentá-lo e criar um clima mais pessoal e receptivo. Ao informar seu nome, a função exibirá uma saudação personalizada, preparando o viajante para a aventura temporal. A figura a seguir apresenta esse exemplo:



```
main.py +
1- def saudar_usuario(nome):
2-     # Esta função recebe um nome e exibe uma saudação personalizada.
3-     # Ao retornar ao console de comando, o operador da máquina do tempo obtém uma mensagem de boas-vindas.
4-     print(f"Bem-vindo à central temporal, {nome}! Prepare-se para sua jornada no tempo.")
5-
6- # Solicita o nome do usuário
7- nome_usuario = input("Insira seu nome: ")
8-
9- # Chama a função de saudação com o nome fornecido
10- saudar_usuario(nome_usuario)
**
Ln: 11, Col: 1
Run Share Command Line Arguments
Insira seu nome:
Martin Seamus McFly
Bem-vindo à central temporal, Martin Seamus McFly! Prepare-se para sua jornada no tempo.
```

Figura 72 – Função que saúda o viajante do tempo

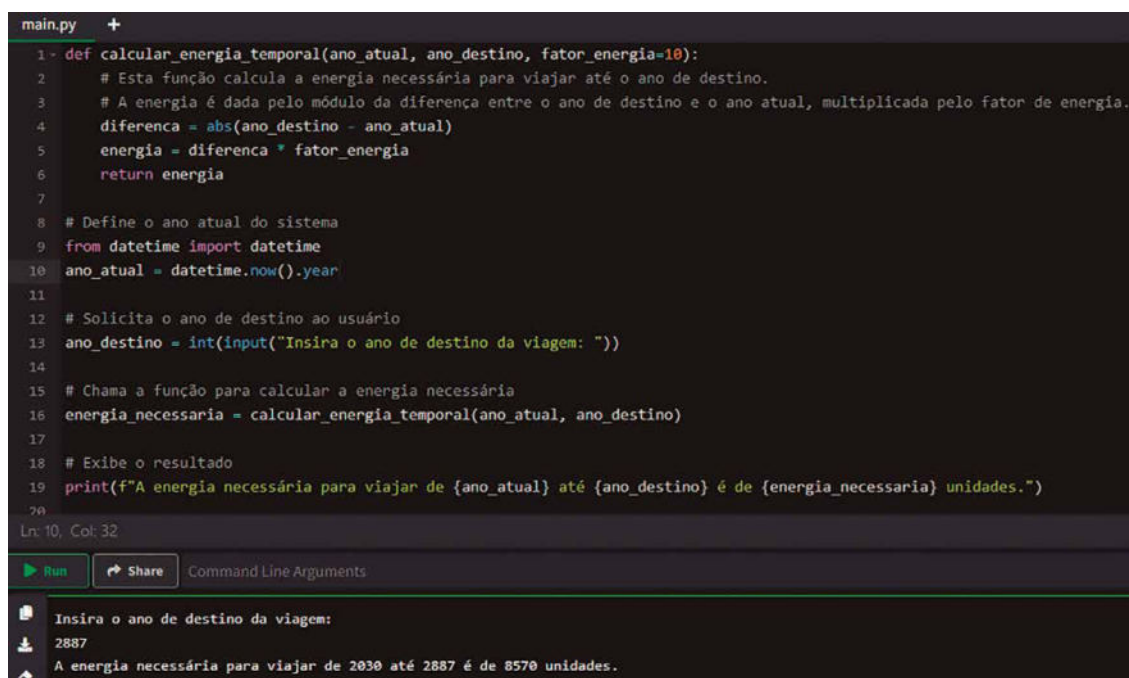
A primeira parte do código define uma função, chamada `saudar_usuario`, cuja tarefa é exibir uma mensagem de boas-vindas. A função recebe como entrada o nome do usuário, que é representado por um **parâmetro** chamado `nome`. Dentro da função, o comando `print` é usado para exibir no console uma mensagem personalizada, e o nome do usuário é incorporado na saudação. Por exemplo, se o nome for "Martin McFly", a mensagem exibida será: "Bem-vindo à central temporal, Martin McFly! Prepare-se para sua jornada no tempo."

Após a definição da função, o código principal começa pedindo ao usuário que insira seu nome. O comando `input` exibe a mensagem `Insira seu nome:` e espera que o usuário digite algo no teclado. Esse valor é armazenado na variável `nome_usuario`.



Com o nome do usuário armazenado, o código **chama a função** `saudar_usuario`, **passando o valor** de `nome_usuario` como **argumento**. Isso significa que a função será executada usando o nome fornecido pelo usuário, exibindo a saudação personalizada no console. Essa separação em uma função torna o código mais organizado e reutilizável, permitindo que a saudação seja usada novamente em outras partes do programa, se necessário.

Ainda no universo das viagens no tempo, deslocar-se por décadas ou séculos requer uma quantidade significativa de energia. A energia necessária pode variar de acordo com a distância temporal entre o ano atual e o ano de destino. Suponha que a energia necessária seja proporcional ao intervalo de anos a ser percorrido multiplicado por um fator constante. Por exemplo, se a cada ano de diferença do presente for exigido um certo número de "unidades de energia temporal", a função poderá calcular o total com base em uma fórmula simples. A figura a seguir implementa essa função:



```
main.py +
1 def calcular_energia_temporal(ano_atual, ano_destino, fator_energia=10):
2     # Esta função calcula a energia necessária para viajar até o ano de destino.
3     # A energia é dada pelo módulo da diferença entre o ano de destino e o ano atual, multiplicada pelo fator de energia.
4     diferenca = abs(ano_destino - ano_atual)
5     energia = diferenca * fator_energia
6     return energia
7
8 # Define o ano atual do sistema
9 from datetime import datetime
10 ano_atual = datetime.now().year
11
12 # Solicita o ano de destino ao usuário
13 ano_destino = int(input("Insira o ano de destino da viagem: "))
14
15 # Chama a função para calcular a energia necessária
16 energia_necessaria = calcular_energia_temporal(ano_atual, ano_destino)
17
18 # Exibe o resultado
19 print(f"A energia necessária para viajar de {ano_atual} até {ano_destino} é de {energia_necessaria} unidades.")
20
```

Ln: 10, Col: 12

Run Share Command Line Arguments

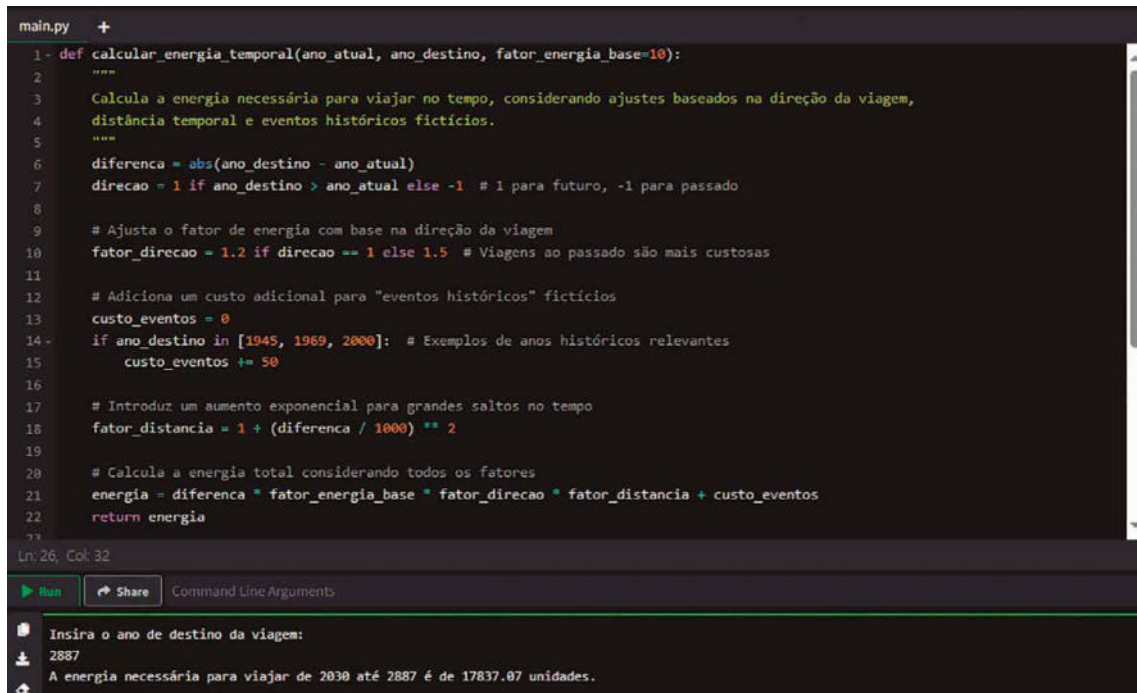
Insira o ano de destino da viagem:  
2887  
A energia necessária para viajar de 2030 até 2887 é de 8570 unidades.

Figura 73 – Função de cálculo da energia necessária em uma viagem temporal

A função `calcular_energia_temporal(ano_atual, ano_destino, fator_energia=10)` recebe como parâmetros o ano atual, o ano de destino e um fator de energia opcional (que assume o valor 10, caso não seja fornecido). Primeiramente, calculamos a diferença entre o ano destino e o atual usando `abs()` para obter o valor absoluto. Em seguida, essa diferença é multiplicada pelo fator de energia para produzir o valor total necessário. Ao retornar o valor para a variável `energia_necessaria`, o programa exibe o resultado para o usuário.

No contexto fictício da viagem no tempo, ajustes nesse fator ou na fórmula poderiam ser feitos para simular custos energéticos mais complexos, dependendo de quantos anos estão sendo percorridos, se o salto é para o passado ou para o futuro, ou mesmo se há eventos históricos que exigem mais ou menos energia para serem "atravessados". O código da figura a seguir mostra a alteração da função

calcular\_energia\_temporal para contemplar esses ajustes. Note que na parte superior da figura só está o fragmento com a função. O restante do código (que não aparece na figura, note a barra de scroll à direita) é o mesmo da figura anterior. Na execução (parte inferior da figura) utilizamos o mesmo ano de 2887. Observe como o valor da energia é maior quando comparado ao exemplo anterior:



```
main.py +
1 - def calcular_energia_temporal(ano_atual, ano_destino, fator_energia_base=10):
2     """
3     Calcula a energia necessária para viajar no tempo, considerando ajustes baseados na direção da viagem,
4     distância temporal e eventos históricos fictícios.
5     """
6     diferenca = abs(ano_destino - ano_atual)
7     direcao = 1 if ano_destino > ano_atual else -1 # 1 para futuro, -1 para passado
8
9     # Ajusta o fator de energia com base na direção da viagem
10    fator_direcao = 1.2 if direcao == 1 else 1.5 # Viagens ao passado são mais custosas
11
12    # Adiciona um custo adicional para "eventos históricos" fictícios
13    custo_eventos = 0
14    if ano_destino in [1945, 1969, 2000]: # Exemplos de anos históricos relevantes
15        custo_eventos += 50
16
17    # Introduz um aumento exponencial para grandes saltos no tempo
18    fator_distancia = 1 + (diferenca / 1000) ** 2
19
20    # Calcula a energia total considerando todos os fatores
21    energia = diferenca * fator_energia_base * fator_direcao * fator_distancia + custo_eventos
22    return energia
23
Ln: 26, Col: 32
Run Share Command Line Arguments
Insira o ano de destino da viagem:
2887
A energia necessária para viajar de 2030 até 2887 é de 17837.07 unidades.
```

Figura 74 – Função de cálculo da energia necessária em uma viagem temporal com sofisticação de cálculos

Vamos detalhar as melhorias implementadas no cálculo:

- **Direção da viagem:** viagens ao futuro têm um fator de ajuste menor (1 . 2), enquanto viagens ao passado têm um custo adicional (1 . 5), simulando a ideia fictícia de que viajar para o passado é mais difícil.
- **Eventos históricos fictícios:** um custo adicional é aplicado para anos de grande relevância histórica, como 1945 (fim da Segunda Guerra Mundial), 1969 (ano da chegada à Lua) e 2000 (marco do novo milênio). Isso sugere que eventos significativos "afetam" o fluxo do tempo e tornam a travessia mais custosa.
- **Distância no tempo:** para grandes saltos no tempo, um aumento exponencial no custo é introduzido, com o fator  $fator\_distancia = 1 + (diferenca / 1000) ** 2$ . Esse fator extra cria uma penalidade crescente para viagens de longa distância, tornando-as mais caras.



### Observação

Python permite o uso de acentos, cedilhas e outros caracteres especiais em nomes de variáveis e funções, desde que estejam em conformidade com as regras gerais de nomenclatura. Isso inclui o uso de letras Unicode para suportar caracteres de diferentes idiomas.

No entanto, embora seja tecnicamente permitido, o uso de acentos e cedilhas em nomes de variáveis e funções não é uma prática comum, principalmente em contextos internacionais. Nem todos os teclados e sistemas têm suporte fácil para esses caracteres, o que pode dificultar a colaboração e manutenção do código por parte de outros desenvolvedores. Por esse motivo estamos nomeando, por exemplo, a variável diferença como `diferenca`, a distância como `distancia` e a direção como `direcao`. Manteremos esse padrão em todos os códigos deste livro-texto.

Como acentuamos anteriormente, a antecipação de paradoxos em viagens temporais é uma tarefa que exige lógica cautelosa e capacidade de prever situações extremamente contraintuitivas. Quando um viajante do tempo intervém em determinados eventos históricos ou interage consigo mesmo em um passado distante, corre-se o risco de comprometer a coerência da linha temporal, criando contradições insolúveis. Para auxiliar nesse desafio, é possível encapsular a lógica de detecção de paradoxos em uma função, tornando a análise mais clara e modular. Ao criar uma função dedicada a verificar se uma ação específica acarretará um paradoxo, o operador da máquina do tempo pode simplesmente fornecer as condições do evento em questão e receber um veredito. Esse processo pode evitar alterações catastróficas na história, permitindo ao viajante tomar decisões mais informadas antes de pressionar o botão que o lançará para épocas remotas.

A seguir, é apresentado um código que implementa tal função. Similarmente ao que fizemos no exemplo da figura 31, o código seguinte considera três parâmetros que, na ficção da viagem temporal, são frequentemente associados à ocorrência de paradoxos: a interferência direta com um antepassado, a alteração de um evento historicamente crucial e a interação direta com uma versão mais jovem de si mesmo. Caso a combinação dessas condições ultrapasse certos limiares, o risco de paradoxo se torna inevitável. Além disso, a função pode avaliar grau de risco, retornando não apenas um resultado booleano, mas também uma mensagem explicativa detalhando o motivo da conclusão.

```
def verifica_paradoxo(interfere_ancestral, altera_evento_crucial, encontra_si_mesmo):  
    # Esta função analisa as condições fornecidas para determinar se as ações do  
    viajante causam um paradoxo.  
    # Se o viajante interfere com um ancestral direto e, ao mesmo tempo, altera  
    um evento crucial, a lógica da linha temporal fica comprometida.  
    # Caso também encontre a si mesmo no passado, a complexidade aumenta, pois  
    interações diretas consigo mesmo podem gerar contradições imediatas.  
    # A função retorna um dicionário contendo se há paradoxo (True/False) e uma  
    mensagem explicativa.
```

```
paradoxo = False
razao = "Nenhum paradoxo detectado."

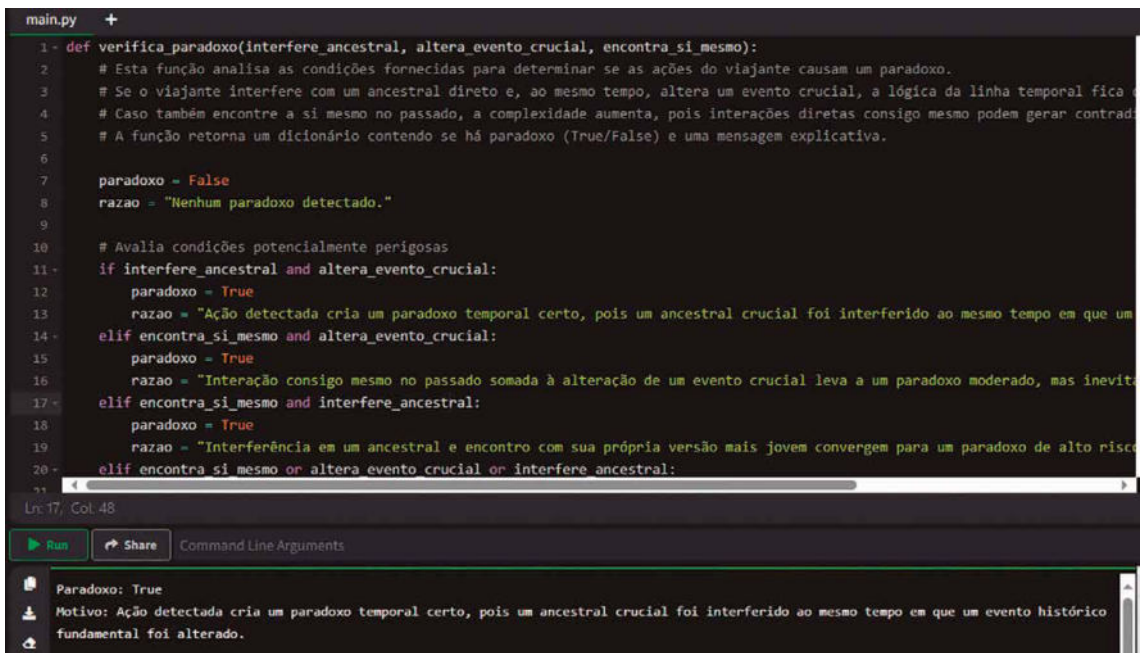
# Avalia condições potencialmente perigosas
if interfere_ancestral and altera_evento_crucial:
    paradoxo = True
    razao = "Ação detectada cria um paradoxo temporal certo, pois um ancestral crucial foi interferido ao mesmo tempo em que um evento histórico fundamental foi alterado."
elif encontra_si_mesmo and altera_evento_crucial:
    paradoxo = True
    razao = "Interação consigo mesmo no passado somada à alteração de um evento crucial leva a um paradoxo moderado, mas inevitável."
elif encontra_si_mesmo and interfere_ancestral:
    paradoxo = True
    razao = "Interferência em um ancestral e encontro com sua própria versão mais jovem convergem para um paradoxo de alto risco."
elif encontra_si_mesmo or altera_evento_crucial or interfere_ancestral:
    # Se apenas uma condição é verdadeira, não há paradoxos certos, mas sim risco.
    razao = "Risco detectado, mas sem paradoxo certo. Aconselha-se cautela."

return {"paradoxo": paradoxo, "razao": razao}

# Exemplo de uso da função:
resultado = verifica_paradoxo(interfere_ancestral=True, altera_evento_crucial=True, encontra_si_mesmo=False)
print("Paradoxo:", resultado["paradoxo"])
print("Motivo:", resultado["razao"])
```

Figura 75

Na parte inferior da figura a seguir vemos o resultado da execução desse código:



```
main.py +
1- def verifica_paradoxo(interfere_ancestral, altera_evento_crucial, encontra_si_mesmo):
2-     # Esta função analisa as condições fornecidas para determinar se as ações do viajante causam um paradoxo.
3-     # Se o viajante interfere com um ancestral direto e, ao mesmo tempo, altera um evento crucial, a lógica da linha temporal fica
4-     # Caso também encontre a si mesmo no passado, a complexidade aumenta, pois interações diretas consigo mesmo podem gerar contrad
5-     # A função retorna um dicionário contendo se há paradoxo (True/False) e uma mensagem explicativa.
6-
7-     paradoxo = False
8-     razao = "Nenhum paradoxo detectado."
9-
10-    # Avalia condições potencialmente perigosas
11-    if interfere_ancestral and altera_evento_crucial:
12-        paradoxo = True
13-        razao = "Ação detectada cria um paradoxo temporal certo, pois um ancestral crucial foi interferido ao mesmo tempo em que um
14-    elif encontra_si_mesmo and altera_evento_crucial:
15-        paradoxo = True
16-        razao = "Interação consigo mesmo no passado somada à alteração de um evento crucial leva a um paradoxo moderado, mas inevit
17-    elif encontra_si_mesmo and interfere_ancestral:
18-        paradoxo = True
19-        razao = "Interferência em um ancestral e encontro com sua própria versão mais jovem convergem para um paradoxo de alto risco
20-    elif encontra_si_mesmo or altera_evento_crucial or interfere_ancestral:
21-
22-
23-
24-
25-
26-
27-
28-
29-
30-
31-
32-
33-
34-
35-
36-
37-
38-
39-
40-
41-
42-
43-
44-
45-
46-
47-
48-
49-
50-
51-
52-
53-
54-
55-
56-
57-
58-
59-
60-
61-
62-
63-
64-
65-
66-
67-
68-
69-
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
```

Run Share Command Line Arguments

Paradoxo: True  
Motivo: Ação detectada cria um paradoxo temporal certo, pois um ancestral crucial foi interferido ao mesmo tempo em que um evento histórico fundamental foi alterado.

Figura 76 – Verificação de paradoxo temporal usando uma função

A função `verifica_paradoxo` recebe três parâmetros: `interfere_ancestral`, `altera_evento_crucial` e `encontra_si_mesmo`. Cada parâmetro é uma variável booleana que indica se determinada ação ocorre durante a viagem no tempo. Esses parâmetros definem as condições avaliadas para definir se há um paradoxo e qual é sua gravidade.

Dentro da função, duas variáveis locais são inicializadas: `paradoxo`, que é definida como `False`, e `razao`, que começa com a mensagem padrão `Nenhum paradoxo detectado`. Essas variáveis servem como o estado inicial e serão modificadas dependendo das condições fornecidas pelos parâmetros. As decisões são tomadas com uma série de instruções condicionais `if`, `elif` e `else`, cada uma representando um cenário hipotético.

Se múltiplas condições forem verdadeiras, como interferir com um ancestral direto e alterar um evento crucial, a variável `paradoxo` é atualizada para `True`, e a variável `razao` recebe uma mensagem explicativa detalhada. Outros cenários, como encontrar a si mesmo no passado e alterar um evento crucial, também resultam em paradoxos, mas com mensagens diferentes para descrever o risco específico. Se apenas uma condição for verdadeira, a função considera que há um risco, mas não um paradoxo definitivo, e ajusta a variável `razao` para refletir essa situação.

O retorno da função utiliza um dicionário, que é uma estrutura de dados versátil para armazenar pares de chave-valor. Veremos essa estrutura com mais detalhes depois. Ela permite que a função retorne tanto o estado do paradoxo (`True` ou `False`) quanto a explicação correspondente. O uso do dicionário como retorno é uma escolha prática, pois permite que o chamador acesse diretamente cada aspecto do resultado de forma clara e organizada, como em `resultado["paradoxo"]` para o estado do paradoxo e `resultado["razao"]` para a mensagem.

Na parte final do código, a função é chamada com valores específicos para os parâmetros, e o resultado é armazenado na variável `resultado`. A impressão do resultado utiliza o comando `print`, acessando os valores do dicionário retornado por meio de suas chaves. A mensagem é formatada de maneira que primeiro exibe o estado do paradoxo e, em seguida, a explicação associada, proporcionando uma saída clara e informativa.

A tarefa de converter anos em diferentes unidades de tempo pode parecer simples quando se lida apenas com um número abstrato de anos. No entanto, o cenário da viagem no tempo frequentemente envolve datas e horas específicas, já que o viajante pode querer saber quanto tempo ainda falta para determinado evento histórico ou quantos anos já se passaram desde a última vez em que realizou um salto temporal.

Ao introduzir datas no formato `ddMMAAAA hh:mm:ss` – por exemplo, `"25091980 13:45:30"` representando o dia 25 de setembro de 1980, às 13 horas, 45 minutos e 30 segundos –, a complexidade aumenta. Agora é preciso interpretar a string fornecida, convertê-la em um objeto de data e hora, calcular a diferença para o tempo atual e finalmente transformar esse intervalo em anos fracionários para, então, decompor esse total de anos em meses, dias, horas, minutos e segundos.

Ao adotar essa abordagem, o programa permite que o operador da máquina do tempo insira uma data específica no passado ou no futuro e veja quanto tempo isso representa em diferentes escalas temporais. Esse tipo de cálculo poderia ser importante, por exemplo, para estimar o custo energético de um salto temporal longo ou para avaliar quantos eventos históricos intermediários precisariam ocorrer entre a data atual e o ponto escolhido.

No código a seguir, primeiramente é definida uma função `converter_anos(anos)` que, dada uma quantidade de anos (possivelmente fracionária), retorna um dicionário contendo a quantidade aproximada de meses, dias, horas, minutos e segundos equivalentes àquele total de anos. Em seguida, o programa solicita que o usuário insira uma data no formato `ddMMAAAA hh:mm:ss` - sem separadores adicionais além do espaço entre a parte da data e a parte do horário. O código então utiliza a biblioteca `datetime` para interpretar essa string e transformá-la em um objeto `datetime`. Em posse desse objeto, o programa calcula a diferença entre a data fornecida e a data atual do sistema, obtendo assim um valor em segundos. Esse valor em segundos é convertido em anos fracionários ao dividir pelo total de segundos em um ano simplificado (365 dias, 24 horas por dia, 60 minutos por hora, 60 segundos por minuto). Por fim, o valor resultante em anos é passado para a função `converter_anos()` para exibir as unidades correspondentes.

No exemplo a seguir, a função converte anos inteiros ou fracionários em meses, dias, horas, minutos e segundos, assumindo um ano de 365 dias e ignorando anos bissextos para manter o cálculo simples. Entretanto, a estrutura da função foi pensada para ser facilmente ajustável, de modo que, se o operador precisar lidar com calendários distintos, poderá fazê-lo inserindo fatores de correção ou parâmetros adicionais.

```
from datetime import datetime

def converter_anos(anos):
    # Esta função recebe um valor em anos (que pode ser inteiro ou fracionário)
    # e converte-o
    # em múltiplas unidades de tempo: meses, dias, horas, minutos e segundos.
    #
    # Suponha um ano com 365 dias (desconsiderando anos bissextos), 12 meses,
    # 24 horas por dia, 60 minutos por hora e 60 segundos por minuto.
    # Esses valores são mantidos constantes para simplificar o cálculo.
    #
    # O resultado é retornado na forma de um dicionário, tornando mais fácil
    # extrair apenas
    # as unidades necessárias em outros trechos do código.

    meses_por_ano = 12
    dias_por_ano = 365
    horas_por_dia = 24
    minutos_por_hora = 60
    segundos_por_minuto = 60

    # Cálculo das unidades
    meses = anos * meses_por_ano
    dias = anos * dias_por_ano
    horas = dias * horas_por_dia
```



```
minutos = horas * minutos_por_hora
segundos = minutos * segundos_por_minuto

return {
    "meses": meses,
    "dias": dias,
    "horas": horas,
    "minutos": minutos,
    "segundos": segundos
}

# Solicita a data de destino do usuário.
# O formato estipulado é ddMMAAAA hh:mm:ss, por exemplo: "25091980 13:45:30"
# Significa: 25/09/1980 às 13h45min30s.
#
# Note que não há barras ou outros separadores na parte da data.
# Por isso, o parsing abaixo precisa de cuidado no uso do.strptime.
#
# Usaremos o formato '%d%m%Y %H:%M:%S' no qual:
# %d = dia (01-31)
# %m = mês (01-12)
# %Y = ano com quatro dígitos
# %H = hora (00-23)
# %M = minuto (00-59)
# %S = segundo (00-59)
#
# Ao ler a entrada do usuário, assumimos que está no formato correto. Caso
# contrário,
# seria necessário adicionar tratamento de erros.
data_str = input("Insira a data de destino no formato ddMMAAAA hh:mm:ss: ")

# Converte a string em um objeto datetime
data_destino = datetime.strptime(data_str, "%d%m%Y %H:%M:%S")

# Obtém a data e hora atuais do sistema
data_atual = datetime.now()

# Calcula a diferença entre a data de destino e a data atual
# O resultado é um objeto timedelta, a partir do qual extraímos o total de
# segundos.
diferenca = data_destino - data_atual
segundos_diferenca = diferenca.total_seconds()

# Agora que temos a diferença em segundos, queremos expressá-la em anos.
# Usamos um ano padrão de 365 dias. Assim:
# 1 ano = 365 dias
# 1 dia = 24 horas
# 1 hora = 3600 segundos (60 minutos * 60 segundos)
# Logo, segundos em um ano = 365 * 24 * 3600 = 31536000 segundos.
segundos_por_ano = 365 * 24 * 3600

# Converte a diferença em anos fracionários
anos_diferenca = segundos_diferenca / segundos_por_ano

# Chama a função para converter essa quantidade de anos em outras unidades
resultado_conversao = converter_anos(abs(anos_diferenca)) # abs() para lidar com
negativo se data_destino < data_atual
```



```
# Exibição do resultado:
# Aqui, escolhemos exibir todas as unidades. Caso seja necessário,
# poderíamos formatar a saída de forma mais elaborada, por exemplo,
# mostrando apenas as unidades que fazem sentido.
print("\nDiferença temporal:")
print(f"{abs(anos_diferenca):.4f} anos aproximadamente.")
print(f"{resultado_conversao['meses']:.2f} meses")
print(f"{resultado_conversao['dias']:.2f} dias")
print(f"{resultado_conversao['horas']:.2f} horas")
print(f"{resultado_conversao['minutos']:.2f} minutos")
print(f"{resultado_conversao['segundos']:.2f} segundos")

if anos_diferenca > 0:
    print("A data de destino está no futuro em relação à data atual.")
elif anos_diferenca < 0:
    print("A data de destino está no passado em relação à data atual.")
else:
    print("A data de destino é exatamente a mesma que a data atual.")
```

Figura 77

Após a definição da função, o usuário fornece a data no formato `ddMMAAAA hh:mm:ss`. Utilizamos o método `datetime.strptime` com a máscara de formatação `"%d%m%Y %H:%M:%S"` para transformar a string em um objeto `datetime`. É fundamental escolher o padrão de formatação correto, já que a ausência de separadores (como barras ou hífens) exige cuidado ao indicar o mês e o ano. A chamada `datetime.now()` retorna a data e hora atuais do sistema. Assim, temos dois objetos `datetime`: um representando o momento atual (`data_atual`) e outro indicando o destino temporal (`data_destino`).

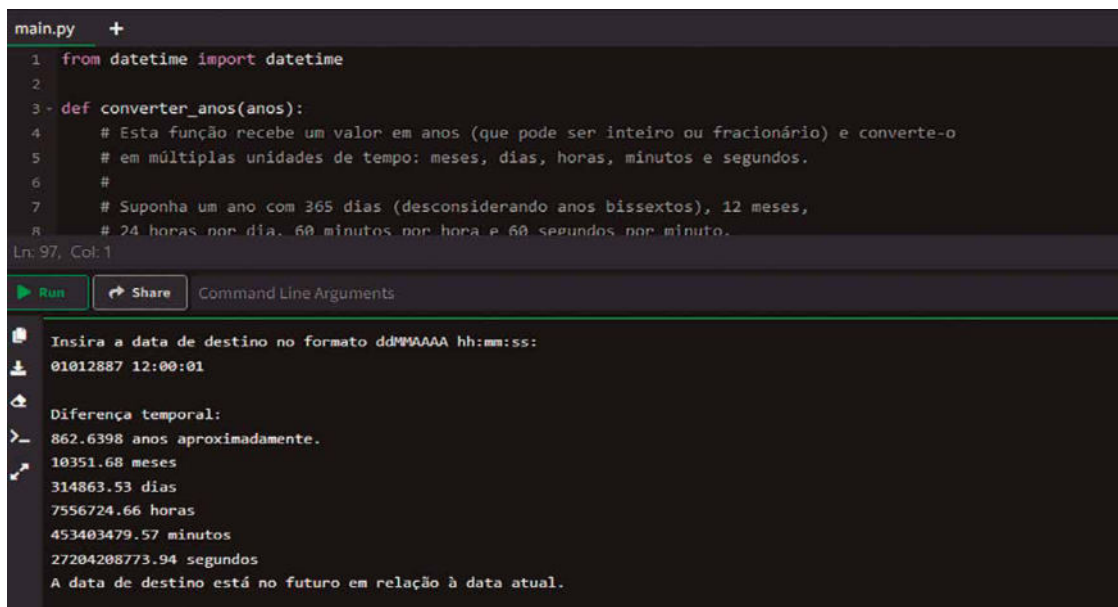
Ao subtrair `data_atual` de `data_destino`, obtemos um objeto `timedelta` que expressa a diferença entre as duas datas em dias, segundos e microssegundos. O método `timedelta.total_seconds()` converte essa diferença em um valor de ponto flutuante representando o total de segundos. Esse total de segundos é então armazenado em `segundos_diferenca`.

Para converter segundos em anos, assumimos um ano padrão de 365 dias (sem anos bissextos), resultando em 31.536.000 segundos por ano. Dividir `segundos_diferenca` por `segundos_por_ano` fornece `anos_diferenca`, um valor que pode ser positivo (futuro), negativo (passado) ou zero (mesma data).

A função `converter_anos(anos)` recebe `anos_diferenca` (em valor absoluto, para simplificar a interpretação das unidades) e retorna um dicionário com meses, dias, horas, minutos e segundos equivalentes. Dentro dessa função, assume-se um ano com 365 dias, 12 meses, 24 horas/dia, 60 minutos/hora e 60 segundos/minuto. Esse conjunto de suposições simplifica o cálculo, embora possa ser ajustado para casos mais realistas ou complexos.

Após obter o dicionário com as unidades de tempo, o código imprime os valores formatados, indicando quantos meses, dias, horas, minutos e segundos correspondem àquela diferença de tempo em anos. Em seguida, verifica-se o sinal de `anos_diferenca` para informar se a data destino está

no passado, no futuro ou coincide exatamente com a data atual. A parte inferior da figura a seguir apresenta o resultado da execução para 01012887 12:00:01:



```
main.py +
1 from datetime import datetime
2
3 def converter_anos(anos):
4     # Esta função recebe um valor em anos (que pode ser inteiro ou fracionário) e converte-o
5     # em múltiplas unidades de tempo: meses, dias, horas, minutos e segundos.
6     #
7     # Suponha um ano com 365 dias (desconsiderando anos bissextos), 12 meses,
8     # 24 horas por dia, 60 minutos por hora e 60 segundos por minuto.
Ln: 97, Col: 1

Run Share Command Line Arguments

Insira a data de destino no formato ddMMAAAA hh:mm:ss:
01012887 12:00:01

Diferença temporal:
862.6398 anos aproximadamente.
10351.68 meses
314863.53 dias
7556724.66 horas
453403479.57 minutos
27204208773.94 segundos
A data de destino está no futuro em relação à data atual.
```

Figura 78 – Resultado da execução do programa para 01012887 12:00:01

Ao final, o programa oferece uma visão detalhada e flexível do intervalo temporal entre a data inserida e o momento atual, apresentada em diversas unidades de tempo. Esse tipo de conversão pode ser extremamente útil para planejamento de saltos temporais, estudos de eventos históricos em escala minuciosa ou apenas para compreender a magnitude temporal entre dois pontos cronológicos.

### 4.2 Passagem de parâmetros e retorno

Vimos que as funções são ferramentas vitais que permitem organizar e reutilizar blocos de código para realizar tarefas específicas. Dois conceitos essenciais para entender como as funções trabalham são a passagem de parâmetros e o retorno de valores. Esses mecanismos são o que tornam as funções flexíveis, permitindo que sejam personalizadas de acordo com as necessidades do programa.

Quando uma função é definida, ela pode incluir parâmetros em sua assinatura. O termo **assinatura** refere-se à forma como uma função é declarada, incluindo seu nome e os parâmetros que ela espera receber. Em outras palavras, a assinatura de uma função é como sua “identidade formal”, que descreve como ela deve ser chamada e quais informações precisam ser fornecidas para que ela funcione corretamente.

A assinatura de uma função é composta pelo nome da função e pela lista de parâmetros **dentro de parênteses**. Esses parâmetros são variáveis que a função utiliza para realizar suas operações. Eles indicam que a função está preparada para receber valores de entrada, conhecidos como argumentos, quando for chamada. Assim, a assinatura define a interface da função, especificando de que maneira ela interage com o restante do programa.

Por exemplo, se uma função tem uma assinatura que inclui dois parâmetros – como soma (a, b) – ela espera receber dois valores quando for chamada. Esses valores serão usados internamente pela função. A assinatura, portanto, serve como um contrato, informando ao programador como utilizar a função corretamente, quais entradas ela exige e, muitas vezes, dando pistas sobre o propósito da função com base nos nomes dos parâmetros.

Esses parâmetros são como “variáveis temporárias” que recebem valores do código que chama a função. Pense nos parâmetros como recipientes que a função usa para trabalhar com os dados fornecidos pelo usuário ou pelo programa.

A **passagem de parâmetros** em Python é feita **por referência**, mas o comportamento pode variar dependendo do tipo de dado passado. **Para objetos imutáveis**, como números, strings ou tuplas, **qualquer modificação feita dentro da função não altera o objeto original**. Isso ocorre porque, ao alterar o valor, Python cria um novo objeto em vez de modificar o existente. Por outro lado, **para objetos mutáveis**, como listas ou dicionários, **alterações feitas dentro da função podem impactar o objeto original fora dela, pois ambos compartilham a mesma referência**.

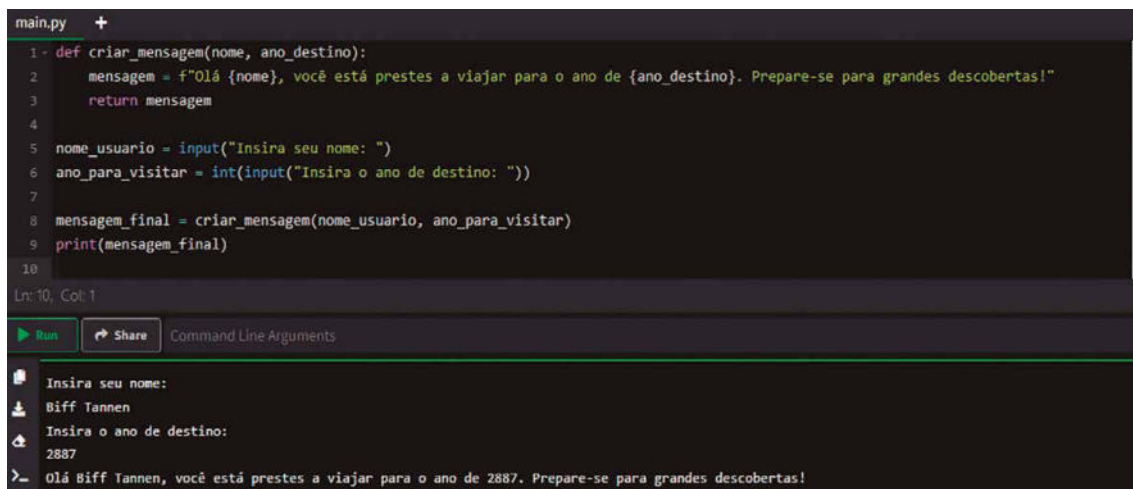
O **retorno de valores** é outro aspecto importante das funções. Quando uma função realiza sua tarefa, ela pode devolver um resultado usando a palavra-chave `return`. Esse valor retornado pode ser usado diretamente, armazenado em uma variável ou até mesmo passado para outra função. Por exemplo, uma função que calcula o dobro de um número pode ser definida como `def dobro(x): return x * 2`. Quando chamada, como em `resultado = dobro(5)`, a função executa sua lógica, calcula  $5 * 2$ , e retorna o valor 10, que é então armazenado na variável `resultado`.

Em Python, uma função pode retornar múltiplos valores ao mesmo tempo, uma vez que a linguagem permite que valores sejam retornados como tuplas, listas ou dicionários. Por exemplo, uma função que realiza várias operações em um número pode retornar os resultados de todas elas de uma vez, como `return soma, produto, diferenca`.

É importante entender que a passagem de parâmetros seria o equivalente a fornecer ingredientes para uma receita, enquanto o retorno é o prato pronto que a função devolve após processar esses ingredientes. Essa analogia ajuda a compreender o fluxo lógico de uma função: os valores são fornecidos na chamada, processados internamente e, se necessário, o resultado é devolvido para uso posterior. Essa combinação de passagem de parâmetros e retorno de valores é o que torna as funções indispensáveis para criar programas organizados, reutilizáveis e eficientes.

A ideia de criar uma mensagem personalizada para o viajante do tempo surge da necessidade de fornecer uma experiência mais envolvente e informativa. Ao ingressar na cabine da máquina temporal, o usuário não quer apenas inserir dados; ele deseja ser reconhecido, saber para qual época seguirá e sentir-se confiante de que tudo está sob controle. Uma simples mensagem pode fazer a diferença, mostrando o nome do viajante e o ano de destino de maneira clara, assegurando que o sistema entendeu corretamente suas escolhas.

Na figura a seguir está o código-fonte em Python que implementa essa funcionalidade. Nele, será definida uma função que recebe dois argumentos, o nome do viajante e o ano de destino, e retorna uma mensagem personalizada. Em seguida, o código solicitará que o usuário informe seu nome e o ano que deseja visitar, chamará a função com esses dados e exibirá o resultado retornado.



```
main.py +
1 def criar_mensagem(nome, ano_destino):
2     mensagem = f"Olá {nome}, você está prestes a viajar para o ano de {ano_destino}. Prepare-se para grandes descobertas!"
3     return mensagem
4
5 nome_usuario = input("Insira seu nome: ")
6 ano_para_visitar = int(input("Insira o ano de destino: "))
7
8 mensagem_final = criar_mensagem(nome_usuario, ano_para_visitar)
9 print(mensagem_final)
10
Ln: 10, Col: 1
Run Share Command Line Arguments
Insira seu nome:
Biff Tannen
Insira o ano de destino:
2887
Olá Biff Tannen, você está prestes a viajar para o ano de 2887. Prepare-se para grandes descobertas!
```

Figura 79 – Função que cria uma mensagem personalizada para o viajante

A primeira linha, `def criar_mensagem(nome, ano_destino):`, define uma função chamada `criar_mensagem` que recebe dois parâmetros: `nome`, uma string representando o nome do usuário, e `ano_destino`, um inteiro que indica o ano para o qual o viajante deseja se deslocar no tempo. O ato de definir uma função com argumentos permite que o desenvolvedor capture dados externos e, dentro do corpo da função, gere resultados adaptados a esses dados.

Dentro da função, a linha `mensagem = f"Olá {nome}, você está prestes a viajar para o ano de {ano_destino}. Prepare-se para grandes descobertas!"` constrói uma string utilizando uma f-string.

A linha `return mensagem` encerra a função `criar_mensagem`. O `return` devolve o valor da variável `mensagem` para o ponto no qual a função foi chamada. **Essa separação** entre a criação da mensagem (**dentro da função**) e a sua exibição (**fora da função**) é importante, pois a função agora não depende de um contexto externo para mostrar o resultado. Ela apenas prepara e fornece a mensagem, deixando para quem chamou a função a responsabilidade de decidir como usá-la, seja exibindo-a na tela, armazenando-a em um arquivo ou enviando-a por uma rede quântica entre épocas remotas.

A próxima linha, `nome_usuario = input("Insira seu nome: ")`, solicita ao usuário que forneça seu nome. A função `input()` pausa a execução do programa até que o usuário digite algo e pressione Enter. Tudo o que for digitado será armazenado na variável `nome_usuario`.

A linha `ano_para_visitar = int(input("Insira o ano de destino: "))` segue uma lógica semelhante, mas, além de pedir ao usuário para inserir o ano, envolve a conversão do valor inserido para um inteiro, através da função `int()`. Isso acontece porque o `input()` retorna sempre uma string, e, para realizar qualquer tipo de validação ou comparação numérica, é importante trabalhar com o tipo inteiro. O contexto da viagem

no tempo exige que a informação do ano seja numérica, pois a máquina do tempo se orienta por valores cronológicos precisos.

Ao ter tanto `nome_usuario` quanto `ano_para_visitar` definidos, o programa executa `mensagem_final = criar_mensagem(nome_usuario, ano_para_visitar)`. Aqui, a função `criar_mensagem` é chamada com os argumentos capturados anteriormente. A função é executada, cria a string personalizada e a retorna, sendo esse valor retornado atribuído à variável `mensagem_final`. Esse momento demonstra o poder do `return`: a função executa códigos internos e devolve um resultado concreto para ser usado depois.

Finalmente, `print(mensagem_final)` exibe a mensagem personalizada na tela. Nesse ponto, o viajante do tempo já forneceu seu nome e o ano desejado, a função já processou essas informações e construiu a mensagem, e agora o programa mostra o resultado final. Essa comunicação direta com o usuário exemplifica como funções, argumentos e retornos trabalham em conjunto para criar aplicações interativas, úteis e personalizadas.

A preparação, a execução e o término de uma viagem no tempo não se resumem a uma única ação. Deve-se coordenar etapas distintas, cada uma com tarefas e objetivos bem definidos. Antes de qualquer salto cronológico, o viajante precisa se preparar, assegurando que as condições técnicas e psicológicas estejam em ordem. Em seguida, a própria execução da viagem requer transformar as especificações de destino e os ajustes do maquinário em um deslocamento temporal real. Por fim, uma vez atingido o ano de destino, é crucial confirmar que a chegada ocorreu no ponto correto e que as condições ambientais, históricas e culturais estão de acordo com o esperado. Separar essas fases em funções diferentes não apenas organiza a lógica do programa, como também facilita a manutenção, o reúso de código e a expansão futura da aplicação. O encapsulamento de cada etapa em uma função distinta, com parâmetros adequados e retornos significativos, resulta em um fluxo mais claro e legível.

O código a seguir apresenta uma simulação simples desse processo. Ele **inclui três funções** principais: uma que cuida da preparação da viagem, outra que executa a viagem em si e uma terceira que trata da chegada. Cada função recebe e fornece informações necessárias à etapa seguinte, demonstrando o uso de **múltiplas funções e a passagem de parâmetros entre elas**. Dessa forma, o código modela a jornada temporal como uma série de transformações de dados, desde as decisões iniciais do viajante até a confirmação do destino alcançado.

```
from datetime import datetime

def preparar_viagem(ano_destino):
    # Esta função representa a preparação da jornada temporal.
    # Ela recebe um ano de destino como parâmetro e realiza os
    # cálculos e verificações necessários antes do salto. Por exemplo,
    # pode calcular o intervalo de tempo, checar se há energia
    # suficiente e estabelecer as coordenadas temporais baseadas no
    # ano atual.
    #
    # Aqui, obtemos o ano atual do sistema para saber quantos anos
    # separam o presente do destino.
    # Quanto maior a distância, maior o desafio energético. A função
    # retorna um dicionário com informações sobre o intervalo, a
```

```
# energia presumida e o ano destino original.

ano_atual = datetime.now().year
diferenca = ano_destino - ano_atual
energia_necessaria = abs(diferenca) * 42 # 42 é um fator arbitrário,
simbolizando complexidade

return {
    "ano_atual": ano_atual,
    "ano_destino": ano_destino,
    "diferenca": diferenca,
    "energia": energia_necessaria
}

def executar_viagem(dados_preparacao):
    # Esta função recebe o dicionário retornado pela etapa de
    # preparação, interpretando os dados e simulando o processo de
    # viagem no tempo.
    # Aqui, podemos imaginar que o tempo de viagem é proporcional à
    # diferença em anos dividida por um fator, ou que certas condições
    # alteram o comportamento.
    # Ao final, a função supõe que a viagem ocorreu. O retorno é um
    # novo dicionário, agora contendo informações sobre o sucesso do
    # deslocamento e eventuais ajustes.
    #
    # Neste exemplo, a função apenas confirma se há energia suficiente
    # para o salto, e sinaliza sucesso ou falha.

    energia_disponivel = 5000 # número fictício de unidades de energia
    armazenadas na máquina
    energia_necessaria = dados_preparacao["energia"]

    if energia_necessaria > energia_disponivel:
        # Se não houver energia suficiente, a viagem não pode ocorrer
        return {
            "sucesso": False,
            "mensagem": "Falha na viagem: energia insuficiente",
            "ano_destino": dados_preparacao["ano_destino"],
            "ano_atual": dados_preparacao["ano_atual"]
        }
    else:
        # Se a energia for suficiente, consideramos a viagem um sucesso
        return {
            "sucesso": True,
            "mensagem": "Viagem executada com sucesso",
            "ano_destino": dados_preparacao["ano_destino"],
            "ano_atual": dados_preparacao["ano_atual"]
        }

def chegar_ao_destino(dados_viagem):
    # Esta função recebe o resultado da execução da viagem.
    # Caso a viagem tenha sido bem-sucedida, a função pode simular as
    # condições de chegada, imprimir informações sobre o ano de
    # destino, e garantir que o viajante emergiu no ponto correto. Se
    # a viagem falhou, a função pode simplesmente reportar a falha.
    #
    # Neste exemplo, a função apenas verifica o campo "sucesso" e,
    # dependendo do valor, imprime mensagens adequadas.
    if dados_viagem["sucesso"]:
```



```
print(f"Chegada confirmada no ano {dados_viagem['ano_destino']}.")
print("Condições temporais estáveis. Linha do tempo integrada com
sucesso.")
else:
    print("A chegada ao destino não foi possível.")
    print("Retorne à etapa de preparação para corrigir os problemas.")

# Código principal:
# Solicita ao usuário o ano de destino e então chama cada função na
# sequência, passando os dados de uma etapa para a seguinte.
ano_desejado = int(input("Insira o ano de destino: "))

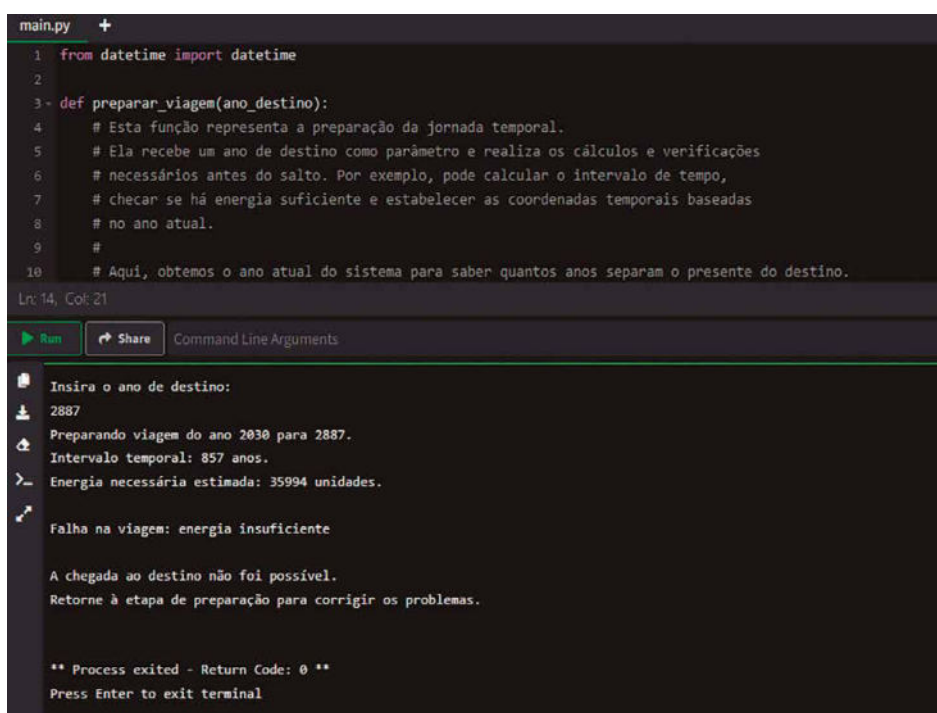
# Etapa de preparação da viagem
dados_preparacao = preparar_viagem(ano_desejado)
print(f"Preparando viagem do ano {dados_preparacao['ano_atual']} para {dados_
preparacao['ano_destino']}.")
print(f"Intervalo temporal: {dados_preparacao['diferenca']} anos.")
print(f"Energia necessária estimada: {dados_preparacao['energia']} unidades.\n")

# Etapa de execução da viagem
dados_viagem = executar_viagem(dados_preparacao)
print(dados_viagem["mensagem"], "\n")

# Etapa de chegada ao destino
chegar_ao_destino(dados_viagem)
```

Figura 80

Na figura a seguir vemos o resultado da execução do programa para o ano de 2887. Note que como a energia disponível é de apenas 5 mil unidades e a viagem demanda 35994 unidades, essa viagem não poderá ser realizada.



```
main.py +
1 from datetime import datetime
2
3 def preparar_viagem(ano_destino):
4     # Esta função representa a preparação da jornada temporal.
5     # Ela recebe um ano de destino como parâmetro e realiza os cálculos e verificações
6     # necessários antes do salto. Por exemplo, pode calcular o intervalo de tempo,
7     # checar se há energia suficiente e estabelecer as coordenadas temporais baseadas
8     # no ano atual.
9     #
10    # Aqui, obtemos o ano atual do sistema para saber quantos anos separam o presente do destino.
Ln: 14, Col: 21
Run Share Command Line Arguments
Insira o ano de destino:
2887
Preparando viagem do ano 2030 para 2887.
Intervalo temporal: 857 anos.
Energia necessária estimada: 35994 unidades.
Falha na viagem: energia insuficiente
A chegada ao destino não foi possível.
Retorne à etapa de preparação para corrigir os problemas.
** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Figura 81 – Programa com múltiplas funções e diversos argumentos



A explicação detalhada de cada parte do código é fundamental para compreender **o papel das funções e a circulação dos dados**.

A primeira função, `preparar_viagem(ano_destino)`, é chamada logo após o usuário informar o ano desejado. Ao obter o ano atual usando `datetime.now().year`, a função calcula quantos anos separam o presente do futuro ou passado pretendido. Esse valor é armazenado na variável `diferenca`. Com base nesse número, determina-se `energia_necessaria`, aqui arbitrariamente proporcional ao número de anos multiplicado por **42**. Esse valor funciona como um símbolo da complexidade técnica exigida pelo salto temporal. A função retorna um dicionário com todos esses dados consolidados, criando um pacote de informações que descreve o estado da preparação.

A segunda função, `executar_viagem(dados_preparacao)`, recebe o dicionário retornado pela primeira função. Ao analisar a quantidade de energia necessária (armazenada em `dados_preparacao["energia"]`) e compará-la com uma quantidade fictícia de `energia_disponivel`, a função decide se a viagem pode ou não ocorrer. Caso a energia seja insuficiente, retorna um dicionário sinalizando falha. Caso contrário, sinaliza sucesso. Esse padrão de retorno assegura a coesão do código e a clareza do fluxo lógico, já que a função indica explicitamente seu resultado sob a forma de chaves compreensíveis, como "sucesso" e "mensagem".

A terceira função, `chegar_ao_destino(dados_viagem)`, consolida o processo. Ao receber o dicionário resultante da execução da viagem, pode verificar o campo "sucesso" para saber se deve relatar uma chegada bem-sucedida ou uma falha. Caso a viagem tenha sido exitosa, exibe mensagens apropriadas sobre a chegada ao ano de destino. Caso contrário, informa ao usuário que a chegada não ocorreu. **Essa abordagem cria um fluxo narrativo dentro do programa**, no qual as funções funcionam como capítulos lógicos da história da viagem temporal, cada uma adicionando valor e contexto ao estado atual dos dados.

No trecho final, fora das funções, está a chamada do código principal. O usuário insere o ano de destino, e então o programa chama `preparar_viagem(ano_desejado)`, imprime algumas informações diagnósticas (como o intervalo de tempo e a energia necessária), passa esses dados para `executar_viagem`, obtém o resultado e imprime a mensagem retornada; finalmente, chama `chegar_ao_destino(dados_viagem)` para concluir a narrativa. Dessa forma, o programa exemplifica claramente o fluxo completo: **da entrada de dados à preparação, da preparação à execução, e da execução à constatação do desfecho** da jornada temporal.

A separação clara entre as três etapas é um recurso pedagógico e uma prática recomendada de engenharia de software. Ao isolar a lógica de cada etapa em sua própria função, o código torna-se mais fácil de entender, testar e modificar. Caso o operador da máquina do tempo deseje alterar a maneira como a energia é calculada, isso pode ser feito exclusivamente na função de preparação, sem afetar a execução ou a chegada. Caso seja preciso ajustar as mensagens ou a forma de apresentar resultados, a função de chegada pode ser alterada independentemente. Essa modularidade é especialmente valiosa em cenários complexos como a viagem no tempo, na qual cada etapa pode ter uma lógica intrincada e suscetível a mudanças futuras.

Quando um viajante do tempo prepara sua jornada, não basta conhecer o intervalo temporal que o separa do destino. É preciso estimar quanto tempo real a viagem consumirá no presente do operador, ou seja, quantos segundos, minutos ou horas decorrerão na perspectiva do viajante enquanto ele percorre séculos em um piscar de olhos. A noção de velocidade temporal surge da ideia de que a máquina do tempo não salta instantaneamente do presente ao destino, mas percorre o contínuo espaço-tempo a uma taxa específica. Se essa taxa for alta, a viagem parecerá quase instantânea; se for baixa, o viajante experimentará um longo período de aceleração, travessias e desaceleração.

Considere que a velocidade do deslocamento temporal possa ser expressa como "anos por segundo", ou seja, quantos anos cronológicos são percorridos em um segundo da experiência do viajante. Se a máquina do tempo deve cobrir um intervalo de 100 anos, viajando a uma velocidade de 10 anos/segundo, então a jornada dura 10 segundos. Com essa lógica simples, é possível criar uma função que receba a quantidade de anos a serem percorridos, bem como a velocidade, e retorne o tempo total da viagem. Ao encapsular esse cálculo em uma função, torna-se mais fácil experimentar diferentes velocidades, adaptando-se às necessidades de cada contexto histórico ou ao nível de energia disponível na máquina. O resultado oferecido pela função pode então ser utilizado pela interface principal do sistema, exibindo ao viajante quanto tempo levará para chegar ao seu destino.

Mostraremos a seguir o código-fonte em Python que implementa essa ideia. Após apresentar o cenário e as variáveis envolvidas, explicaremos detalhadamente cada linha, tornando claro o raciocínio subjacente ao cálculo do tempo total de viagem.

```
def calcular_tempo_viagem(anos, velocidade):
    # Esta função recebe dois parâmetros: 'anos' e 'velocidade'.
    # 'anos' representa quantos anos no contínuo histórico
    # precisam ser percorridos.
    # 'velocidade' representa quantos anos cronológicos são
    # atravessados por segundo real de viagem.
    #
    # Suponhamos que a máquina do tempo mantenha uma velocidade
    # constante.
    # O tempo total em segundos necessário para percorrer 'anos'
    # será simplesmente <anos / velocidade>.
    # Por exemplo, se anos=100 e velocidade=10 anos/segundo, o
    # tempo total será 10 segundos.
    #
    # A função retorna esse tempo total em segundos. Dessa
    # forma, o código chamador pode converter,
    # se desejar, para outras unidades de medida. É uma
    # separação de responsabilidades: aqui apenas o cálculo,
    # mais adiante a apresentação.

    tempo_em_segundos = anos / velocidade
    return tempo_em_segundos

# Parte principal do código:
# Aqui, solicita-se ao usuário o ano atual e o ano de destino,
# calculando a quantidade de anos a serem percorridos.
# Em seguida, pergunta-se a velocidade desejada, inserida como
# anos/segundo.
```

```
# Por fim, chama-se a função calcular_tempo_viagem e exibe-se o
# resultado.

from datetime import datetime
ano_atual = datetime.now().year
ano_destino = int(input("Insira o ano de destino: "))

# Calcula a diferença em anos
diferenca = ano_destino - ano_atual
# Caso a diferença seja negativa, significa viajar para o
# passado. O tempo de viagem permanece lógico,
# pois o tempo necessário se baseia na quantidade absoluta de
# anos a percorrer.
# O abs() garante que a distância seja positiva, evitando
# complicações no cálculo.
distancia_temporal = abs(diferenca)

# Solicita a velocidade da viagem.
# Por exemplo, se o usuário disser 5 anos/segundo, significa que
# a cada segundo de viagem real, a máquina percorre 5 anos
# históricos. Mais velocidade implica menos tempo de viagem.
velocidade_viagem = float(input("Insira a velocidade da viagem (anos/segundo): "))

# Chama a função para calcular o tempo total da viagem em
# segundos.
tempo_total_segundos = calcular_tempo_viagem(distancia_temporal, velocidade_viagem)

# Aqui, o código principal decide como apresentar o resultado.
# Como a função retornou o valor em segundos,
# é fácil converter para outras unidades. Por exemplo, um minuto
# tem 60 segundos, uma hora 3600 segundos.
# Isso não é tarefa da função, mas sim desta parte do código,
# mantendo a função genérica e reutilizável.

# Converte para minutos e horas, caso o usuário queira ter uma
# noção mais intuitiva.
tempo_em_minutos = tempo_total_segundos / 60
tempo_em_horas = tempo_em_minutos / 60

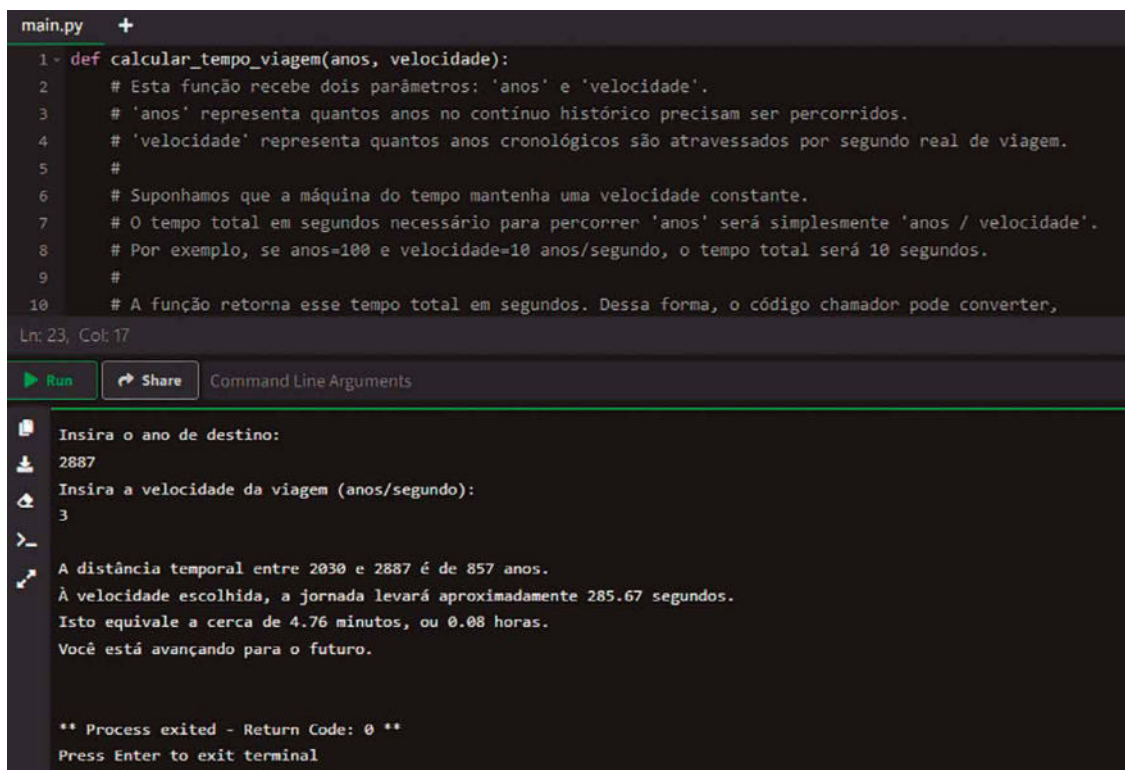
print(f"\nA distância temporal entre {ano_atual} e {ano_destino} é de
{distancia_temporal} anos.")
if tempo_total_segundos < 1:
    print("A viagem é praticamente instantânea na perspectiva do viajante!")
else:
    print(f"À velocidade escolhida, a jornada levará aproximadamente {tempo_
total_segundos:.2f} segundos.")
    if tempo_em_minutos >= 1:
        print(f"Isto equivale a cerca de {tempo_em_minutos:.2f} minutos, ou
{tempo_em_horas:.2f} horas.")

if diferenca > 0:
    print("Você está avançando para o futuro.")
elif diferenca < 0:
    print("Você está retornando ao passado.")
else:
    print("Você permanece no ano atual.")
```

Figura 82

A primeira função `calcular_tempo_viagem(anos, velocidade)` é o cerne lógico do exercício. Ela isola o cálculo necessário para definir a duração do percurso no tempo, separando a lógica de determinação do intervalo e da velocidade da lógica de exibição dos resultados. A função simplesmente divide o número de anos a percorrer pela velocidade (em anos/segundo), produzindo um resultado em segundos. A escolha de retornar o tempo apenas em segundos é proposital: manter a função simples e focada em um único cálculo.

Na parte principal do código, é obtido o ano atual do sistema e solicitado ao usuário o ano de destino. Calcula-se a diferença entre ano atual e ano de destino, determinando quantos anos separaram o presente da época escolhida. Em seguida, pede-se ao usuário a velocidade da viagem em anos/segundo. Com esses dados, chama-se `calcular_tempo_viagem`, fornecendo a distância temporal e a velocidade. O resultado é então armazenado em `tempo_total_segundos` e, a partir daí, o programa converte e exibe o resultado de forma mais amigável, como vemos na parte inferior da figura a seguir:



```
main.py +
1 def calcular_tempo_viagem(anos, velocidade):
2     # Esta função recebe dois parâmetros: 'anos' e 'velocidade'.
3     # 'anos' representa quantos anos no contínuo histórico precisam ser percorridos.
4     # 'velocidade' representa quantos anos cronológicos são atravessados por segundo real de viagem.
5     #
6     # Suponhamos que a máquina do tempo mantenha uma velocidade constante.
7     # O tempo total em segundos necessário para percorrer 'anos' será simplesmente 'anos / velocidade'.
8     # Por exemplo, se anos=100 e velocidade=10 anos/segundo, o tempo total será 10 segundos.
9     #
10    # A função retorna esse tempo total em segundos. Dessa forma, o código chamador pode converter,
Ln: 23, Col: 17

Run Share Command Line Arguments

Insira o ano de destino:
2887
Insira a velocidade da viagem (anos/segundo):
3
A distância temporal entre 2030 e 2887 é de 857 anos.
À velocidade escolhida, a jornada levará aproximadamente 285.67 segundos.
Isto equivale a cerca de 4.76 minutos, ou 0.08 horas.
Você está avançando para o futuro.

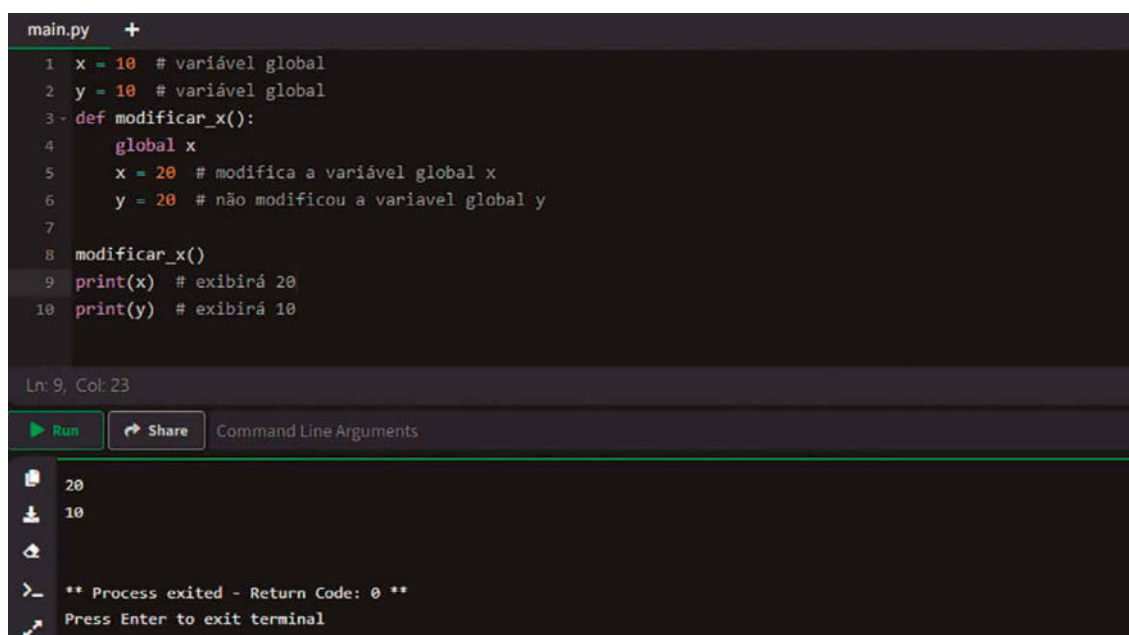
** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Figura 83 – Calculando o tempo da viagem. Uma viagem de 857 anos é percorrida em menos de 5 minutos

Essa **separação em camadas** – coleta de dados do usuário, cálculo na função e formatação da saída – reflete boas práticas de programação, tornando o código mais legível, flexível e fácil de manter. Caso o programador queira modificar a forma de apresentar o resultado, não precisa alterar a função. Caso deseje alterar a lógica do cálculo (por exemplo, incluir aceleração e desaceleração, transformando o cálculo em algo não linear), basta alterar a função, sem mexer na lógica de input e output.

Quando se declara uma variável dentro de uma função, por padrão, ela é considerada **local** àquela função. Por exemplo, no código anterior, cuja execução foi apresentada na figura anterior, a variável `tempo_em_segundos` é local. Isso significa que **seu escopo é limitado ao interior da função e, ao sair dela, essa variável deixa de existir**. Por outro lado, quando se declara uma variável fora de qualquer função, ela está no escopo **global** do módulo em que o código está sendo executado, **podendo ser acessada por qualquer parte do programa** que seja executada **após sua declaração**.

Se, dentro de uma função, você tentar modificar uma variável que foi definida fora dela (isto é, no escopo global), o Python, por padrão, **criará uma nova variável local com o mesmo nome em vez de alterar a global**. Para alterar o valor de uma variável global dentro de uma função, é necessário usar a palavra-chave `global` antes de atribuí-la. Para entender melhor, considere o exemplo da figura a seguir:



```
main.py +
1 x = 10 # variável global
2 y = 10 # variável global
3 def modificar_x():
4     global x
5     x = 20 # modifica a variável global x
6     y = 20 # não modificou a variável global y
7
8 modificar_x()
9 print(x) # exibirá 20
10 print(y) # exibirá 10

Ln: 9, Col: 23

Run Share Command Line Arguments

20
10

> ** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Figura 84 – O valor da variável global `x` é modificada dentro da função. O valor da variável global `y` não é alterado

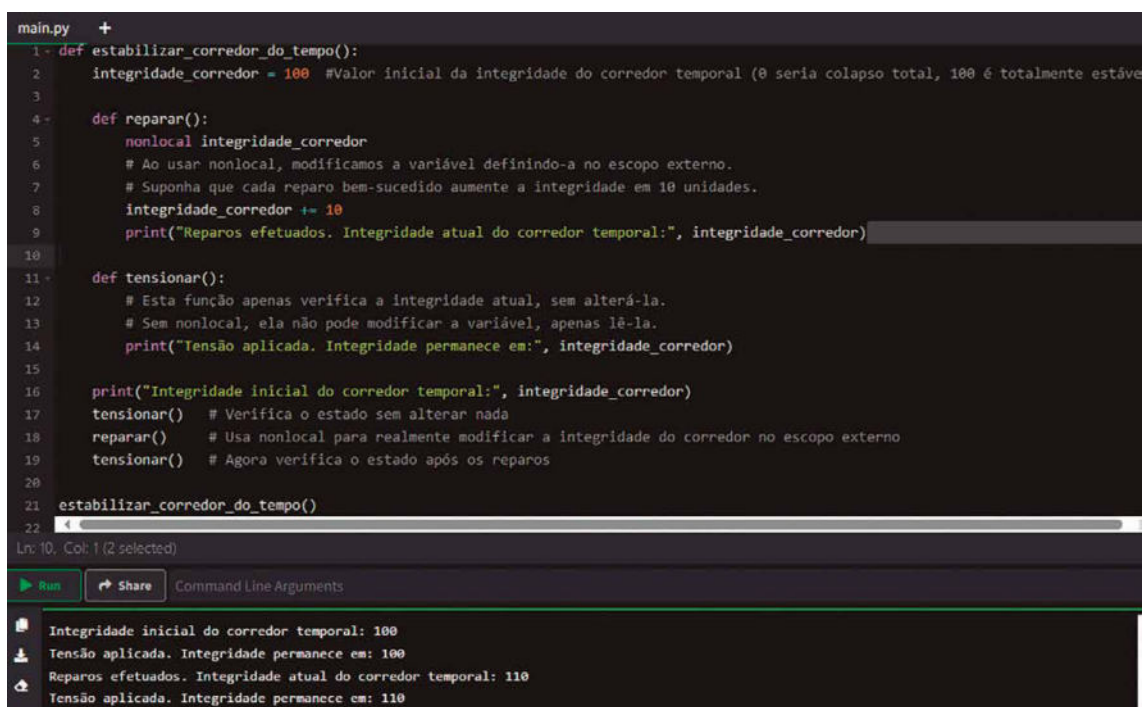
Caso não se utilizasse a declaração `global x`, a atribuição dentro da função criaria uma nova variável local `x`, sem modificar a `x` global; é o que aconteceu com a variável `y`.

Há também o caso de variáveis em escopos intermediários, como funções aninhadas (nested functions). Nesses casos, se você quiser modificar ou acessar uma variável definida no escopo externo, mas não global, pode utilizar a palavra-chave `nonlocal`. Essa palavra-chave permite que se acesse e modifique variáveis de um escopo não global, mas também não local, como o de uma função envolvente.

Para dar um exemplo, podemos pensar em um corredor temporal, um espaço abstrato que precisa ser estabilizado para permitir viagens seguras. A integridade desse corredor é representada por um valor numérico, e temos uma função interna que tenta realizar reparos aumentando esse valor.

Sem `nonlocal`, essa função criaria uma nova variável local, deixando a integridade original inalterada. Ao usar `nonlocal`, ela de fato altera a variável no escopo envolvente, refletindo o esforço de reparo no corredor.

No exemplo da figura a seguir, a função externa `estabilizar_corredor_do_tempo()` define a integridade inicial do corredor temporal. Duas funções internas, `reparar()` e `tensionar()`, lidam com essa integridade. A função `reparar()` usa `nonlocal` para incrementar o valor da integridade, simulando a ação de restaurar o corredor, enquanto `tensionar()` apenas acessa o valor, simulando um teste de estresse sem alterar sua integridade.



```
main.py +
1 def estabilizar_corredor_do_tempo():
2     integridade_corredor = 100 #Valor inicial da integridade do corredor temporal (0 seria colapso total, 100 é totalmente estável)
3
4     def reparar():
5         nonlocal integridade_corredor
6         # Ao usar nonlocal, modificamos a variável definindo-a no escopo externo.
7         # Suponha que cada reparo bem-sucedido aumente a integridade em 10 unidades.
8         integridade_corredor += 10
9         print("Reparos efetuados. Integridade atual do corredor temporal:", integridade_corredor)
10
11     def tensionar():
12         # Esta função apenas verifica a integridade atual, sem alterá-la.
13         # Sem nonlocal, ela não pode modificar a variável, apenas lê-la.
14         print("Tensão aplicada. Integridade permanece em:", integridade_corredor)
15
16     print("Integridade inicial do corredor temporal:", integridade_corredor)
17     tensionar() # Verifica o estado sem alterar nada
18     reparar()   # Usa nonlocal para realmente modificar a integridade do corredor no escopo externo
19     tensionar() # Agora verifica o estado após os reparos
20
21     estabilizar_corredor_do_tempo()
22
```

Ln: 10, Col: 1 (2 selected)

Run Share Command Line Arguments

Integridade inicial do corredor temporal: 100  
Tensão aplicada. Integridade permanece em: 100  
Reparos efetuados. Integridade atual do corredor temporal: 110  
Tensão aplicada. Integridade permanece em: 110

Figura 85 – Corredor temporal: tensionamento e reparo com a utilização de funções aninhadas e `nonlocal`

Em resumo, Python diferencia variáveis locais (declaradas dentro de funções, acessíveis apenas ali), variáveis globais (declaradas no nível do módulo, acessíveis em qualquer parte do módulo) e ainda oferece mecanismos (`global` e `nonlocal`) para controlar explicitamente quais variáveis em escopos externos devem ser modificadas.

Observe também que as funções `tensionar()` e `reparar()` **não podem ser usadas fora da função** `estabilizar_corredor_do_tempo()` porque elas são definidas dentro dela, o que limita seu escopo ao contexto interno no qual foram criadas. Esse comportamento é uma característica do Python que segue o modelo de escopos, conhecido como **LEGB** (Local, Enclosing, Global, Built-in), que define onde as variáveis e funções podem ser acessadas.



No caso dessas funções, elas estão no **escopo local** de `estabilizar_corredor_do_tempo()`. Em outras palavras, elas só existem enquanto a função principal está sendo executada, e sua visibilidade é restrita a esse contexto. Uma vez que a execução de `estabilizar_corredor_do_tempo()` termina, **as funções internas deixam de existir, porque o escopo local é descartado**. Por isso, tentar chamar `tensionar()` ou `reparar()` fora de `estabilizar_corredor_do_tempo()` resultaria em um erro, já que elas não são reconhecidas no escopo global ou em outro contexto.

Outra razão para essa restrição está relacionada à variável `integridade_corredor`, que também pertence ao escopo de `estabilizar_corredor_do_tempo()`. As funções internas dependem dessa variável para realizar suas operações. Lembre-se que no caso de `reparar()`, a palavra-chave `nonlocal` foi usada para modificar diretamente a variável do escopo mais externo, mas ainda no local a `estabilizar_corredor_do_tempo()`. Já `tensionar()` lê o valor da variável, mas também só pode fazer isso porque compartilha o mesmo contexto em que a variável foi definida. Se essas funções fossem chamadas fora de `estabilizar_corredor_do_tempo()`, a variável `integridade_corredor` simplesmente não existiria, tornando impossível seu funcionamento.

Definir funções dentro de outra função é uma prática útil para encapsular lógica que só faz sentido dentro de um contexto específico. No caso de `estabilizar_corredor_do_tempo()`, `tensionar()` e `reparar()` servem apenas como ferramentas auxiliares para gerenciar a integridade do corredor temporal. Esse design promove o encapsulamento e reduz o risco de conflitos ou acessos indesejados a essas funções de fora do contexto no qual elas têm relevância. Portanto, a impossibilidade de usar `tensionar()` ou `reparar()` fora de `estabilizar_corredor_do_tempo()` não é uma limitação arbitrária, mas uma escolha deliberada para manter o código organizado, seguro e fácil de entender.





## Resumo

Esta unidade abordou os fundamentos e avanços do uso de Python, destacando tipos de dados, manipulação de variáveis, operadores e a aplicação desses conceitos em diferentes contextos, como cálculos matemáticos, manipulação de texto e resolução de problemas complexos. Os tipos numéricos em Python, incluindo inteiros, números de ponto flutuante e números complexos, foram explicados detalhadamente, acentuando características como a precisão arbitrária para inteiros e a representação de números de ponto flutuante segundo o padrão IEEE 754.

As strings receberam atenção especial, com ênfase na manipulação textual através de métodos nativos que permitem operações como substituições, formatações e alterações de capitalização.

As variáveis em Python foram explicadas como referências a objetos na memória, destacando a tipagem dinâmica da linguagem, que permite a reutilização de nomes de variáveis para armazenar diferentes tipos de dados. A manipulação de objetos foi discutida em detalhes, incluindo a imutabilidade de strings e a mutabilidade de listas, bem como os mecanismos de gerenciamento de memória baseados em referência.

Esta unidade também aprofundou o tema do uso de operadores, que incluem aritméticos, relacionais, lógicos e bit a bit, exemplificando seu uso em cálculos complexos e controle de fluxo. O uso de estruturas condicionais, loops e operadores avançados foi ilustrado com programas que simulam cenários, como cálculos de intervalos temporais e manipulações no plano complexo.

Depois, abordamos de forma detalhada o papel fundamental das funções na organização e execução de tarefas em Python, ilustrando sua relevância na programação estruturada e funcional. A definição de funções utiliza a palavra-chave `def`, seguida por parâmetros opcionais e um corpo indentado contendo as instruções, permitindo encapsular lógica específica para ser reutilizada ao longo de um programa.

A passagem de parâmetros e o retorno de valores foram explicados como os principais mecanismos que tornam as funções flexíveis e adaptáveis. A abordagem distinguiu entre objetos mutáveis e imutáveis, mostrando como alterações dentro de uma função podem ou não impactar os objetos originais. Exemplos práticos incluem o retorno de múltiplos

valores utilizando tuplas, listas ou dicionários, demonstrando a capacidade de Python de tratar dados de forma estruturada e eficiente.

Por fim, vimos que a organização do código em etapas lógicas – preparação, execução e chegada – reflete boas práticas de engenharia de software.



## Exercícios

**Questão 1.** Considere o cenário de um aplicativo de gerenciamento financeiro pessoal, no qual o usuário pode registrar transações, calcular o saldo atual e visualizar um resumo das despesas por categoria. O código Python a seguir é uma parte simplificada desse aplicativo, que inclui uma função para adicionar uma nova transação e outra para calcular o saldo total:

```
def adicionar_transacao(transacoes, valor, categoria):
    transacoes.append({"valor": valor, "categoria": categoria})

def calcular_saldo(transacoes):
    saldo = 0
    for transacao in transacoes:
        saldo += transacao['valor']
    return saldo

# Lista inicial de transações
transacoes = []
adicionar_transacao(transacoes, 50, "Alimentação")
adicionar_transacao(transacoes, -20, "Transporte")
```

Figura 86

Baseado nesse cenário e no código fornecido, qual das seguintes alternativas é correta sobre a manipulação de tipos de dados e variáveis no Python?

- A) O método `append()` pode ser usado para adicionar novos elementos apenas ao final de listas, e não de dicionários.
- B) A função `calcular_saldo` retornará um erro se `transacoes` for uma lista vazia, pois não é possível iterar sobre uma lista vazia.
- C) A variável `saldo` na função `calcular_saldo` é inicialmente uma string vazia para garantir que a soma dos valores possa ser convertida em texto para exibição.
- D) As chaves do dicionário dentro da lista `transacoes` são definidas como valores numéricos para garantir cálculos precisos.
- E) Na função `adicionar_transacao`, `valor` e `categoria` são armazenados como itens de um dicionário, permitindo que sejam facilmente recuperados e manipulados posteriormente.

Resposta correta: alternativa E.

### Análise das alternativas

A) Alternativa incorreta.

Justificativa: afirma que `append()` adiciona elementos apenas ao final de listas, mas isso não implica erro no contexto do código, já que o uso é correto e não se refere a dicionários.

B) Alternativa incorreta.

Justificativa: a função `calcular_saldo` pode iterar sobre uma lista vazia sem causar erro, resultando apenas em um saldo de 0, o que é um comportamento válido e esperado.

C) Alternativa incorreta.

Justificativa: `saldo` é inicialmente definido como um inteiro (`saldo = 0`), e não como uma string. Isso é necessário para realizar a soma dos valores das transações.

D) Alternativa incorreta.

Justificativa: as chaves do dicionário (`valor` e `categoria`) são definidas como strings, que é a maneira correta de usar chaves em dicionários no Python, enquanto os valores associados podem ser de diferentes tipos, incluindo numéricos.

E) Alternativa correta.

Justificativa: ilustra adequadamente a funcionalidade da função `adicionar_transacao`, na qual `valor` (um número) e `categoria` (uma string) são armazenados em um dicionário, o que facilita a sua recuperação e a sua manipulação em funções subsequentes do programa.

**Questão 2.** A empresa alunosACME tem um sistema para gerenciamento de eventos no qual uma função em Python é utilizada para registrar participantes e outra para verificar a capacidade total de um evento. A seguir, temos um exemplo simplificado do código para esse sistema.

```
capacidade_maxima = 100

def registrar_participante(lista_participantes, nome):
    if len(lista_participantes) < capacidade_maxima:
        lista_participantes.append(nome)
        return True
    else:
        return False

def verificar_capacidade():
    def capacidade_atual():
        return len(lista_participantes)
    return capacidade_maxima - capacidade_atual()
```

```
lista_participantes = []  
  
# Exemplo de uso das funções  
registrar_participante(lista_participantes, "João")  
registrar_participante(lista_participantes, "Maria")  
print(verificar_capacidade())
```

Figura 87

Com base nesse cenário e no código fornecido, qual das seguintes alternativas é correta sobre o uso de funções e escopo de variáveis em Python?

- A) A função `capacidade_atual` dentro de `verificar_capacidade` é um exemplo de função aninhada, que acessa a variável `lista_participantes` do escopo global.
- B) A variável `capacidade_maxima` deve ser passada como parâmetro para `registrar_participante` para evitar referência ao escopo global.
- C) O retorno de `False` na função `registrar_participante` indica que a função não altera a lista de participantes.
- D) A função `verificar_capacidade` demonstra o uso incorreto de função aninhada, pois `capacidade_atual` deveria estar definida no escopo global para acessar `lista_participantes`.
- E) A capacidade de `lista_participantes` não é modificada pelas funções chamadas, mantendo a lista sempre vazia.

Resposta correta: alternativa A.

### Análise das alternativas

A) Alternativa correta.

Justificativa: a função `capacidade_atual` está definida dentro de outra função e faz uso da variável `lista_participantes` que, apesar de estar manipulada dentro da função `registrar_participante`, é originalmente definida no escopo global do script. Esse é um bom exemplo de como funções aninhadas podem acessar variáveis de escopos superiores.

B) Alternativa incorreta.

Justificativa: embora seja uma boa prática evitar dependências do escopo global, no caso específico, o código não exige que `capacidade_maxima` seja passada como parâmetro; a função pode operar corretamente com a variável global, dado o contexto simplificado do exemplo.

